A Survey of Zero-Knowledge Succinct Non-Interactive Arguments of Knowledge

Sunkwan Kim

kim.sunk@northeastern.edu
Northeastern University

Abstract. Zero-knowledge succinct non-interactive arguments of knowledge (zk-SNARKs) allow a verifier to check a computation with a proof far shorter than the computation, without learning the prover's private inputs. Over the last decade they have moved from theory to deployed systems, and the number of constructions has grown quickly. This survey organizes the main construction families by their setup model: circuit-specific trusted setup (GGPR, Pinocchio, Groth16), universal and updatable setup (Sonic, PLONK, Marlin), and transparent setup (Fractal, the random-oracle variant of Spartan, and Supersonic with class groups), with recursion and folding (Halo, Nova) built on top. We give a single formal definition with a strict succinctness requirement, explain the modular pipeline behind most schemes, and compare the schemes in normalized tables of asymptotic costs and setup parameters, together with their assumptions and the exact counts the papers report, with a source for every entry. Schemes that are often called zk-SNARKs but miss the strict bound are compared separately. A further section draws together the developments since 2022: small and binary fields with challenges from extension fields, lookup-centric and STARK-based zkVMs, folding in place of recursive verification, the gap between proven and conjectured security of hash-based commitments, and lattice-based folding. We conclude with open problems, including linear-time provers, post-quantum security and attacks on the Fiat–Shamir transform.

Keywords: zk-SNARKs · Zero-Knowledge Proofs · Polynomial Commitment Schemes · Interactive Oracle Proofs · Sum-Check Protocol · GKR Protocol · Arithmetization · Small and Binary Fields · Fiat–Shamir Transform · Trusted Setup · Recursion and Folding · Lookup Arguments · zkVMs · Proximity Gaps · Post-Quantum Security · Lattice-Based Cryptography

Contents

1 Introduction **3**

2 Preliminaries **5**
2.1 Notation and Mathematical Background 5
2.2 Interactive Proofs and Arguments 7
2.3 Cryptographic Tools 7

3 Defining a zk-SNARK **9**
3.1 Why Knowledge Assumptions and Random Oracles? 11

4 Anatomy of a Modern zk-SNARK **14**
4.1 Arithmetization 14
4.2 Information-Theoretic Protocols: PCPs, IOPs and Polynomial IOPs . . . 17
4.3 Lookup Arguments 20
4.4 Polynomial Commitment Schemes 22
4.5 Compiling to a Non-interactive Argument 26

5 Construction Families of zk-SNARKs **28**
5.1 Pairing-based / QAP — GGPR, Pinocchio, Groth16 28
5.2 Universal and Updatable Setup — Sonic, PLONK, Marlin 30
5.3 Transparent — Fractal, Spartan, Supersonic (DARK) 32
5.4 Zero-Knowledge Arguments Outside the Strict Definition 35

6 Recursion, Accumulation and Folding **35**
6.1 Incrementally verifiable computation 35
6.2 Halo and accumulation schemes 36
6.3 Folding: Nova 36
6.4 Generalizations 37

7 Comparison **38**

8 Developments since 2022 **45**

9 Open problems and future directions **47**
9.1 Prover efficiency 47
9.2 Post-quantum zk-SNARKs 48
9.3 Recursion overhead and the security of folding 49
9.4 Fiat–Shamir in practice 50
9.5 Foundations: SNARGs from standard assumptions 50

10 Conclusion **50**

A Supplementary Tables and Timeline **64**

1 Introduction

Zero knowledge began as a theoretical notion: Goldwasser, Micali and Rackoff asked how much knowledge a proof must communicate beyond the validity of the statement, and they defined proofs that communicate none [GMR85]. About three decades later, short non-interactive proofs underlie deployed private-payment systems and blockchain scaling, the former relying on zero knowledge. This survey is about the combination of all three properties, the zero-knowledge succinct non-interactive argument of knowledge (zk-SNARK).

Why succinct, non-interactive and zero-knowledge. The original motivation is verifiable computation: a computationally weak client outsources a computation to a powerful but untrusted server and checks a short proof instead of redoing the work [Tha22, Ch. 1, pp. 119–120; Nit20, §1.2, p. 4]. Each property in the name serves this goal. *Succinctness* makes checking much cheaper than computing. Kilian gave the first argument whose communication can be smaller than the NP witness [Kil92, abstract, p. 723], and in a zk-SNARK both the proof and the verifier's work are polylogarithmic in the size of the computation, apart from the time to read the public input (§3). *Non-interactivity* means a proof is a single message that anyone can check at any time. A proof can therefore be stored and checked later by anyone, for instance on a blockchain, where proofs may be stored permanently [Tha22, §7.2, p. 294], and it can be verified inside another proof, which is the basis of recursive composition [Val08] (§6). *Knowledge soundness* guarantees that a prover who convinces the verifier actually knows a witness, which matters for statements that are always true, such as knowing a preimage of a public hash [Tha22, §7.4, pp. 305–306] (§2.2). *Zero knowledge* lets the witness stay private, as in anonymous payments, where the proof shows that a transaction is valid and spends no coin twice while the payment stays anonymous [BCG+14, abstract, §1.3, p. 6; Nit20, §7, p. 23].

Applications. zk-SNARKs are used wherever a verifier must check a computation it cannot or will not redo, or a statement about data it should not see. Pinocchio already supported *public* verifiable computation, in which anyone can check the worker's proof against a published verification key [PHGR13, abstract]. On a blockchain, a single SNARK can replace a batch of transaction signatures that every node would otherwise store and verify [AST24, §1, p. 1]. Deployed ZK-rollups such as zkSync Era, Polygon zkEVM and Scroll are among the systems Chaliasos et al. study [CET+24, §4, p. 7]. They note that zk-EVMs use SNARKs "without the zero-knowledge property, solely relying on non-interactivity and succinctness", and that composition sets the trust model: Polygon zkEVM proves the verifier of a FRI-based SNARK inside Groth16, so "the trust assumption reduces to the weakest component, i.e., Groth16 still requires a trusted setup" [CET+24, §3.1, p. 5]. A zkVM is "a SNARK that allows the witness-checking procedure to be specified as a computer program written in the assembly language of a specific instruction set architecture" [AST24, abstract]. Zero knowledge, by contrast, is the point of private payments and of credentials, where the verifier "learns that the worker holds a valid credential, but learns nothing about the credential itself" [PHGR13, §2.1, p. 2]. Deployment has also exposed failures that a security proof does not cover. Of the 141 vulnerabilities Chaliasos et al. collected, 99 are in circuits and 124 break soundness [CET+24, Tables 2–4], and a SNARK variant deployed in Zcash had extra group elements in its SRS that would have permitted unlimited counterfeiting [Tha22, §17.5, fn. 15, p. 573; CET+24, App. A.1, p. 20].

From theory to practice. Micali's CS proofs made Kilian's argument non-interactive with a random oracle [Mic94], but PCP-based constructions remained far from practical. Gennaro, Gentry, Parno and Raykova replaced PCPs with quadratic span and arithmetic

programs [GGPR13], and Pinocchio built on them a general-purpose system whose verification was cheaper than native execution for some applications [PHGR13] (§5.1). Groth's 2016 scheme reduced a proof to three group elements, 128 bytes on the BN-256 curve [Gro16; CHM+20, Fig. 1]. Since then, the field has diversified along several axes: reusable universal setups [GKM+18; MBKM19; GWC19; CHM+20], transparent setups with no trapdoor [BBHR18; COS20; Set20; BFS20], recursion and folding [BGH19; BCMS20; KST22], and faster provers (§9.1). The constructions now number in the dozens. Liang et al., for example, classify over 40 zk-SNARKs published since 2013 [LHW+25, abstract].

The problem this survey addresses. With this growth, the term "zk-SNARK" has come to cover schemes with very different guarantees. The papers do not agree on what "succinct" means. Hyrax asks for proofs "sub-linear in the size of the statement and the witness" [WTS+18, §1, p. 1], Aurora calls a scheme with a linear-time verifier "a zkSNARK" [BCR+19, abstract], and Binius classifies as SNARKs protocols "whose verifiers scale merely sublinearly, as opposed to strictly polylogarithmically" [DP25, §1, p. 1]. Setup models are often conflated: "preprocessing" is used both for circuit-specific trusted setups and for deterministic indexing over a universal one [CHM+20, §1, p. 3]. Reported costs use incompatible units, curves, security levels and benchmarks (§7). A reader comparing schemes therefore has to translate each paper into a common language first.

Contributions. This survey does that translation for the main families.

- **One definition** (§3). We define zk-SNARKs for arithmetic circuits with a two-phase setup, a global setup $\mathcal{S}_1$ and a circuit setup $\mathcal{S}_2$. Circuit-specific, universal and transparent schemes are then instances of one syntax (Remark 1, Table 3.1). We adopt strict succinctness, polylogarithmic in $|C|$ for the proof and for the verifier (which may also read the statement x), and use it as a classification criterion, together with a rule for when a scheme counts as zero-knowledge. We also explain why every scheme in this survey relies on a knowledge-type assumption, an idealized group model or a random oracle (§3.1).
- **A pipeline view** (§4). We trace how arithmetization, an information-theoretic protocol, a polynomial commitment and the Fiat–Shamir transform fill each part of the definition, and where trust enters; lookup arguments, which run alongside the constraint system, have their own subsection (§4.3).
- **A taxonomy** (§5). We organize the families by setup model, and treat separately the zero-knowledge arguments that miss the strict bound, each with the reason it misses (§5.4).
- **A normalized, sourced comparison** (§7). We express every scheme's setup parameters, proof size, prover and verifier as $O_\lambda(\cdot)$ bounds, which hide $\mathrm{poly}(\lambda)$ factors, in $|C|$ wherever the sources allow, and say why when a bound also depends on depth, input or witness size. We cite a source for every cell.
- **A map of recent developments** (§8). We organize the work since 2022 around five questions: why fields have become small and what that costs, how zkVMs are built, why recent IVC schemes fold instead of verifying recursively, how much of the security of hash-based commitments is proven, and which post-quantum options are not hash-based. Each answer connects back to the components of §§4–6 and to Figures 2, 5 and 6, and Table 8.1 lists the fields in use.
- **Open problems** (§9), including attacks on the Fiat–Shamir transform, recursion that is proven secure with the concrete hash it uses, and a lattice-based zk-SNARK that is built and analyzed end to end, zero knowledge included.

Related surveys. Thaler's monograph gives a unified, accessible treatment of the main ideas and approaches behind interactive proofs, arguments and zero-knowledge protocols [Tha22, Ch. 1, p. 121]. Nitulescu's gentle introduction [Nit20] follows the evolution of the soundness notion from interactive proofs to arguments of knowledge and presents the modular frameworks behind state-of-the-art SNARKs. Liang et al. [LHW+25] systematize zk-SNARKs from theory to practice: they classify over 40 schemes and study 11 libraries. Sheybani et al. [SAKK25] survey and benchmark open-source frameworks, and Chaliasos et al. [CET+24] systematize 141 real-world implementation vulnerabilities. Our survey is narrower and more definitional than these. It applies one strict definition throughout, separates the schemes that meet it from those that do not, and normalizes the comparison to the circuit size.

Scope. We treat zk-SNARKs at the level of their constraint systems, information-theoretic protocols, commitments and compilers. Several neighbouring topics are out of scope. We do not cover front-end compilers that turn programs into circuits (§4.1 points to [Tha22, Ch. 6]), the engineering of zkVMs beyond their two main designs (§8), or hardware acceleration of provers. Trusted-setup ceremonies appear only as a component of the setup model (§5.1), not as a subject of their own. Σ-protocols and other classical zero-knowledge proofs appear only as background (§2.2); arguments that miss only the strict succinctness bound are treated in §5.4. For libraries and implementation-level comparisons we refer to the systematizations of Liang et al. [LHW+25] and Sheybani et al. [SAKK25].

2 Preliminaries

This section fixes notation and recalls only the background that §§3–5 use. For standard definitions we refer to Thaler's monograph [Tha22] and to Boneh and Shoup [BS23]. Non-interactive definitions are deferred to §3.

2.1 Notation and Mathematical Background

Notation

The table lists the symbols used across sections. Symbols used within one section are defined where they appear.

Table 2.1: **Notation.**

Symbol	Meaning	Introduced in
λ	security parameter	§3
$\mathrm{negl}(\lambda)$	a negligible function of λ	§3
$\mathrm{poly}(\cdot)$	some fixed polynomial in the listed quantities	§3
$O_\lambda(\cdot)$	big-O that hides $\mathrm{poly}(\lambda)$ factors	§1, §7
$\mathbb{F}$; $\mathbb{K}$	field of the circuit; extension field for challenges in small-field systems	§2.1; §8
C, $\lvert C\rvert$	public arithmetic circuit; its number of gates	§3
N	largest $\lvert C\rvert$ the global setup supports	§3
x, $\lvert x\rvert$; w	public statement and its size; private witness	§3
$\mathcal{R}_C$	relation $\{(x, w) : C(x, w) = 0\}$	§3
π	proof (the single prover message)	§3

continued on next page

Table 2.1, continued

Symbol	Meaning	Introduced in
$\mathcal{S}_1$, gp; $\mathcal{S}_2$	global setup and the global parameters; circuit setup	§3
$\mathsf{S_p}$, $\mathsf{S_v}$	prover's and verifier's parameters for one circuit	§3
$\mathcal{P}$, $\mathcal{V}$; $\tilde{\mathcal{P}}$, $\mathcal{A}$	prover and verifier; cheating prover, adversary	§2.2, §3
$\mathcal{E}$; Sim	knowledge extractor; zero-knowledge simulator	§2.2, §3
ρ	random oracle $\{0,1\}^* \to \{0,1\}^\lambda$	§2.3, §3
τ	secret evaluation point of a structured reference string	§4.4, §5.1
$\mathrm{RS}[\mathbb{F}, \mathcal{L}, d]$	Reed–Solomon code on domain $\mathcal{L}$, degree $< d$	§2.1
R; δ	code rate $d/\lvert\mathcal{L}\rvert$; proximity parameter	§2.1
d	degree bound; depth of a layered circuit	§2.1; §4.2

Fields and Polynomial Identities

Circuits are defined over a finite field $\mathbb{F}$, either a prime field $\mathbb{F}_p$ or an extension field $\mathbb{F}_{p^w}$. Three constraints shape its choice [Tha22, §19.3.1, pp. 619–621]. Soundness errors of at least $1/|\mathbb{F}|$ force a large field. Schemes over discrete-logarithm or pairing groups must use the group's scalar field, a 256-bit prime field for bn256 and bls381 [Tha22, p. 621; BS23, p. 626]. Provers that run FFTs (§4) need an *FFT-friendly* field, with $2^\ell \mid p-1$ for large ℓ. Hash-based schemes escape the second constraint and can use binary fields [Tha22, p. 621], such as STARK's $\mathbb{F}_{2^{64}}$ [BBHR18, §1.3.2, p. 11] and the towers of Binius [DP25, §2.3]; §8 discusses small and binary fields. Most soundness proofs below rest on the Schwartz–Zippel lemma [Sch80], [Zip79]: a nonzero polynomial $g : \mathbb{F}^m \to \mathbb{F}$ of total degree at most d vanishes at a uniform point of S^m, for a finite $S \subseteq \mathbb{F}$, with probability at most $d/|S|$ [Tha22, Lemma 3.3, p. 157]. A verifier can therefore test a claimed polynomial identity at one random point instead of everywhere.

Reed–Solomon Codes and Proximity

Let $\mathcal{L} \subseteq \mathbb{F}$ be an evaluation domain, usually a multiplicative subgroup or coset of power-of-2 size. The Reed–Solomon code $\mathrm{RS}[\mathbb{F}, \mathcal{L}, d]$ consists of the evaluations on $\mathcal{L}$ of polynomials of degree less than d. Its *rate* is $R = d/|\mathcal{L}|$ [ACFY24, §1, p. 1]; the coding literature writes ρ, which this survey reserves for the random oracle. Distinct polynomials of degree less than d agree on fewer than d points [Tha22, §10.5.2, p. 388], so two codewords differ on more than a $1-R$ fraction of $\mathcal{L}$. A function is δ-*far* from the code if it differs from every codeword on more than a δ fraction of $\mathcal{L}$, and δ-*close* otherwise. A *proximity test* accepts codewords and rejects δ-far functions. Its verifier reads only a few positions, possibly with a prover's help [BBHR18b, §1, p. 14:2].

Three radii matter. Below the *unique-decoding radius* $(1-R)/2$, at most one codeword is δ-close to a function [Tha22, §10.4.2, p. 374]. Up to the *Johnson radius* $1-\sqrt{R}$, list decoding (finding all δ-close codewords) is feasible [CS26, §1.1, p. 131]. Interpolation makes every function $(1-R)$-close, so *capacity* $1-R$ is the limit [CS26, §1, p. 129]. A *proximity gap* says that a line $\{f_0 + cf_1 : c \in \mathbb{F}\}$ is either almost entirely δ-close to the code or almost entirely δ-far [CS26, §1, p. 128]. *Correlated agreement* adds that if more than a small fraction of the line is δ-close, then f_0 and f_1 agree with codewords on one common set of $(1-\delta)|\mathcal{L}|$ points [ACFY25, §1.2, p. 8]. Testing one random combination then covers both functions. Correlated agreement is proven up to the Johnson radius and was conjectured up to capacity [ACFY25, §1.2, p. 8]; §5.3 says what survives.

2.2 Interactive Proofs and Arguments

We use the standard notions of interactive proofs, arguments, zero knowledge and knowledge soundness [Tha22, Defs. 3.1–3.2, pp. 149–151, Def. 11.1, pp. 398–399, §7.4, pp. 305–307] and recall only the points the later sections depend on.

Proofs, arguments and public coins. A prover $\mathcal{P}$ holding (x, w) and a probabilistic polynomial-time verifier $\mathcal{V}$ holding x exchange messages until $\mathcal{V}$ accepts or rejects, written $\langle \mathcal{P}(w), \mathcal{V} \rangle(x)$ [GMR85, §2, p. 293; Tha22, §3.1, pp. 148–149]; §3 requires perfect completeness. In a *public-coin* protocol the verifier's messages are just its random coins [Tha22, §3.3, p. 153]. Only such protocols admit the Fiat–Shamir transform (§2.3). Sum-check and GKR (§4) are public coin. The interactive argument behind GGPR, Pinocchio and Groth16 is not, since its verifier holds a secret key, so these schemes are made non-interactive with a structured reference string instead [Tha22, §17.5.1, p. 566] (§4.5). An *argument*, due to Brassard, Chaum and Crépeau [BCC88], requires soundness only against polynomial-time provers [Tha22, Def. 3.2, pp. 150–151], and succinctness essentially requires one: an interactive proof for circuit satisfiability whose communication is logarithmic in the witness length would imply coNP $\subseteq$ AM [Tha22, §7.2, fn. 2, p. 295]. Kilian [Kil92] has the prover commit to a probabilistically checkable proof (§4.2) with a Merkle tree (§2.3) and open only the queried positions, "the first construction in which the total amount of communication can be less than that needed to transmit the NP witness" [Kil92, abstract, p. 723]. Micali's CS proofs remove the interaction with a random oracle [Mic94, p. 440]. Every scheme in this survey is an argument.

Zero knowledge and extraction. Zero knowledge is defined by simulation [GMR85, §3.2, p. 296]: an efficient simulator Sim that never sees w must produce a view indistinguishable from the verifier's view of a real interaction, perfectly, statistically or computationally [Tha22, Def. 11.1, pp. 398–399]. *Honest-verifier* zero knowledge, which asks this only for the honest $\mathcal{V}$, suffices before Fiat–Shamir, because there the simulator can program the hash [Tha22, pp. 399–400, p. 402, fn. 3]. Soundness alone can be vacuous for statements that are always true, such as "y has a preimage" under a surjective h [Tha22, §7.4, pp. 305–306]. A *proof or argument of knowledge* therefore requires an efficient extractor $\mathcal{E}$ that, given repeated access to any convincing prover $\tilde{\mathcal{P}}$, outputs a valid witness [Tha22, p. 306]. Interactive extractors usually rewind the prover, as the special soundness of Σ-protocols does [Tha22, §12.2.1, p. 418], which a real verifier cannot do [Tha22, fn. 13, pp. 306–307]. Properties 2 and 4 of §3 are the non-interactive versions, and §3.1 explains what knowledge soundness costs in assumptions.

2.3 Cryptographic Tools

Commitments

A commitment $c = \mathsf{Com}(m; z)$, computed with randomness z, is *binding* if no one can open it to two different messages and *hiding* if it reveals nothing about m. Each property holds statistically or computationally, and no scheme is statistically both [Tha22, §12.3, pp. 429–430; BS23, §8.12, Def. 8.8, p. 344]. Binding is what turns an oracle protocol into an argument, and hiding is what zero knowledge needs; §4 extends the notion to polynomials. Two commitments recur. A *Merkle tree* [Mer87] commits to a vector with a collision-resistant hash h; opening an entry reveals it with the $O(\log n)$ sibling hashes on its path to the root [BS23, §8.9, pp. 320–321; Tha22, §7.3.2.2, pp. 299–300]. The root is binding but not hiding, so zero knowledge needs hiding leaves, such as $h(s_i, o_i)$ with random o_i [BS23, §8.12, p. 346]. Because it needs only a hash and no setup, it is the commitment layer of Kilian's and Micali's arguments and of the hash-based schemes (§5.3,

§5.4). A *Pedersen commitment* [Ped91] to $m \in \mathbb{Z}_q$ is $g^m h^z$ for random generators g, h of a group of prime order q. It is perfectly hiding, binding under the discrete-logarithm assumption, transparent because no one knows $\log_g h$, and additively homomorphic, as is its vector form $h^z \prod_i g_i^{u_i}$ [Tha22, §12.3, pp. 432–434, §14.2, p. 470; BS23, §10.6.1, Thm. 10.4, p. 415]. It underlies Bulletproofs, Hyrax and Nova (Tables A.1–A.2).

Bilinear Pairings

A *pairing* is an efficiently computable map $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ between cyclic groups of prime order q. It is *bilinear*, $e(u^a, v^b) = e(u, v)^{ab}$, and *non-degenerate*: $e(g_1, g_2)$ generates $\mathbb{G}_T$ for generators g_1, g_2 [BS23, Def. 15.2, p. 625; Tha22, §15.1, p. 498]. It is *symmetric* if $\mathbb{G}_1 = \mathbb{G}_2$. In Type III pairings no efficient homomorphism exists between $\mathbb{G}_1$ and $\mathbb{G}_2$ in either direction, and Groth calls them "the most efficient type" [Gro16, §2.1, p. 6]. On pairing-friendly curves $\mathbb{G}_T \subseteq \mathbb{F}_{p^k}^*$ for the *embedding degree* k, which is 12 for bn256 and bls381 [BS23, p. 626]. Encodings g^m can be added homomorphically but not multiplied, and a pairing supplies exactly one multiplication: $e(g_1^a, g_2^b) = e(g_1^c, g_2)$ holds exactly when $c = ab$ [Tha22, p. 500]. A verifier can therefore check quadratic relations among hidden values. KZG uses this to check the evaluation identity $f(X) - f(z) = \psi(X)\,(X - z)$ at a secret point [KZG10, §3.2], and linear-PCP SNARKs such as Groth16 use it to check their verification equations [Tha22, §17.5.1, p. 566] (§4, §5.1).

Random Oracle Model

In the random oracle model [BR93] every party, honest or adversarial, may query a uniformly random function ρ, and security is bounded in terms of the number of queries [BS23, §8.10.2, pp. 327–329; Tha22, §5.1, pp. 236–237]. Deployed schemes replace ρ with a concrete hash function such as SHA-3, so for them a ROM proof is only a heuristic [Tha22, p. 237]. The heuristic can fail. Canetti, Goldreich and Halevi construct schemes that are secure in the ROM but insecure under every instantiation [CGH04], although Thaler calls such examples "typically contrived" [Tha22, fn. 2, p. 237]. Remark 2 of §3 fixes how ρ enters the definitions, and §3.1 covers its roles and its price [KRS25].

Fiat–Shamir Transform

Take a public-coin protocol with prover messages $a_1, \ldots, a_{r+1}$ and verifier challenges $c_1, \ldots, c_r$. The Fiat–Shamir transform [FS86] replaces each challenge by an oracle output,

$$c_i = \rho\big(\mathsf{gp}, \mathsf{S}_\mathsf{v}, x, a_1, \ldots, a_i\big), \qquad i = 1, \ldots, r.$$

The proof $\pi = (a_1, \ldots, a_{r+1})$ is a single message, and the verifier checks it by recomputing the challenges [Tha22, §5.2, p. 240; DMWG23, Def. 3, p. 5]. This is *strong* Fiat–Shamir. *Weak* Fiat–Shamir omits the public parameters and the statement from the hash [DMWG23, Def. 3]. A prover can then pick a false x after fixing the transcript, so hashing x is needed for adaptive soundness [Tha22, p. 241]; §3.1 describes the vulnerabilities this has caused in practice.

The result is an argument in the ROM, and it inherits the assumptions of the interactive protocol [Tha22, §5.3, fn. 6, p. 242]. For constant-round protocols with negligible soundness error it is sound against polynomial-time provers [Tha22, p. 242], as Thaler proves for 3 messages [Tha22, Thm. 5.1, p. 247]. The generic loss, though, is exponential in the number of rounds: a non-interactive prover that succeeds with probability ε in time T yields an interactive one that succeeds with probability at least $(\varepsilon/T)^{O(r)}$ for r rounds, because the non-interactive prover can retry messages offline until it gets favourable challenges [Tha22, pp. 242, 244]. For many-round protocols such as sum-check, the condition that suffices is *round-by-round soundness* [CCH+19]. A false statement starts in a "doomed" state, and

no prover message escapes it except with negligible probability. Sum-check-based protocols have this property, while sequential repetition of a protocol does not [Tha22, pp. 242–243, fn. 7].

3 Defining a zk-SNARK

In this section, we define the zero-knowledge Succinct Non-interactive ARgument of Knowledge that covers the main families of schemes.

Setting. Fix a finite field $\mathbb{F}$. An arithmetic circuit C over $\mathbb{F}$ defines the relation $\mathcal{R}_C = \{(x, w) : C(x, w) = 0\}$, where x is the public statement and w the private witness. Here $C(x, w) = 0$ means that every output wire of C evaluates to 0 on input (x, w), i.e. the circuit is satisfied (some papers write $C(x, w) = 1$ for the same condition). We write $|C|$ for the number of gates of C, λ for the security parameter, and $\mathrm{negl}(\lambda)$ for a negligible function. Adversaries are stateful: an adversary called more than once in an experiment keeps its state between calls.

Definition 1 (zero-knowledge Succinct Non-interactive ARgument of Knowledge)**.** (Adapted from [CHM+20] for the two-phase setup, i.e. a generator plus an indexer, and the adaptive choice of the circuit; [Gro16] for the extractor that receives the prover's random coins, the simulation trapdoor, and the term *fully succinct*; [GKM+18] for zero knowledge against many adaptively chosen statements; and [BCCT12] for the notion of a SNARK and a succinctness bound that grows only polylogarithmically with the size of the computation.) A zk-SNARK for arithmetic circuits is a tuple of efficient algorithms $(\mathcal{S}_1, \mathcal{S}_2, \mathcal{P}, \mathcal{V})$:

- $\mathcal{S}_1(1^\lambda, N) \to \mathsf{gp}$, the *global setup*: it runs once and outputs public parameters gp for all circuits of size at most N;
- $\mathcal{S}_2(\mathsf{gp}, C) \to (\mathsf{S_p}, \mathsf{S_v})$, the *circuit setup*: it preprocesses a circuit C with $|C| \le N$ and outputs public parameters $\mathsf{S_p}$ for the prover and $\mathsf{S_v}$ for the verifier;
- $\mathcal{P}(\mathsf{S_p}, x, w) \to \pi$, the *prover*: it outputs a single proof message π;
- $\mathcal{V}(\mathsf{S_v}, x, \pi) \to \{0, 1\}$, the *verifier*: it accepts or rejects,

satisfying the following properties for every size bound N. In each experiment the circuit C is chosen after gp is seen, and we only consider adversaries that always output a circuit with $|C| \le N$ (as in [CHM+20, §7], where the relation is restricted to circuits of size at most N).

1. Completeness. For every efficient adversary $\mathcal{A}$,

$$\Pr\left[\begin{array}{c|c} (x,w) \notin \mathcal{R}_C \\ \vee \\ \mathcal{V}(\mathsf{S_v}, x, \pi) = 1 \end{array} \middle| \begin{array}{c} \mathsf{gp} \leftarrow \mathcal{S}_1(1^\lambda, N) \\ (C, x, w) \leftarrow \mathcal{A}(\mathsf{gp}) \\ (\mathsf{S_p}, \mathsf{S_v}) \leftarrow \mathcal{S}_2(\mathsf{gp}, C) \\ \pi \leftarrow \mathcal{P}(\mathsf{S_p}, x, w) \end{array}\right] = 1.$$

2. Knowledge soundness. For every efficient adversary $\tilde{\mathcal{P}}$ there exists an efficient extractor $\mathcal{E}$ such that

$$\Pr\left[\begin{array}{c|c} (x,w) \notin \mathcal{R}_C \\ \wedge \\ \mathcal{V}(\mathsf{S_v}, x, \pi) = 1 \end{array} \middle| \begin{array}{c} \mathsf{gp} \leftarrow \mathcal{S}_1(1^\lambda, N) \\ C \leftarrow \tilde{\mathcal{P}}(\mathsf{gp}; r) \\ (\mathsf{S_p}, \mathsf{S_v}) \leftarrow \mathcal{S}_2(\mathsf{gp}, C) \\ (x, \pi) \leftarrow \tilde{\mathcal{P}}(\mathsf{S_p}, \mathsf{S_v}) \\ w \leftarrow \mathcal{E}(\mathsf{gp}, \mathsf{S_p}, \mathsf{S_v}; r) \end{array}\right] \le \mathrm{negl}(\lambda),$$

where r is $\tilde{\mathcal{P}}$'s random coins, which the extractor also receives. Both the circuit and the statement are chosen adaptively: C after gp, and x after $(\mathsf{S_p}, \mathsf{S_v})$.

3. Succinctness. The proof size and the verifier's running time are polylogarithmic in the circuit size, for fixed polynomials that do not depend on C:

$$|\pi| = \mathrm{poly}(\lambda, \log|C|), \qquad \mathrm{time}\big(\mathcal{V}(\mathsf{S_v}, x, \pi)\big) = \mathrm{poly}(\lambda, |x|, \log|C|).$$

In particular $|\mathsf{S_v}| = \mathrm{poly}(\lambda, |x|, \log|C|)$, because $\mathcal{V}$ reads it. The setups may take time $\mathrm{poly}(\lambda, N)$ and $\mathrm{poly}(\lambda, |C|)$, since they run once (*preprocessing*); for the latter bound $\mathcal{S}_2$ has oracle access to gp, as Marlin's indexer has to the SRS [CHM+20, §7]. Adapting the term of [Gro16, §2.2], which requires the whole reference string to have size $\mathrm{poly}(\lambda)$, we call the scheme *fully succinct* if all parameters generated with fresh randomness have size $\mathrm{poly}(\lambda)$: $|\mathsf{gp}| = \mathrm{poly}(\lambda)$ and $\mathcal{S}_2$ is deterministic. Halo uses the term for the verifier instead, and calls itself not fully succinct because its verifier's work is linear in the circuit size (§6.2).

4. Zero knowledge. There is an efficient simulator $\mathsf{Sim} = (\mathsf{Sim}.\mathcal{S}_1, \mathsf{Sim}.\mathcal{S}_2, \mathsf{Sim}.\mathcal{P})$ such that for every efficient adversary $\mathcal{A}$,

$$\left| \Pr\left[\mathcal{A}^{\mathcal{O}_{\mathcal{P}}}(\mathsf{S_p}, \mathsf{S_v}) = 1 \;\middle|\; \begin{array}{c} \mathsf{gp} \leftarrow \mathcal{S}_1(1^\lambda, N) \\ C \leftarrow \mathcal{A}(\mathsf{gp}) \\ (\mathsf{S_p}, \mathsf{S_v}) \leftarrow \mathcal{S}_2(\mathsf{gp}, C) \end{array} \right] - \Pr\left[\mathcal{A}^{\mathcal{O}_{\mathsf{Sim}}}(\mathsf{S_p}, \mathsf{S_v}) = 1 \;\middle|\; \begin{array}{c} (\mathsf{gp}, \mathsf{td}) \leftarrow \mathsf{Sim}.\mathcal{S}_1(1^\lambda, N) \\ C \leftarrow \mathcal{A}(\mathsf{gp}) \\ (\mathsf{S_p}, \mathsf{S_v}, \mathsf{td}_C) \leftarrow \mathsf{Sim}.\mathcal{S}_2(\mathsf{gp}, \mathsf{td}, C) \end{array} \right] \right| \leq \mathrm{negl}(\lambda),$$

where both oracles take a query (x, w) and answer $\perp$ if $(x, w) \notin \mathcal{R}_C$. Otherwise the real oracle $\mathcal{O}_{\mathcal{P}}$ returns $\mathcal{P}(\mathsf{S_p}, x, w)$, and the simulated oracle $\mathcal{O}_{\mathsf{Sim}}$ returns $\mathsf{Sim}.\mathcal{P}(\mathsf{td}_C, x)$. So the adversary may ask for proofs of as many statements as it likes, each chosen after seeing the earlier proofs (*multi-statement* zero knowledge). The simulator never sees w; td and td_C are simulation trapdoors. Zero knowledge is *perfect* if the difference is 0, and *statistical* if the bound holds against unbounded $\mathcal{A}$.

Remark 1 (setup models). The three kinds of setup are three ways of filling $\mathcal{S}_1$ and $\mathcal{S}_2$. In each case the experiments above give the right adaptivity.

- *Circuit-specific (trusted)*: $\mathcal{S}_1$ outputs nothing (gp is empty), and $\mathcal{S}_2$ uses secret randomness for each circuit, which must be discarded after setup. Choosing C "after gp" then just means C is fixed before the circuit's parameters (e.g. GGPR, Pinocchio, Groth16). Letting the adversary choose C is stronger than the definition in [Gro16, §2.2], where the relation comes from a benign relation generator; but Groth's definition also gives the adversary an auxiliary input, which the definition above omits, so neither definition implies the other [Gro16, §2.2, pp. 8–9].
- *Universal (trusted, often updatable)*: $\mathcal{S}_1$ uses secret randomness once, and $\mathcal{S}_2$ is deterministic, so anyone can preprocess a new circuit without a new trusted setup (e.g. Sonic, PLONK, Marlin, whose $\mathcal{S}_2$ is Marlin's indexer).
- *Transparent*: neither $\mathcal{S}_1$ nor $\mathcal{S}_2$ uses secret randomness; they are deterministic apart from public random coins (e.g. STARK, Fractal, Spartan, Supersonic with class groups).

A *common reference string* (CRS) is any reference string that a setup outputs. A *structured reference string* (SRS) is one with algebraic structure, such as powers of a secret, that only secret randomness can produce. We keep each paper's own term, so Groth16's circuit-specific CRS and PLONK's universal SRS are both structured in this sense; a transparent setup needs no SRS.

Remark 2 (random oracle model). Following [COS20, §9], for schemes made non-interactive by the Fiat–Shamir transform, a uniformly random function $\rho : \{0,1\}^* \to \{0,1\}^\lambda$ is sampled first in each experiment, and $\mathcal{S}_1, \mathcal{S}_2, \mathcal{P}, \mathcal{V}$ and every adversary get oracle access to ρ. In knowledge soundness, $\tilde{\mathcal{P}}$ makes at most $\mathrm{poly}(\lambda)$ queries, and the extractor also receives the list of those queries and their answers ([COS20, §9] fixes one extractor for all such provers, with black-box access to $\tilde{\mathcal{P}}$, so it sees those queries). In zero knowledge, the simulator answers the adversary's queries to ρ itself, so it may program ρ, in addition to or instead of using td and td_C.

Remark 3 (simulation extractability). Knowledge soundness concerns a single, isolated prover. An attacker who sees other parties' proofs may instead try to *maul* one into a proof of a different statement without knowing its witness [FR26, §1, pp. 35–36]. *Simulation extractability* requires extraction "even after seeing simulated proofs for potentially false statements" [FR26, abstract, p. 35]; trivial forgeries, such as returning a received proof, do not count [FR26, §3, p. 48]. Sahai's simulation soundness was extended to this notion by De Santis et al. [FR26, §1, p. 36]. The definition above does not require it, but it matters in practice: Nova's proofs of incrementally verifiable computation (IVC, §6.1), for example, are malleable (§6.3). Section 9.4 returns to simulation extractability for sum-check-based schemes.

Note that non-interactivity is built into the syntax with one prover message. A Succinct Non-interactive ARGument (SNARG) is the weaker notion that drops zero knowledge and weakens knowledge soundness to soundness. We do not state soundness separately, because knowledge soundness implies it: for a false statement x no w satisfies $(x, w) \in \mathcal{R}_C$, so the event bounded in Property 2 reduces to $\mathcal{V}$ accepting, which therefore has negligible probability.

Besides strict succinctness, which separates the schemes of Tables 7.1 and A.2 (§5.4), we need a rule for when a scheme counts as zero-knowledge. We count a scheme as zero-knowledge only if its paper proves zero knowledge, for the scheme itself or for a variant that the paper specifies. If the paper leaves zero knowledge to standard or known techniques, or does not treat it, we call the scheme *not zero-knowledge as presented*, even when zero knowledge could plausibly be added (§4.5 describes how). Such schemes are still discussed, for example as polynomial commitments (§4.4) or as results on open problems (§9), but they have no rows in Tables 7.1 and A.2, which compare only zero-knowledge schemes. Plonky2 (Table A.2) is an exception: its whitepaper describes how it blinds the prover's polynomials, but refers to a blog post for the proof, makes FRI zero-knowledge by "standard techniques", and leaves the zero knowledge of FRI openings unfinished [Pol22, §3.6 and fn. 6, pp. 8–9]. We keep it because it is the deployed instance of the hash-based, cycle-free recursion route of §5.3 and §6.4, and its row in Table A.2 states this caveat.

3.1 Why Knowledge Assumptions and Random Oracles?

The definition above does not say which assumptions a zk-SNARK may rest on, and in practice every scheme in this survey relies on a knowledge-type assumption, an idealized group model or a random oracle. This is not an accident of the constructions but a consequence of a barrier identified by Gentry and Wichs [GW11].

The barrier. A cryptographic assumption is *falsifiable* if it can be phrased as a game between an adversary and an efficient challenger that can decide whether the adversary has won. Factoring, discrete logarithm, DDH and LWE are all falsifiable in this sense. Gentry and Wichs show that, assuming some NP language has a sub-exponentially hard subset-membership problem, no SNARG for NP can be proven adaptively sound in the standard (CRS) model by a black-box reduction to any falsifiable assumption, unless that assumption is already false [GW11, Thm. 5.1, Cor. 5.2]. The reason is that soundness of a

Table 3.1: How representative zk-SNARKs instantiate the definition. Each scheme fills the global setup $\mathcal{S}_1$ and the circuit setup $\mathcal{S}_2$ differently (Remark 1). Schemes made non-interactive by Fiat–Shamir are analyzed in the random oracle model (Remark 2). Sizes of the randomly generated parameters are in Table 7.1, column "Setup params".

Scheme	**Setup model**	$\mathcal{S}_1$ **(global setup)**	$\mathcal{S}_2$ **(circuit setup)**	**Random oracle (Remark 2)**	**Source**
Groth16	circuit-specific, trusted	empty	trusted CRS for C from secret randomness, whose simulation trapdoor must stay secret	no: non-interactive by construction	[Gro16, §2.2, §3.1]
PLONK	universal, updatable, trusted	KZG SRS of powers of a secret τ, for all circuits up to the size bound	deterministic preprocessing: commitments to the selector and permutation polynomials	yes: Fiat–Shamir	[GWC19, §1, §8]
Marlin	universal, updatable, trusted	universal SRS for size bound N	deterministic indexer output (index proving and verifying keys)	yes: Fiat–Shamir	[CHM+20, Fig. 1, §1.1, §7]
Fractal	transparent	none beyond the random oracle	deterministic holographic preprocessing of C	yes: preprocessing and Fiat–Shamir in the ROM	[COS20, §2.2, Thm. 3]
Spartan	transparent (KE variant: universal, trusted)	public parameters of the chosen polynomial commitment	deterministic computation commitment to C, with no trusted setup or secret trapdoor	yes: Fiat–Shamir	[Set20, §1, §6, Fig. 2]
Supersonic (DARK)	transparent with class groups (trusted with RSA)	group description and a generator, constant size	deterministic preprocessing of the underlying polynomial IOP	yes: Fiat–Shamir	[BFS20, abstract, §1, §4.3]

succinct argument is itself not falsifiable: to decide whether an adversary has proven a false statement, the challenger would have to decide whether $x \notin L$ [GW11, §1.1]. Moreover, because the proof is short, an inefficient adversary can pair false statements with fake proofs that no efficient reduction can tell apart from honestly generated ones [GW11, Lemma 4.1]. Since knowledge soundness implies soundness, the barrier implies that any standard-model security proof of a zk-SNARK for circuit satisfiability must use either a non-falsifiable assumption or a non-black-box reduction.

Knowledge assumptions. A knowledge assumption states that any efficient algorithm producing a certain kind of output must "know" a corresponding secret, in the sense that an efficient extractor can recover it; for example, whoever outputs a pair (c, c^α) given (g, g^α) must know a with $c = g^a$. The q-power knowledge-of-exponent assumption (q-PKE) of GGPR and Pinocchio extends this to a reference string of many encodings: whoever outputs two encodings related by a secret factor α must know a linear combination of reference-string elements that explains them [GGPR13, §5.1]. Such assumptions are not (obviously) falsifiable, because they assert the existence of an extractor for every adversary [GW11, §2.2], but they are essentially unavoidable: extractable collision-resistant hashing suffices for (designated-verifier) SNARKs, and SNARKs in turn imply a variant of it [BCCT12, §1.2, Thm. 1, §7, Prop. 7.1].

Pairing-based zk-SNARKs therefore take one of two routes. GGPR and Pinocchio assume q-power knowledge-of-exponent (q-PKE) assumptions directly [GGPR13; PHGR13], while Groth16 is proven in the generic group model [Gro16] and Sonic, PLONK and Marlin in the algebraic group model [MBKM19; GWC19; CHM+20], idealized models in which the adversary's group operations can be traced, so that extraction is built in: in the algebraic group model, for instance, an adversary must give each group element's representation in terms of the SRS [GWC19, §2.2]. Sonic, PLONK and Marlin are also made non-interactive by Fiat–Shamir, so their analysis combines the algebraic group model with the random oracle model; Marlin's polynomial commitment can also be realized in the plain model under concrete knowledge assumptions [CHM+20, App. B.2].

Black-box extraction is not an alternative in the standard model: under the discrete-logarithm assumption some NP relation then needs proofs of size $\Omega(|w|)$, a bound that Fiat–Shamir arguments escape because their extractor sees the prover's oracle queries [CGKS23, Thm. 1, Cor. 2, §1].

Random oracles. The random oracle model plays three roles. First, it makes public-coin protocols non-interactive through the Fiat–Shamir transform, which is how Kilian's interactive argument becomes Micali's CS proofs [Mic94] and how every scheme built from a polynomial IOP (PIOP, §4.2) obtains a single-message proof. Second, it provides extraction without a knowledge assumption: the extractor observes the prover's oracle queries and reconstructs the string it committed to [Val08, §4.3]. Third, it lets the zero-knowledge simulator program the oracle instead of using a trapdoor (Remark 2). Because Gentry and Wichs' barrier concerns the standard model, it does not apply here, and in the random oracle model falsifiable assumptions suffice: hash-based schemes such as STARK, Aurora and Fractal need only the oracle itself, and PLONK has been proven knowledge-sound in the random oracle model from two falsifiable assumptions, the adaptive rational strong Diffie–Hellman assumption (ARSDH) and SplitRSDH [LPS25] (§5.2).

The price. The random oracle model is a heuristic: a deployed scheme replaces the oracle with a concrete hash function, and security then rests on that hash behaving like a random function. This can fail: there is a practical attack on the Fiat–Shamir transform of a standard GKR-based argument [KRS25] (§4.2), and Dao et al. found 36 implementations across 12 proof systems that hash too little (§9.4) [DMWG23]. Recursion also forces the

heuristic step, since a verifier that queries the oracle must compute the hash inside the circuit [Val08, Remark 2; BCMS20, §1.1], and §6.1 shows that this is hard to avoid. In short, zk-SNARKs trade one of two costs: non-falsifiable or idealized-group assumptions in the standard model, or falsifiable assumptions in a heuristic random oracle model.

4 Anatomy of a Modern zk-SNARK

Most zk-SNARKs in this survey are built by one modular recipe. Thaler states it in three steps: design a public-coin polynomial IOP for circuit or R1CS satisfiability, replace each polynomial the prover sends by a polynomial commitment, and remove interaction with the Fiat–Shamir transform; he notes that all SNARKs in his monograph follow it, "with the lone exception of those based on linear PCPs" [Tha22, §10.2, p. 360]. Preceded by an arithmetization step, this gives the four-stage pipeline of Figure 1, and each stage fills a specific slot of the definition in §3. The arithmetization fixes the circuit C and hence the relation $\mathcal{R}_C$ (§4.1). The information-theoretic protocol provides soundness without any setup or cryptographic assumption (§4.2). Lookup arguments, which prove that committed values lie in a fixed table, run alongside the constraint system and build on both (§4.3). The setup of the polynomial commitment scheme is the global setup $\mathcal{S}_1$: depending on the scheme, gp is a structured reference string, a list of random group generators, the description of a group of unknown order, or only a hash function (§4.4). Preprocessing, also called indexing, is the circuit setup $\mathcal{S}_2$: it commits to polynomials that encode C, and these commitments, placed in $\mathsf{S_v}$, let the verifier run in time sublinear (often polylogarithmic) in $|C|$ without reading C (§4.2, §4.5). Finally, the Fiat–Shamir transform is where the random oracle ρ of Remark 2 enters (§4.5). The linear-PCP schemes (GGPR, Pinocchio, Groth16) and, in part, the GKR-based schemes take other routes; the end of §4.5 treats them.

4.1 Arithmetization

The first stage re-expresses the computation as a system of polynomial constraints over $\mathbb{F}$. (Compiling high-level programs into circuits, the "front end", is outside our scope; see [Tha22, Ch. 6].) The choice of format affects prover cost and which protocols of §4.2 apply, but, for SNARKs built from polynomial commitments, not the setup model, which the commitment scheme alone fixes (§4.5).

R1CS and QAPs. A rank-1 constraint system consists of three matrices $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}^{m \times n}$ and is satisfied by a vector z with $z_1 = 1$ such that $(\mathbf{A}z) \circ (\mathbf{B}z) = \mathbf{C}z$, where $\circ$ is the entrywise product [Tha22, §8.4, Eq. (8.3), p. 329]; the statement x and the witness w are entries of z. An arithmetic circuit converts into an R1CS instance whose dimensions are proportional to $|C|$ and whose rows have at most fan-in many nonzero entries [Tha22, §8.4.1, p. 330].

A quadratic arithmetic program (QAP) is the same object in polynomial form [Tha22, §17.4, fn. 7, p. 560]. Fix distinct points $\theta_1, \ldots, \theta_m \in \mathbb{F}$, one per constraint, interpolate each column of the R1CS matrices, and let the target polynomial t vanish on these points:

$$v_k(\theta_i) = \mathbf{A}_{i,k}, \quad w_k(\theta_i) = \mathbf{B}_{i,k}, \quad y_k(\theta_i) = \mathbf{C}_{i,k}, \qquad t(X) = \prod_{i=1}^{m} (X - \theta_i).$$

Then z satisfies the R1CS instance if and only if

$$t(X) \;\Big|\; \Big(\sum_k z_k\, v_k(X)\Big)\Big(\sum_k z_k\, w_k(X)\Big) - \sum_k z_k\, y_k(X)$$

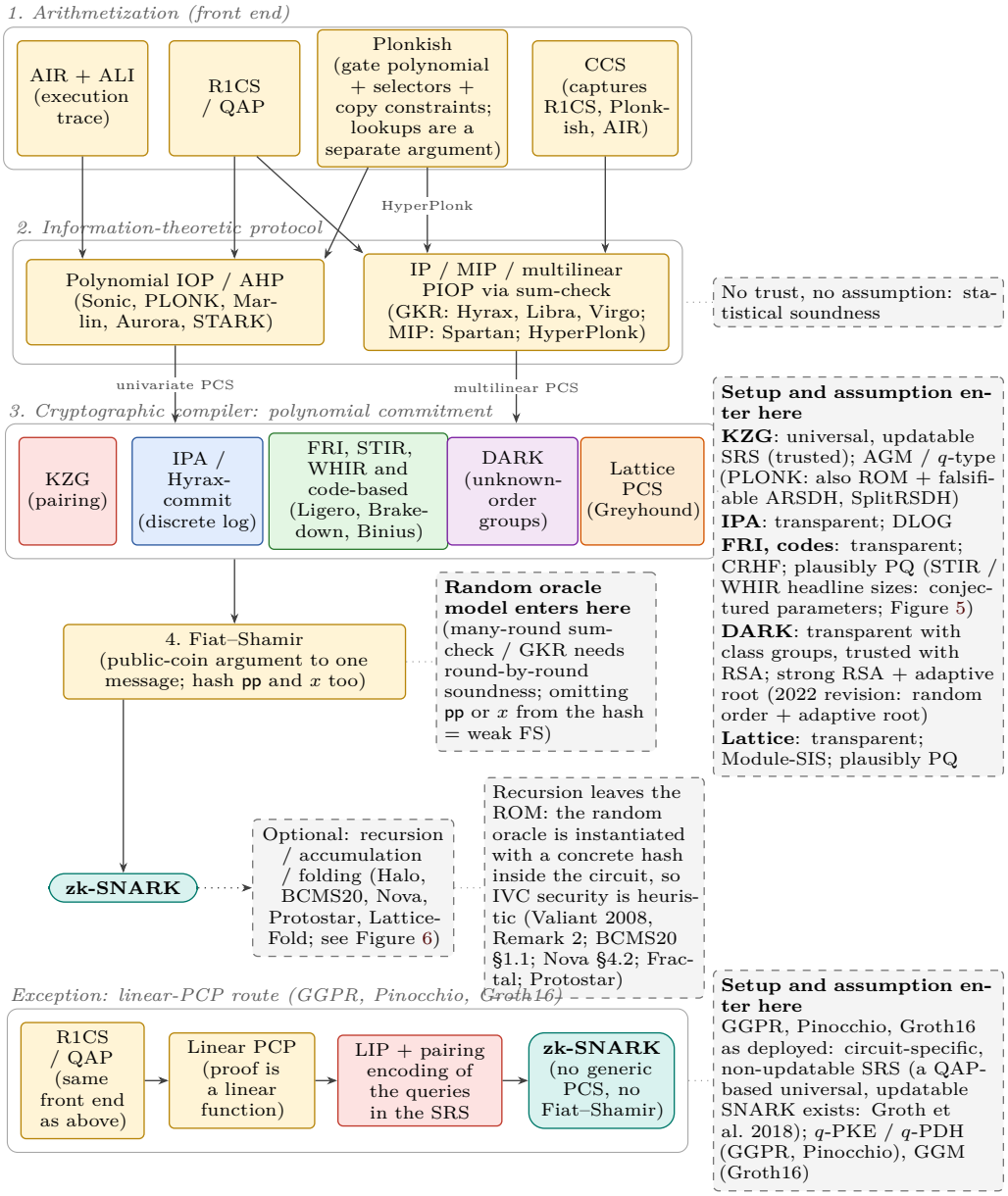


Figure 1: The modular zk-SNARK pipeline and where trust enters. The modular recipe behind most modern zk-SNARKs. A computation is arithmetized into a constraint system (ALI is STARK's algebraic linking IOP [BBHR18, §2.3]). An information-theoretic protocol (a polynomial IOP, or an IP/MIP run through sum-check) proves the constraints hold with statistical soundness and no cryptographic assumption. A polynomial commitment scheme compiles the oracles into a succinct interactive argument, and this is where the setup and the hardness assumption enter. Fiat–Shamir then removes interaction in the random oracle model. The linear-PCP route (bottom) is the exception. Its queries are fixed in a circuit-specific SRS and answered "in the exponent", so it uses neither a generic commitment scheme nor Fiat–Shamir. Colours (also in Figures 3 and 4; Figure 6 has its own key): red = pairing-based (trusted SRS), blue = discrete-log (transparent, not post-quantum), green = hash-based (transparent, plausibly post-quantum), purple = groups of unknown order, orange = lattice-based.

[Tha22, §17.4, Eq. (17.9), pp. 560–562; GGPR13, Def. 11, §7.1], and the prover's evidence is the quotient $h(X)$. Addition gates cost nothing, because they are absorbed into the linear combinations [Gro16, §2.3]. Computing h takes interpolation and polynomial division, which is why QAP-based provers are quasi-linear (Table 7.1).

R1CS, or its QAP form, is the constraint system of GGPR, Pinocchio, Groth16, Marlin, Fractal, Spartan and Aurora (Table A.1).

Plonkish. PLONK's constraint system assigns gate i a left, right and output wire a_i, b_i, c_i and five selector values, and requires $(q_L)_i z_{a_i} + (q_R)_i z_{b_i} + (q_O)_i z_{c_i} + (q_M)_i z_{a_i} z_{b_i} + (q_C)_i = 0$ [GWC19, §6, p. 23]. The wiring is enforced separately by *copy constraints*, which PLONK checks with a permutation argument over the prover's wire polynomials [GWC19, §5.2, §7, step 2, p. 24] (explained below). "Plonkish" arithmetization generalizes the gate equation to a fixed higher-degree polynomial whose terms are switched on by selectors, of degree up to 9 in Zcash's Orchard protocol [Tha22, §19.3.3, p. 627]. Formally, a Plonkish structure fixes a polynomial g in t variables with q monomials of total degree at most d, selectors $s \in \mathbb{F}^e$, and index vectors $T_0, \ldots, T_{m-1} \in \{0, \ldots, n+e-1\}^t$, and a witness w satisfies an instance x if

$$g\big(z[T_i[1]], \ldots, z[T_i[t]]\big) = 0 \quad \text{for all } i \in \{0, \ldots, m-1\}, \qquad z = (w, x, s)$$

[STW23, Def. 2.3, p. 5]. Copy constraints disappear in this form, because the assignment is deduplicated [STW23, Remark 3, p. 5].

AIR. An algebraic intermediate representation, the format of STARK [BBHR18], applies one constraint polynomial g in t variables (t even) to every pair of adjacent rows of an execution trace. The assignment z consists of $m+1$ rows $z_0, \ldots, z_m \in \mathbb{F}^{t/2}$, each holding a machine's registers; the public input x supplies the first and last rows and the witness the rows in between. It is satisfied if

$$g(z_{i-1}, z_i) = 0 \quad \text{for all } i \in \{1, \ldots, m\}$$

[STW23, Def. 2.4, pp. 6–7]. Because the same constraint applies at every step, AIR suits uniform, repeated computations; this is also why STARK's succinctness is claimed for "a uniform computation" [BFS20, §1, p. 2] (Table A.2).

CCS. Customizable constraint systems unify the three formats. A CCS instance is satisfied if $\sum_{i=0}^{q-1} c_i \cdot \bigcirc_{j \in S_i} M_j z = \mathbf{0}$, for sparse matrices M_j, multisets S_i of size at most d, constants c_i, and $\bigcirc$ the Hadamard product [STW23, Def. 2.2, p. 4]. CCS captures "R1CS, Plonkish, and AIR without overheads" and, unlike Plonkish and AIR, "is not tied to any particular proof system" [STW23, abstract]. HyperNova folds CCS instances (§6.4).

Copy constraints by a grand product. PLONK interpolates the left, right and output values of row i at ω^i into wire polynomials f_L, f_R, f_O, where ω generates a multiplicative subgroup H [GWC19, §7, step 1, p. 24], and turns copy constraints into a permutation check. With $n = |H|$ rows, list the $3n$ cells of (f_L, f_R, f_O) as one sequence $f_{(1)}, \ldots, f_{(3n)}$, and let σ cycle through the cells of each wire. The wiring holds exactly when $f_{(\ell)} = f_{(\sigma(\ell))}$ for every cell ℓ [GWC19, §5.2, p. 22]. The verifier sends random $\beta, \gamma \in \mathbb{F}$, and each cell is tagged twice, once with its own position and once with its image under σ:

$$\prod_{\ell=1}^{3n} \big(f_{(\ell)} + \beta\ell + \gamma\big) \;=\; \prod_{\ell=1}^{3n} \big(f_{(\ell)} + \beta\sigma(\ell) + \gamma\big).$$

If the wiring is violated, the two sides differ except with probability at most $3n/|\mathbb{F}|$. The proof uses unique factorization: as polynomials in β, γ, both sides are products of linear

factors, so they are equal only if their factors match [GWC19, Claim A.1, p. 34]. The prover shows the equality with one more polynomial, an accumulator U (PLONK's Z) with $U(\omega) = 1$ whose value at ω^i is the running ratio of the two products over rows $1, \dots, i-1$. The verifier checks two identities on H: $L_1(X)(U(X) - 1) = 0$, where L_1 is the Lagrange polynomial that is 1 at ω and 0 on the rest of H [GWC19, §5, pp. 19–20, §5.1, p. 22], and $U(X)f'(X) = g'(X)U(\omega X)$, where f' and g' multiply the three position-tagged and σ-tagged wire polynomials. Because $\omega^{n+1} = \omega$, the step identity carries the ratio once around H and back to 1, which forces the full products to be equal [GWC19, §5, pp. 19–21, §5.1, pp. 21–22].

Native arithmetic beyond one prime field. Arithmetizing over a fixed prime field also costs whenever the computation lives in a different algebraic structure. Writing secp256k1 ECDSA verification over an FFT-friendly field makes the circuit about 25× larger than over its native field [BFK+24, §1, p. 277]. Field-agnostic commitments (§4.4) remove this overhead for other prime fields. The choice among primes matters too. Univariate STARKs need a large power-of-two subgroup of $\mathbb{F}_p^*$, a condition on $p - 1$; Circle STARKs move it to $p + 1$ by working over the circle curve $x^2 + y^2 = 1$, whose $p + 1$ points form a cyclic group, which makes the Mersenne prime $2^{31} - 1$ usable [HLP24, Def. 1, p. 3, Lemma 1, p. 4, §3.1, p. 5]. For the rings that CPUs use, Jia et al. build transparent SNARKs over $\mathbb{Z}_{2^k}$ via Galois-ring extensions, with a linear-time verifier and no claim of zero knowledge [JLX+25, abstract, Thm. 1.2].

Hash functions inside circuits. Merkle commitments, Fiat–Shamir challenges and recursion all require hashing inside the circuit, where the cost is the number of multiplications over the native field rather than speed on a CPU. Poseidon, a sponge designed for this cost model, needs about two orders of magnitude fewer R1CS constraints than SHA-256 for a Merkle membership proof [GKRRS21, §2.2, p. 5, §6.3, Table 4, pp. 14–15]. Poseidon2 replaces its dense linear layers with cheap structured matrices, which leaves R1CS and AIR costs about the same and speeds up the native hashing that the prover does outside the circuit [GKS23, abstract, §8, p. 20].

4.2 Information-Theoretic Protocols: PCPs, IOPs and Polynomial IOPs

From PCPs to IOPs. The oldest route to succinctness is the probabilistically checkable proof (PCP), a proof string that the verifier spot-checks in a few positions, compiled into an argument by Kilian's Merkle commitment and made non-interactive by Micali's random oracle (§2.2) [Kil92; Mic94]. Valiant proved that this compilation is knowledge-sound in the random oracle model, by extracting the PCP from the prover's recorded oracle queries [Val08]; Thaler sketches the argument [Tha22, §9.2.1, pp. 342–344]. The weak point is the prover, who must write down the entire PCP. Interactive oracle proofs (IOPs) [BCS16] [RRR16] generalize both PCPs and interactive proofs: prover and verifier interact over several rounds, but the verifier only queries the prover's messages instead of reading them in full [Tha22, §10.1, p. 358]. Ben-Sasson, Chiesa and Spooner showed that any IOP compiles into a non-interactive argument in the random oracle model by Merkle-hashing each prover message and applying Fiat–Shamir, "in a manner entirely analogous to the Kilian-Micali transformation" [Tha22, §10.1, p. 358]. STARK, Aurora and Fractal are compiled this way.

Polynomial IOPs. In a polynomial IOP (PIOP), some prover messages are polynomials of bounded degree, and the verifier learns only their evaluations at points of its choice [Tha22, §10.2, p. 359]. Bünz, Fisch and Szepieniec defined the notion and view "polynomial commitment schemes as a compiler for Polynomial IOPs" [BFS20, §1.1, p. 3]; Marlin's

algebraic holographic proofs (AHPs) are "essentially equivalent" [BFS20, §1.2, p. 6], and PLONK's polynomial protocols [GWC19, §4, Def. 4.1] play the same role. Soundness at this layer is statistical: the existing universal and transparent SNARKs "provide a variety of statistically-sound Polynomial IOPs" [BFS20, §1.1, p. 3], and no cryptographic assumption is involved until the compilation step (Figure 1).

Two styles dominate. *Univariate* PIOPs (PLONK, Marlin, Aurora, STARK; Sonic instead shows that a bivariate Laurent polynomial has zero constant term [MBKM19, §5]) encode vectors as evaluations over a set H and prove that a constraint polynomial vanishes on H by exhibiting its quotient by the vanishing polynomial Z_H; for a multiplicative subgroup, $Z_H(X) = X^{|H|} - 1$, which the verifier evaluates in $O(\log|H|)$ time [Tha22, §10.3.1, p. 363]; STARK instead uses additive subspaces of a binary field [BBHR18]. *Multilinear* PIOPs encode a vector of length 2^v by its multilinear extension, the unique multilinear polynomial in v variables that agrees with it on the Boolean hypercube [Tha22, §3.5, Fact 3.5]. They reduce claims about sums over the hypercube to one evaluation with the sum-check protocol of Lund, Fortnow, Karloff and Nisan, described below. Spartan proves R1CS this way [Set20, abstract], SuperSpartan extends it to CCS [STW23, abstract], and HyperPlonk is "an adaptation of Plonk to the boolean hypercube, using multilinear polynomial commitments", which removes the large FFT that is "the bottleneck in generating a Plonk proof" [CBBZ23, abstract]. Thaler also recasts the GKR- and MIP-based arguments (both below) as multilinear PIOPs in which the prover's only polynomial message is the multilinear extension of the witness or of the circuit transcript [Tha22, §10.6, p. 395].

The sum-check protocol. The prover claims that a v-variate polynomial g over $\mathbb{F}$, of degree at most k in each variable, sums to T over the Boolean hypercube (we write k for this degree because d is the circuit depth below). In round j it sends the univariate polynomial g_j, where $r_1, \ldots, r_{j-1}$ are the verifier's earlier challenges:

$$T = \sum_{\boldsymbol{b} \in \{0,1\}^v} g(\boldsymbol{b}), \qquad g_j(X_j) = \sum_{(b_{j+1},\ldots,b_v) \in \{0,1\}^{v-j}} g(r_1, \ldots, r_{j-1}, X_j, b_{j+1}, \ldots, b_v).$$

The verifier checks that g_j has degree at most k and that $g_{j-1}(r_{j-1}) = g_j(0) + g_j(1)$, with T on the left when $j = 1$. It then sends a fresh uniform $r_j \in \mathbb{F}$. After round v it checks $g_v(r_v) = g(r_1, \ldots, r_v)$ with a single evaluation query. Until then it needs to know only the degree bound [Tha22, §4.1, pp. 168–171, Remark 4.2, p. 174]. A false claim survives only if some g_j differs from the honest polynomial yet agrees with it at r_j. That happens with probability at most $k/|\mathbb{F}|$ per round, so the soundness error is at most $vk/|\mathbb{F}|$ [Tha22, Prop. 4.1, p. 172]. The verifier does $O(vk)$ work plus the query. The prover evaluates g at $(k+1)\,2^{v-j}$ points in round j, which is $O(2^v)$ in total for constant k [Tha22, p. 174].

Multilinear extensions make that prover cost real. The extension $\tilde{f}(X) = \sum_{\boldsymbol{b}} f(\boldsymbol{b})\,\chi_{\boldsymbol{b}}(X)$, with $\chi_{\boldsymbol{b}}(X) = \prod_{i=1}^{v} \big(X_i b_i + (1 - X_i)(1 - b_i)\big)$, can be evaluated at any point in $O(2^v)$ time [Tha22, Lemma 3.6, p. 159, Lemma 3.8, p. 163]. If g is a product of k multilinear polynomials, the prover runs in time $O(k \cdot 2^v)$, linear in the data and with no FFT [Tha22, Lemma 4.5, p. 194]. This is why sum-check drives the multilinear systems. HyperPlonk adopts it to cut the prover from $O(|C|\log|C|)$ to $O(|C|)$ [CBBZ23, §1, p. 4]. Lasso (§4.3) is named for "Lookup Arguments via Sparse-polynomial-commitments and the Sum-check protocol" [STW24, fn. 3, p. 3]. HyperNova folds by adding a running instance's claims to the polynomial of SuperSpartan's sum-check [KS24, §1.3, p. 8] (§6.4).

The GKR protocol. GKR runs one sum-check per layer of a *layered* arithmetic circuit, whose wires join only adjacent layers, numbered from the output (layer 0) to the input (layer d) [Tha22, §4.6.3, p. 210]. Let layer i have 2^{k_i} gates, and let $\tilde{W}_i$ be the extension of its gate values. The *wiring predicates* $\mathrm{add}_i(a, b, c)$ and $\mathrm{mult}_i(a, b, c)$ equal 1 exactly when

gate a of layer i adds, respectively multiplies, gates b and c of layer $i+1$. They depend on C, not on its input [Tha22, §4.6.4, pp. 212–214]. Each layer is a sum over the next [Tha22, Lemma 4.7, p. 215]:

$$\tilde{W}_i(z) \;=\; \sum_{b,c\in\{0,1\}^{k_{i+1}}} \widetilde{\mathrm{add}}_i(z,b,c)\big(\tilde{W}_{i+1}(b)+\tilde{W}_{i+1}(c)\big) \;+\; \widetilde{\mathrm{mult}}_i(z,b,c)\,\tilde{W}_{i+1}(b)\,\tilde{W}_{i+1}(c).$$

The verifier starts from the claimed outputs' extension at a random point. Each iteration runs sum-check on this identity, which reduces a claim about $\tilde{W}_i$ at one point to claims about $\tilde{W}_{i+1}$ at two points. The prover merges these into one by sending $\tilde{W}_{i+1}$ restricted to the line through them [Tha22, pp. 214–218]. The last claim concerns the input alone. In the argument, the prover commits to the witness extension $\tilde{w}$ first and opens it once at the end [Tha22, §6.5.2, pp. 265–266; KRS25, §2.1, pp. 7–8]. No intermediate layer is ever committed. That is why GKR must go layer by layer: "the verifier never materializes the intermediate gates of the circuit" [Tha22, §8.2.1.2, p. 321]. The price is depth. Rounds and communication are $O(d\log|C|)$, and the soundness error is $O(d\log|C|/|\mathbb{F}|)$ [Tha22, §4.6.5, p. 221]. The verifier must also evaluate $\widetilde{\mathrm{add}}_i$ and $\widetilde{\mathrm{mult}}_i$ at a random point for every layer, which is fast for circuits with repeated wiring structure but not in general [Tha22, §4.6.6, p. 222] (Table A.2). Libra made the prover linear in $|C|$ by splitting each layer's sum-check into two, each over a product of two multilinear polynomials [XZZ+19, abstract, §1.2, p. 4].

Why GKR came back, and why that is fragile. Committing only to the input is what makes GKR provers fast: Hyrax uses cryptography "only for the witness and for the prover's messages" [WTS+18, §1, pp. 1–2], and Virgo's prover takes $O(|C|+n\log n)$ time for n inputs: field arithmetic linear in the circuit, plus a commitment to the input layer [ZXZS20, §4.1, p. 19, Thm. 3, p. 20]. logUp-GKR, a lookup argument (§4.3), uses GKR to compute LogUp's fractional sum, so the prover commits to the multiplicities but to no helper columns [PH23, abstract, §2, p. 2]. Khovratovich, Rothblum and Soukhanov's attack on the Fiat–Shamir version of such arguments uses the same property. For any multilinear commitment and Fiat–Shamir hash, they build a slightly deeper circuit that computes both internally, together with an accepting proof of a false statement, so the compiled argument "is not adaptively sound" [KRS25, Thm. 1, p. 4, §1.1.2, p. 4]. Nothing commits the prover to that internal computation: GKR's prover "does not commit to the full computation trace (indeed, this is one of the most compelling features of this protocol). Unfortunately, the fact that the computation is not committed to also enables our attack" [KRS25, §5, p. 16]. One mitigation they suggest, reducing the circuit's depth "possibly by committing to some of the intermediate values" [KRS25, §5, p. 16], gives back part of that speed. For the simple, canonical circuits of lookup and grand-product arguments, they say the attack "does not seem to be applicable" [KRS25, Remark 3, p. 6]. Section 5.4 lists the schemes it affects.

Multi-prover interactive proofs. In a multi-prover interactive proof (MIP) [BGKW88] the verifier questions two or more provers, who may agree on a strategy beforehand but cannot communicate during the protocol [Tha22, Def. 8.1, pp. 312–313]. A second prover never sees the questions put to the first, so it "essentially functions as a polynomial commitment scheme" [Tha22, §8.1.1, pp. 313, 316]. For R1CS (§4.1), a claimed multilinear extension $\tilde{z}$ of the assignment is valid exactly when, for every row index $\boldsymbol{a}$,

$$\Big(\sum_{\boldsymbol{b}} \tilde{\mathbf{A}}(\boldsymbol{a},\boldsymbol{b})\,\tilde{z}(\boldsymbol{b})\Big)\cdot\Big(\sum_{\boldsymbol{b}} \tilde{\mathbf{B}}(\boldsymbol{a},\boldsymbol{b})\,\tilde{z}(\boldsymbol{b})\Big) \;-\; \sum_{\boldsymbol{b}} \tilde{\mathbf{C}}(\boldsymbol{a},\boldsymbol{b})\,\tilde{z}(\boldsymbol{b}) \;=\; 0$$

[Tha22, §8.4.2, Eq. (8.4), p. 331], which a few sum-checks reduce to evaluations of $\tilde{\mathbf{A}}$, $\tilde{\mathbf{B}}$, $\tilde{\mathbf{C}}$ and $\tilde{z}$ at random points [Tha22, pp. 332–334]. In the argument the prover commits

to $\tilde{z}$ instead, which removes the second prover and the low-degree test [Tha22, §8.3, pp. 327–328]. Unlike GKR, the number of sum-checks does not grow with the depth, because the committed polynomial extends the entire assignment and not only the input; the price is a commitment to more than the witness [Tha22, pp. 321, 328]. Spartan (§5.3) compiles the two-prover protocol of Babai, Fortnow and Lund [BFL90] into a public-coin succinct argument "without employing FHE or low-degree tests" [Set20, §3.2, pp. 14–15].

Holography and preprocessing. A verifier that reads C cannot run in time polylogarithmic in $|C|$. Holographic protocols remove this obstacle. In an AHP, an indexer "receives as input the index i to be preprocessed, and outputs one or more univariate polynomials over F encoding i", and the verifier queries these polynomials instead of reading the circuit [CHM+20, §2.1, p. 10]. After compilation, the indexer becomes a deterministic circuit setup $\mathcal{S}_2$ whose output contains commitments to the index polynomials; its cost is the AHP indexer's plus the cost of committing to them [CHM+20, Thm. 8.1, p. 41]. Fractal establishes "a connection between holography and preprocessing in the random oracle model", which gives a transparent $\mathcal{S}_2$ [COS20, abstract]. For sum-check-based arguments, Spartan's computation commitments play the same role, letting the verifier achieve sublinear costs "after investing a one-time, public computation to preprocess a given NP statement" [Set20, abstract]. This is the precise sense in which these schemes are preprocessing without being circuit-specific (§3, Remark 1).

4.3 Lookup Arguments

A lookup argument proves that committed values lie in a fixed table. It replaces expensive bit decompositions for range checks, bitwise operations and big-number arithmetic [GW20, §1] [STW24, abstract], and it runs alongside the constraint system of §4.1 rather than being a format of its own. Figure 2 groups the arguments by technique, following Qu et al.'s account of the line that starts with Arya [QGX+26, §1.2, Table 1]. Two shifts organize the line: from grand products to logarithmic derivatives, and from table-sized commitments to sparse sums proven by sum-check (§4.2). Throughout, m is the number of lookups and n the size of the table.

From grand products to logarithmic derivatives. plookup sorts the concatenation of the queries and the table and checks it with a single grand product, "similar to the one used in" PLONK's permutation argument [GW20, §1, p. 2], whose accumulator it reuses (§4.1); the products agree exactly when every query lies in the table, again by unique factorization [GW20, Claim 3.1, pp. 4–5]. LogUp replaces sorting with the identity $\sum_i 1/(X + a_i) = \sum_i m_i/(X + b_i)$, where m_i is the multiplicity of b_i, proven with one batched sum-check over the Boolean hypercube [Hab22, Lemma 5, p. 6, §3.1, pp. 10–11]. The identity can be proven in more than one way. cq runs it directly on the full table with precomputed "cached quotient" KZG commitments, so that after $O(n \log n)$ preprocessing the prover needs only $O(m \log m)$ operations, independent of the table size [EFG22, abstract, §1.2, p. 3]. logUp-GKR proves the fractional sum with the GKR protocol over a binary tree of fraction additions, so that, besides the looked-up columns, the prover "has to commit only to a single extra column, i.e. the multiplicities of the table entries" [PH23, abstract, §3.1, pp. 3–4]. Celer, aimed at queries far outnumbering the table, returns to one grand-product check and commits to fewer than m elements [QGX+26, §1, fn. 1, p. 245, Table 1, p. 247].

Sparse sums and tables that are never committed. Spark (written SPARK in Spartan [Set20, abstract]) proves evaluations of a sparse polynomial, such as the extension of an R1CS matrix that Spartan's verifier needs (§5.3). The prover commits to the values it

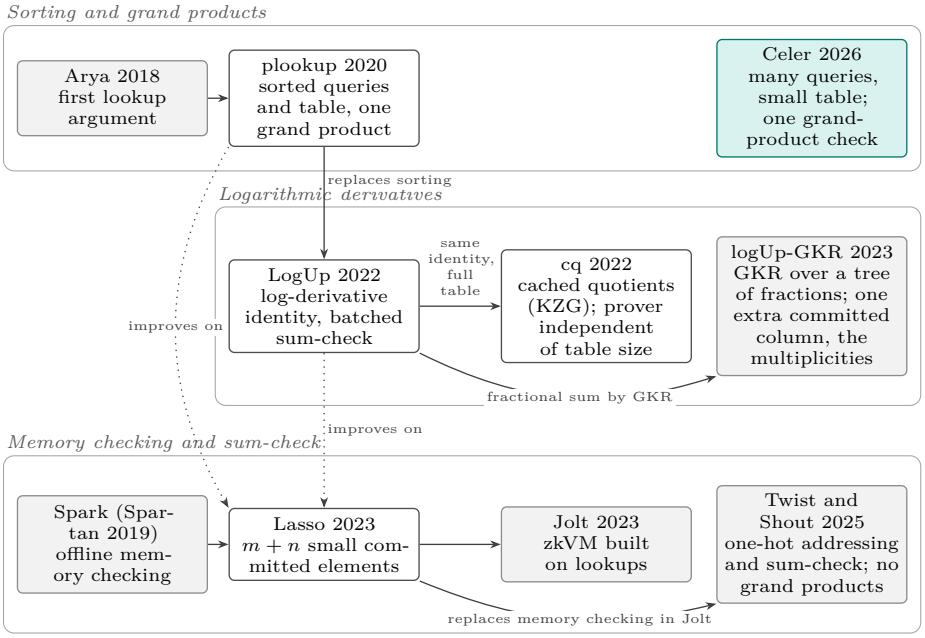


Figure 2: Lookup arguments by technique. Solid arrows mark a construction that builds on or replaces the earlier one; dotted arrows mark a comparison the later paper makes ("improves on"). White nodes commit to a number of field elements linear in the number of lookups m; Celer (teal) commits to fewer than m when $m \gg n$ [QGX+26, §1.1–1.2, Table 1]. Arya is drawn as Qu et al. describe it [QGX+26, §1.2]. Sources: [GW20], [Hab22], [EFG22], [PH23], [Set20], [STW24], [AST24], [QGX+26], [STZ26].

read and shows that every read was correct by offline memory checking, which compares fingerprints of read and write multisets with a grand product [Set20, §7.2, pp. 25–28; STW24, §4.1, pp. 17–19]. Lasso turns lookups into one such sparse sum. The prover claims an $m \times n$ matrix $\mathbf{M}$ with a single 1 in each row such that $\mathbf{M} \cdot \mathrm{Val} = \boldsymbol{a}$ for the vector $\boldsymbol{a}$ of looked-up values, which up to negligible soundness error is the sum-check identity $\sum_{\boldsymbol{y}} \tilde{\mathbf{M}}(\boldsymbol{r}, \boldsymbol{y})\, \widetilde{\mathrm{Val}}(\boldsymbol{y}) = \tilde{\boldsymbol{a}}(\boldsymbol{r})$ at a random $\boldsymbol{r}$ [STW24, §3, Eq. (5), p. 12]. For a *decomposable* table, whose entries are fixed multilinear functions of entries of $O(c)$ small subtables, "no party needs to commit to" the table, so it may have 2^{128} entries or more, and the prover "pays" only for the entries that the lookups read [STW24, abstract, §1.1, p. 4]. Jolt's never-materialized instruction table (§8) relies on this.

One-hot addressing. Twist and Shout drop the fingerprints and the grand product. In Shout, the prover commits to each address in one-hot form, so that correct reads become one sum-check over the address space, together with checks that enforce the one-hot form [STZ26, §4.3, Fig. 5, Thm. 1, p. 283, §4.4, pp. 284–285]. Only m of the $n \cdot m$ entries are nonzero, and the sum-check prover pays only for those [STZ26, §1.1, pp. 269–270]. Twist handles read/write memory by committing to per-cycle increments instead of memory contents [STZ26, §1.1, p. 270]. Together they replace the memory checking of earlier versions of the Jolt zkVM, making the Jolt prover about 3× faster and proofs about 5× smaller on RV32IM benchmarks [STZ26, abstract].

4.4 Polynomial Commitment Schemes

Definition. A polynomial commitment scheme (PCS) lets a prover commit to a polynomial f of bounded degree with a short string and later prove claims $f(z) = y$. It consists of a setup, which outputs public parameters for a maximum degree; a commitment algorithm; and an evaluation protocol, an interactive argument that the committed f satisfies $f(z) = y$. Kate, Zaverucha and Goldberg introduced and formally defined the notion [KZG10, abstract]. *Binding* says that a commitment cannot be opened to inconsistent evaluations. Compiling a PIOP needs the stronger *extractability*: "extractability is to binding as knowledge-soundness is to standard soundness", meaning that any committer who passes the evaluation checks must know a polynomial of the claimed degree that explains its answers [Tha22, §7.4, p. 307]. For zero knowledge via masking, the commitment must also be hiding and the evaluation protocol zero-knowledge [Tha22, §13.3, p. 454]. Chiesa et al. have recently argued that extractability is more than standard soundness needs and does not by itself give Fiat–Shamir security. They identify a state-restoration variant of *function binding*, a generalization of position binding, as the right property, and note that it is incomparable to extractability: the linearized KZG used in PLONK satisfies it, while extending an earlier analysis shows that it is not extractable [CGKY26, §1, pp. 191–192, 195] (§4.5).

Pairing-based: KZG. A KZG commitment to f is $g^{f(\tau)}$ for a secret τ, computed from a reference string of powers g^{τ^i}. The evaluation proof is a single group element, a commitment to the quotient $(f(X)-f(z))/(X-z)$ [KZG10, §1, p. 2], and verification takes two pairings (Table 7.2). In the notation of §2.3, the verifier holds the commitment $c_f = g_1^{f(\tau)}$ and the element g_2^{τ} of the reference string. For a claim $f(z) = y$ with proof $\pi_z = g_1^{\psi(\tau)}$, where $\psi(X) = (f(X) - y)/(X - z)$, it checks

$$e\big(c_f \cdot g_1^{-y},\ g_2\big) \ = \ e\big(\pi_z,\ g_2^{\tau} \cdot g_2^{-z}\big),$$

which is the identity $f(\tau) - y = \psi(\tau)(\tau - z)$ in the exponent [KZG10, §3.2, p. 7; Tha22, §15.2, Eq. (15.2), p. 502]. Only g_2 and g_2^{τ} are needed to verify [Tha22, p. 502]. If $f(z) \neq y$, then ψ is not a polynomial, and passing the check requires "dividing in the exponent" by $\tau - z$, which is what the t-strong Diffie–Hellman (t-SDH) assumption rules out [Tha22, p. 503]. Binding rests on t-SDH [KZG10, §1, p. 2, Def. 2.3, p. 4], and extractability is typically proved in the algebraic group model, for instance for PLONK under $2d$-DLOG [GWC19, §3, pp. 12–13, Lemma 4.7, p. 17]. Belohorec et al. give the first standard-model extractability proofs for multivariate KZG and many of its variants, under the falsifiable n-variate ARSDH assumption or a stronger variant, GARSDH [BDH+25, abstract, §2, Thms. 2–3]; §5.2 covers the univariate case behind PLONK. The reference string is a trusted but universal $\mathcal{S}_1$. It is updatable because it consists only of powers of τ, so any party can re-randomize it [Tha22, §16.3, pp. 545–547]. For multilinear polynomials the proofs grow to logarithmic size [Tha22, p. 545], as in the multilinear KZG that HyperPlonk uses (Table 7.2). Sonic, PLONK and Marlin use KZG.

PLONK batches its openings: a random combination merges the claims about several committed polynomials at one point into a single check, so that its proof carries just two opening elements, for its challenge point z and for $z\omega$, and verification still takes two pairings [GWC19, §3.1, pp. 12–13, §8.3, pp. 30–31].

Discrete-log-based: IPA, Hyrax-commit, Dory. The inner-product argument of Bootle et al. [BCC+16, abstract], refined in Bulletproofs [BBB+17, §3], gives, via the construction in [Tha22, §14.4], a transparent polynomial commitment from discrete log alone. Its evaluation proofs have $O(\log s)$ group elements for a polynomial with s coefficients, but its verifier performs a multi-exponentiation of size $O(s)$ [Tha22, Table 16.1, p. 546]. The

argument halves the committed vector in each round, folding the vector, the generators and the commitment with a random challenge and its inverse, so that after $\log_2 s$ rounds of two group elements each one scalar remains (two in Bulletproofs, which folds both vectors of an inner product) [Tha22, §14.4.1, Protocol 12, pp. 480–482; BBB+17, §3, Protocol 2, pp. 13–16]. The verifier, however, must fold the generators itself, $O(s)$ work in total, which Bulletproofs merges into one multi-exponentiation without shrinking it [Tha22, pp. 481–482; BBB+17, §3.1, pp. 15–17]. The proof length, by contrast, is optimal: over pairing-free prime-order groups, Dietz and Giunta show in the generic group model that $\Omega(\log s)$ group elements per proof are necessary, a bound that pairings (KZG) and groups of unknown order (DARK) escape [DG26, abstract, §1.1, p. 220, Cor. 1, p. 246].

The linear verifier is why Bulletproofs is not a zk-SNARK in the strict sense of §3 (Table A.2, §5.4), and why Halo, which uses the same kind of commitment, must still perform one linear-time final check after amortizing the others across recursive proofs (§6). Hyrax-commit arranges the coefficients as a $\sqrt{s} \times \sqrt{s}$ matrix and commits to it row by row, which gives $O(\sqrt{s})$-size commitments and $O(\sqrt{s})$ verification [WTS+18, §6.1; Tha22, Table 16.1]. Dory reaches $O(\log s)$ communication and verification with a transparent setup, using pairings, and is secure under the standard symmetric external Diffie–Hellman (SXDH) assumption [Lee21, abstract].

Hash-based: how FRI works. FRI, the Fast Reed–Solomon IOP of Proximity [BBHR18b], is an IOP of proximity (§2.1, §4.2) for $\mathrm{RS}[\mathbb{F}, \mathcal{L}_0, d]$, where $\mathcal{L}_0$ is a multiplicative subgroup and $|\mathcal{L}_0|$ and d are powers of 2 [BBHR18b, §2.1, p. 14:10]. In the *commit phase*, the prover splits $f_0(X) = f_{0,\mathrm{e}}(X^2) + X f_{0,\mathrm{o}}(X^2)$ into even and odd parts of degree less than $d/2$. The verifier sends a random $c_0 \in \mathbb{F}$. The prover answers with the fold $f_1 = f_{0,\mathrm{e}} + c_0 f_{0,\mathrm{o}}$ on $\mathcal{L}_1 = \{u^2 : u \in \mathcal{L}_0\}$. Squaring is two-to-one on $\mathcal{L}_0$, so the domain and the degree bound halve and the rate stays the same [BBHR18b, p. 14:11; Tha22, §10.4.4, pp. 378–379]. After about $\log_2 d$ rounds the function is constant, and the prover sends that constant. In the compiled argument each f_i is sent as a Merkle root, and Fiat–Shamir supplies the challenges [BBHR18b, §1.3.2, p. 14:8]. In the *query phase*, the verifier picks a random $u_0 \in \mathcal{L}_0$ and sets $u_{i+1} = u_i^2$. At each layer it opens $f_i(\pm u_i)$ with authentication paths and checks

$$f_{i+1}(u_i^2) \;=\; \frac{f_i(u_i) + f_i(-u_i)}{2} \;+\; c_i \cdot \frac{f_i(u_i) - f_i(-u_i)}{2u_i},$$

whose right side is $f_{i,\mathrm{e}}(u_i^2) + c_i f_{i,\mathrm{o}}(u_i^2)$ for an honest prover [BBHR18b, §2.1, p. 14:11].

The fold is a random combination of two functions on $\mathcal{L}_{i+1}$. If both were δ-close to polynomials p and q of degree less than $d/2$ on a common set of points (correlated agreement), then f_i would be δ-close to $p(X^2) + Xq(X^2)$. So if f_i is δ-far, a proximity gap keeps f_{i+1} δ-far except with small probability [Tha22, pp. 382–383; BBHR18b, §2.2, pp. 14:12–14:13]. Each query repetition then rejects with probability at least δ, so ℓ repetitions leave error $(1-\delta)^\ell$ plus the chance of an unlucky fold [Tha22, Thm. 10.4, p. 383]. The prover's main cost is the FFT that evaluates f_0 on $\mathcal{L}_0$ [Tha22, §10.4.3, p. 376]. About $\lambda / \log_2(1/(1-\delta))$ repetitions give proofs of $O(\lambda \log^2 d)$ hash values [Tha22, pp. 376–377, Table 16.1].

Used as a PCS, the commitment is the Merkle root of f on $\mathcal{L}_0$. To prove $f(z) = y$, the prover runs FRI with degree bound $d-1$ on the quotient $(f(X) - y)/(X - z)$, as in KZG. Each value of the quotient costs the verifier one query to f. For $\delta < (1-R)/2$, acceptance binds the prover to a polynomial f with $f(z) = y$ [Tha22, §10.4.2, pp. 374–375]. STIR reduces the degree by a factor $k \geq 4$ per round but the domain only by 2, so the rate falls to $2R/k$ and later rounds need fewer queries [ACFY24, §2.1, p. 5]. WHIR tests *constrained* Reed–Solomon codes, whose codewords, read as multilinear polynomials $\hat{f}$, also satisfy a sum-check constraint such as $\hat{f}(\boldsymbol{z}) = y$. The proximity test therefore also proves an evaluation [ACFY25, §1.1, Def. 1, p. 5].

Hash-based: FRI's successors and linear codes. Figure 3 shows how the later hash-based commitments descend from FRI and from Ligero, and Table 7.2 gives their costs. STIR lowers the query complexity from FRI's $O(\lambda \cdot \log d)$ to $O(\log d + \lambda \cdot \log \log d)$ for degree d, "including variants of FRI based on conjectured security assumptions", with arguments 1.25–2.46× smaller than FRI's when both use conjectured security parameters [ACFY24, abstract, §1, p. 3]. WHIR handles queries to both multilinear and univariate polynomials and reports verification in "a few hundred microseconds" at 100-bit security under conjectured parameters [ACFY25, abstract, §1, p. 7, Table 2, p. 47].

Commitments based on linear codes trade proof size for prover speed, starting from the Ligero commitment's square-root-size proofs [Tha22, Table 16.1, §10.5]. Brakedown makes the prover linear-time and field-agnostic with a new linear-time-encodable code [GLS+23, abstract]; it appears here only as a PCS, since it is not zero-knowledge as presented (§3, §5.4). Orion makes such proofs polylogarithmic by code switching [XZS22, abstract]. It claims a polylogarithmic verifier after preprocessing, but its theorem proves only a linear one [XZS22, Table 1, p. 3, Thm. 5, p. 23] (§9.1). BaseFold generalizes FRI from Reed–Solomon codes to any foldable linear code and interleaves it with sum-check, giving a field-agnostic multilinear PCS with an $O(s \log s)$ prover for s coefficients [ZCF24, abstract]. Block et al. show that random expand-accumulate codes over any finite field have constant rate and distance, which gives a Brakedown-style PCS with proofs 1.9–2.8× smaller [BFK+24, abstract].

Binius adapts Brakedown to towers of binary fields and keeps square-root verification [DP25, §1, p. 1, Thm. 3.14, p. 20]. Its successor removes this limitation. A "ring-switching" compiler turns a multilinear commitment over a large extension field into one over the ground field, and combined with a characteristic-2 adaptation of BaseFold it gives polylogarithmic verification, with security proven in the unique-decoding regime [DP26, abstract, §4.4, p. 44, §5.2, p. 47]. Blaze, also over binary fields, gets a linear-time prover from repeat-accumulate-accumulate codes followed by BaseFold on a short random combination of rows [BCF+25, abstract, Thm. 1, p. 4]. Ligerito recurses Ligero's matrix–vector protocol with partial sum-checks, with soundness argued in the unique-decoding regime and a polylogarithmic verifier for suitable codes such as Reed–Solomon [NA25, abstract, §6.3, p. 14, §6.6]. Like FRI-based commitments generally [Tha22, §13.3, p. 454], none of these three is zero-knowledge as presented (§3): Blaze "does not guarantee any hiding properties" [BCF+25, Remark 2, p. 5], and DP26 and NA25 do not treat zero knowledge. Hash-based schemes are plausibly post-quantum [Tha22, Table 16.1 caption]. Their concrete parameters, however, often rest on unproven proximity-gap conjectures for Reed–Solomon codes; §5.3 discusses which bounds are proven and which supporting conjectures have been disproved.

Groups of unknown order and lattices. DARK (Diophantine Arguments of Knowledge) commits to polynomials in a group of unknown order [BFS20, §1.1, p. 3]; it is transparent with class groups and trusted with RSA groups (§5.3, Supersonic). Greyhound is a lattice-based PCS from the Module Short Integer Solution problem (Module-SIS, or MSIS) in the random oracle model, with no trusted setup, polylogarithmic evaluation proofs and an $O(\sqrt{s})$ verifier [NS24, §1.1, p. 3]. Its short evaluation proofs come from LaBRADOR, a Module-SIS proof of knowledge for quadratic constraints over a polynomial ring, which proves its last message recursively instead of sending it, so that only $O(\log \log n)$ iterations are needed for a witness of size n [BS23b, §1.1, pp. 4–5]. DewTwo, also in groups of unknown order, is transparent, with doubly logarithmic proofs (Table 7.2), from assumptions that are falsifiable in its interactive form [BMSS25, abstract, §1, fn. 3, p. 551]. Its prover, however, is estimated at just over 18 hours for multilinear polynomials with $s = 2^{24}$ coefficients on one thread [BMSS25, Remark 3, p. 572].

Other lattice-based commitments trade setup, assumptions and proof size differently.

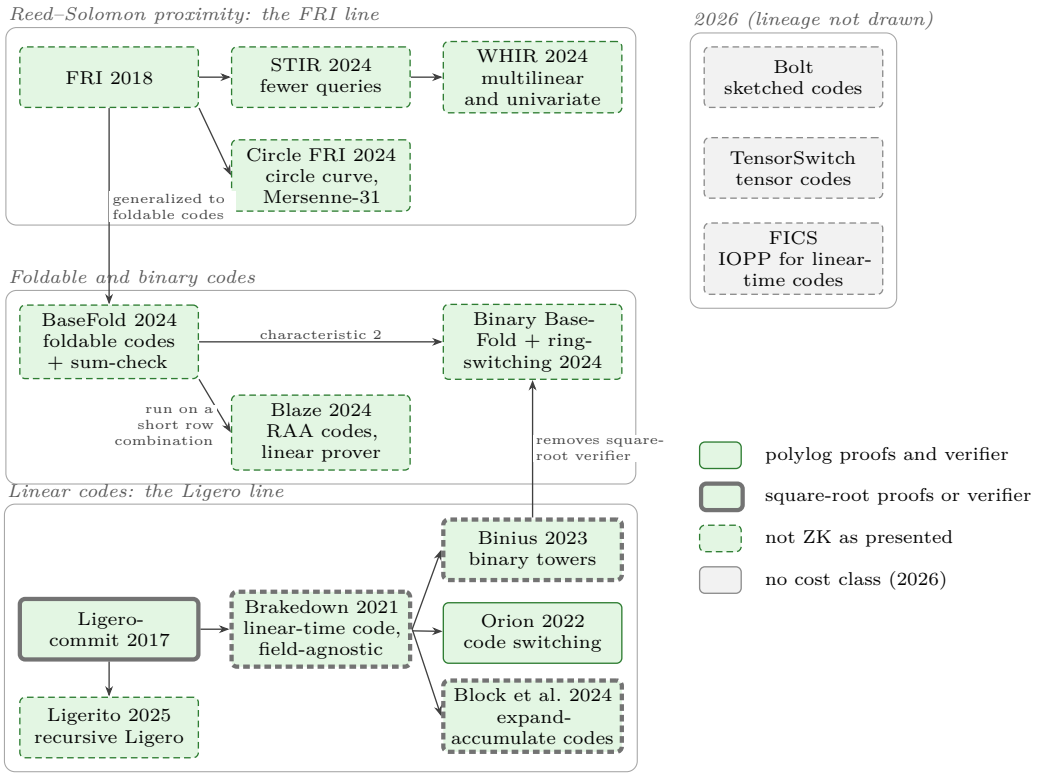


Figure 3: Hash-based polynomial commitments and their lineage. Arrows mark a scheme that adapts or generalizes the earlier one. All nodes are hash-based. The FRI- and Ligero-line nodes are green, as in Figure 4; the 2026 schemes are grey and carry no cost class, because their verifier bounds are not in Table 7.2. Among the green nodes, a thin border marks polylogarithmic evaluation proofs and verification (Orion after preprocessing, as claimed but not proven as a theorem, §9.1; Ligerito for tensorizable codes), so a holographic PIOP compiled with them can meet the strict bound of §3; a thick grey border, as in Figure 4, marks square-root-size proofs or verification. A dashed border marks schemes that are not zero-knowledge as presented, which includes FRI-based commitments [Tha22, §13.3, p. 454]; only Ligero and Orion are presented with zero knowledge, since circle STARKs obtain it only "in the usual manner" [HLP24, §5.4, p. 27], which the rule of §3 does not count. The 2026 schemes, Bolt and TensorSwitch (§9.1) and FICS (§5.3), are drawn without lineage edges. Costs are in Table 7.2. Sources: [BBHR18b], [ACFY24], [ACFY25], [HLP24], [ZCF24], [DP26], [BCF+25], [Tha22, Table 16.1], [GLS+23], [XZS22], [BFK+24], [DP25], [NA25], [GNR26], [BFRW26], [BMMS26].

Cini et al. give a transparent PCS from Module-SIS with a knowledge-soundness proof against quantum adversaries, whose concretely efficient variant has 403 KB proofs at degree 2^{20} [CMNW24, abstract, Fig. 2, p. 209]. Several earlier lattice SNARKs rely on the knowledge k-R-ISIS assumption, and Wee and Wu's oblivious-sampling attack gives evidence against it, although it does not directly break these schemes' soundness [WW23, abstract, §1, p. 11].

Which commitments give strict zk-SNARKs. The commitment, more than the PIOP, decides whether the final argument meets the strict bound of §3. A holographic PIOP whose verifier makes polylogarithmically many queries inherits the commitment's "efficiency bottlenecks" [Tha22, Ch. 19, p. 608], here its per-evaluation cost. Constant (KZG), logarithmic (DARK, and Dory after an $O(\sqrt{s})$-exponentiation preprocessing [Tha22, Table 16.1 caption]) and polylogarithmic (FRI and its successors; Orion with preprocessing, as claimed but not proven, §9.1) evaluation costs therefore give zk-SNARKs in our sense, whereas linear (IPA) or square-root (Hyrax-commit, Ligero, Brakedown, Binius, Greyhound) verification does not. This explains why Bulletproofs, Hyrax and Spartan$_{\mathrm{DL}}$ appear in Table A.2 rather than Table 7.1, even though some of these papers call their schemes zk-SNARKs under sublinear notions of succinctness (§1, §5.4).

Table 7.2 in §7 summarizes these families: setup, commitment and evaluation-proof sizes, verifier cost, assumption, post-quantum plausibility, and whether each yields a zk-SNARK in the strict sense of §3.

4.5 Compiling to a Non-interactive Argument

From PIOP to interactive argument. Compilation replaces each polynomial message by a commitment and each evaluation query by an evaluation proof [Tha22, §10.2, p. 360]. Several compiler theorems make this precise. Marlin shows that any public-coin AHP combined with an extractable PCS yields a public-coin preprocessing argument with a universal SRS, that knowledge soundness carries over, that zero knowledge carries over when the PCS is hiding, and that the SRS is updatable whenever the PCS's is [CHM+20, Thm. 1, p. 5, Thms. 8.1, 8.3, 8.4 and Remark 8.2, p. 41]. DARK proves the analogue for polynomial IOPs, assuming the evaluation protocol has witness-extended emulation, and shows that a hiding PCS with an honest-verifier zero-knowledge evaluation preserves honest-verifier zero knowledge [BFS20, §1.1, p. 4]. PLONK proves the same directly for KZG in the algebraic group model [GWC19, Lemma 4.7, p. 17]. The commitment must match the PIOP: IP- and MIP-based protocols need a multilinear PCS, and univariate PIOPs a univariate one [Tha22, Ch. 19, p. 608].

Knowledge soundness. The extractor $\mathcal{E}$ of §3 is built in two layers. The PCS extractor recovers the polynomials behind the prover's commitments, and the PIOP's knowledge soundness turns them into a witness [Tha22, §7.4, pp. 307–308, for the GKR-based argument; CHM+20, Thm. 8.3].

How zero knowledge is obtained. Zero knowledge also needs both layers: hiding commitments and evaluation proofs, and revealed evaluations that say nothing about w. In every scheme the prover adds fresh randomness so that each value the verifier sees is either uniform or fixed by the verification equations, and the simulator of §3 samples that distribution without w. Groth16 adds random multiples of its setup trapdoor δ_{G} (Groth's δ, subscripted to keep it apart from the proximity parameter δ of §2.1) to two proof elements and corrects the third, so that real and simulated proofs have the same distribution and zero knowledge is perfect [Gro16, §3.2, p. 17, Thm. 1, p. 15, Thm. 2, p. 18]. PLONK's KZG commitments are not hiding; instead it adds random low-degree multiples of Z_H to

the prover's polynomials and draws challenges outside H, so that the encoded values are unchanged and every revealed value is "either completely uniform or determined by verifier equations" [GWC19, §8, p. 26, §8.3, p. 28]. Hash-based IOPs must be zero knowledge against a bounded number of queries, because a Merkle opening reveals codeword entries in the clear: Aurora and STARK mask with random higher-degree extensions and a random codeword [BCR+19, §7.1, pp. 30–31, §8.1, p. 37; BBHR18, App. D.1, p. 63], and with salted Merkle leaves the BCS compiler turns an honest-verifier zero-knowledge IOP into a statistically zero-knowledge argument [COS20, Thm. 10.1, p. 40]. Sum-check is masked by running it on $g + \alpha p$ for a committed random p, which can be a sum of one small univariate polynomial per variable [Tha22, §13.3, p. 455; XZZ+19, §4.1, p. 20]. Zero knowledge can also be added afterwards by commit-and-prove, but with Pedersen commitments each verifier multiplication can cost at least nine group elements, and post-quantum soundness is lost [Tha22, §13.4, pp. 460–461].

Not zero-knowledge as presented. The commitments that §4.4 and §9.1 mark this way (§3) would need such a step. Blaze proposes an extra proof-composition step [BCF+25, Remark 2, p. 5]. Bolt's authors find zero knowledge "at odds with having a systematic encoding" and suggest recursing into a zero-knowledge proof [GNR26, §1.3, p. 407]. Orion shows that masking a linear code is delicate: it salts its hashes and appends a random vector to each masked codeword, and its first mask was not zero-knowledge [XZS22, §2.2, p. 10, §4.3, fn. 4, p. 23].

Fiat–Shamir. The final step is the Fiat–Shamir transform of §2.3, which must hash the statement and the public parameters (strong Fiat–Shamir) to avoid the vulnerabilities of §3.1 [DMWG23]. Proving that the result is still sound is subtle. There are k-round arguments with soundness error ε whose Fiat–Shamir transform has error $m^{\Omega(k)} \cdot \varepsilon$ against adversaries making m oracle queries, so standard soundness "is not the target" [CGKY26, §1, p. 190]. The right target is *state-restoration soundness*, which implies Fiat–Shamir security. Round-by-round soundness also suffices, but it applies "only to interactive proofs (rather than arguments)" [CGKY26, §1, p. 190]. Chiesa, Guan, Knabenhans and Yu prove the general theorem for the PIOP-plus-PCS recipe: if the information-theoretic protocol is state-restoration sound and the commitment satisfies a state-restoration form of function binding, the compiled argument is state-restoration sound [CGKY26, Thm. 1, p. 192]. Their framework covers IOPs with vector commitments, linear PCPs with linear commitments, and PIOPs with polynomial commitments [CGKY26, abstract, pp. 189–192], and it re-derives PLONK's security from falsifiable assumptions (§5.2).

Where trust enters. As Figure 1 shows, the only trusted component of a PIOP-based zk-SNARK is the setup of its commitment scheme. In Bünz et al.'s words, the trusted portion "is entirely confined to the polynomial commitment scheme's setup" [BFS20, §1, p. 3], and the argument "will inherit the cryptographic and setup assumptions ... of the chosen polynomial commitment scheme" [Tha22, Ch. 19, p. 608]. KZG gives a universal, updatable but trusted $\mathcal{S}_1$; FRI, an IPA, or DARK with class groups gives a transparent one (Table 7.2; see also [GKM+18] on updatable reference strings). The random oracle is a second, separate assumption that enters only at the Fiat–Shamir step (§3.1).

Two routes outside the recipe. The linear-PCP schemes GGPR, Pinocchio and Groth16 compile a linear PCP whose queries are sampled during setup and hidden "in the exponent" of a pairing group. The route goes back to Ishai, Kushilevitz and Ostrovsky, who showed that a prover can commit to a linear PCP, a structured proof of exponential length, without writing it out [IKO07; Tha22, §17.1, p. 549]. The interactive argument built from a linear PCP hides the verifier's queries under an encryption key that only the verifier may know, so it is not public coin and "we cannot apply the Fiat-Shamir transformation" [Tha22,

§17.5.1, p. 566]; instead, "a trusted preprocessing step ... preselects the verifier's linear PCP queries" [BFS20, §1.1, p. 3]. This makes $\mathcal{S}_2$ circuit-specific (Remark 1) and gives the smallest proofs, as few as 3 group elements for Groth16 [Tha22, §19.2, p. 613] (§5.1).

The GKR-based schemes Hyrax, Libra and Virgo run sum-check layer by layer and commit only to the witness's multilinear extension [Tha22, §10.6, p. 395]. Their costs grow with circuit depth, and their verifier needs fast evaluation of the wiring predicates, so they meet the strict bound at most for restricted circuits (Table A.2, §5.4). Fiat–Shamir for such GKR-based arguments also admits practical attacks, for any concrete hash function, on adversarially constructed circuits [KRS25, abstract] (§4.2).

5 Construction Families of zk-SNARKs

We group the construction families by setup model (§3 Remark 1), since it fixes how $\mathcal{S}_1$ and $\mathcal{S}_2$ are filled and thus most of what must be trusted. Sections 5.1–5.3 cover the circuit-specific, universal and transparent families, which are the strict zk-SNARKs of Table 7.1. Section 5.4 covers the arguments outside the strict definition (Table A.2). HyperPlonk (§4.2, §9.1), Orion (§4.4, §9.1) and Plonky2 (§6.4, §8), rows that the tables add for later work, are discussed with their components instead. Recursion and folding, which are built on top of a base scheme, follow in Section 6. Figure 4 arranges the same schemes by information-theoretic route (§4), and Figure 7 in Appendix A by date.

5.1 Pairing-based / QAP — GGPR, Pinocchio, Groth16

This family compiles a *linear PCP*, in which the honest proof is a linear function (§4.5) [IKO07; Tha22, §17.1, p. 549], through pairings with a circuit-specific SRS. GGPR instantiate it with the quadratic span program (QSP) and the quadratic arithmetic program (QAP) of §4.1, which turns circuit satisfiability into a single divisibility check with the quotient $h(X)$ as evidence [GGPR13, §1]. The QAP approach is what most later implementations refine [Gro16, §1, p. 2].

GGPR and Pinocchio

GGPR's CRS encodes the QAP polynomials and the powers of a secret point τ (GGPR's s; it plays the role of the SRS secret of §4.4) as group elements such as $g^{v_k(\tau)}$. The prover uses them to compute encodings of the three combinations and of $h(\tau)$. The verifier checks the identity at τ with a pairing; in GGPR's simplified description of the QSP version it confirms $e(g^v, g^w) = e(g^h, g^{t(\tau)})$, and the QAP version adds the third combination y [GGPR13, §1.2, §8]. Knowledge soundness comes from the q-PKE assumption of §3.1, which forces the prover to build its encodings from the CRS [GGPR13, §5.1]. GGPR prove the QAP-based zk-SNARK secure under the q-power Diffie–Hellman assumption (q-PDH), which is falsifiable, and d-PKE (q-PKE at degree d), which is not. The theorem requires $q \geq \max\{2d-1, d+2\}$, where d is the QAP degree [GGPR13, §8.1, Thm. 14]. A proof has 9 group elements (Table A.1).

Pinocchio [PHGR13] turns this scheme into "a built system" [PHGR13, abstract]. It shrinks the proof "from nine group elements to eight" [PHGR13, §3, p. 5], which is 288 bytes. Its authors call it "the first general-purpose system to demonstrate verification cheaper than native execution (for some apps)" [PHGR13, abstract].

Groth16

Groth16 [Gro16] redesigns the argument so that a proof is only three group elements (§4.5). Verification checks one pairing-product equation with three pairings, after ℓ exponentiations for the statement [Gro16, abstract, §3.2]. A proof is 128 bytes on BN-256 [CHM+20,

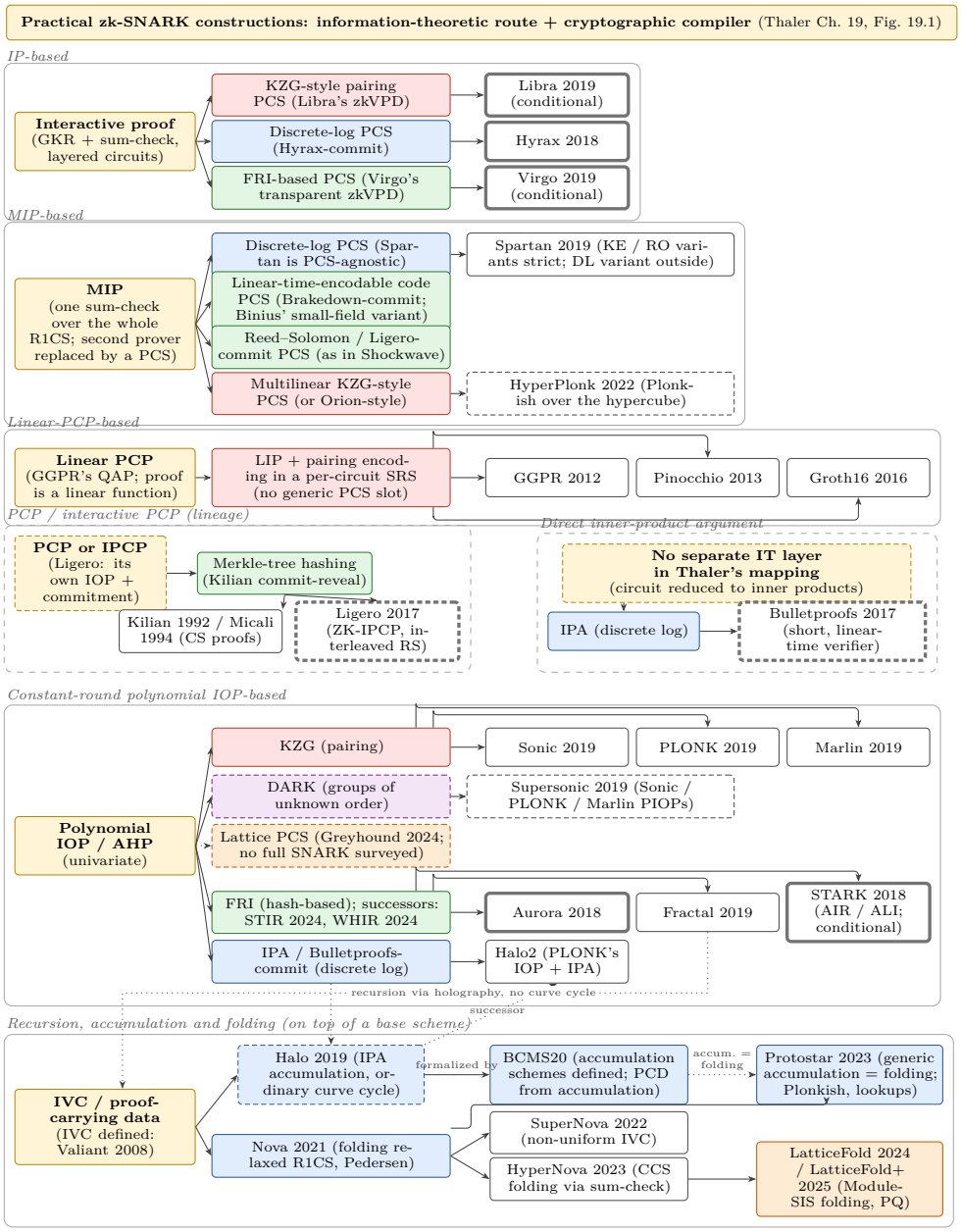


Figure 4: Taxonomy of zk-SNARK construction families. Practical zk-SNARKs as an information-theoretic protocol plus a cryptographic compiler, following Thaler's Ch. 19 taxonomy [Tha22, §19.1, Fig. 19.1, Halo2: §19.1(f), p. 610]. The IP-, MIP- and constant-round polynomial-IOP-based routes can each be combined with any polynomial commitment scheme (multilinear for IP/MIP, univariate for PIOPs). The commitment, not the IOP, decides whether the result is transparent and whether it is plausibly post-quantum. The linear-PCP route (GGPR, Pinocchio, Groth16) has no generic commitment slot. It compiles directly through pairings with a circuit-specific SRS. The classical PCP route (Kilian, Micali) is shown for lineage. Ligero is placed by its own framing as a zero-knowledge interactive PCP [AHIV17]; Thaler lists it as a constant-round polynomial IOP with its own commitment [Tha22, §19.1(g)]. Recursion, accumulation and folding sit on top of a base scheme, not inside the taxonomy (Figure 6). Apart from the lineage route (Kilian, Micali) and the recursion layer, only zero-knowledge schemes are drawn. A thick grey border marks zero-knowledge arguments that miss the strict succinctness bound of §3, or meet it only for restricted circuits (marked "conditional"); see Table A.2. Brakedown, Shockwave and Binius are not zero-knowledge as presented, so their codes appear only as commitment-scheme leaves. Colours as in Figure 1; a dashed border marks an uncertain placement. In the STARK leaf, ALI is STARK's algebraic linking interactive oracle proof [BBHR18, §2.3].

Fig. 1]. Groth16 beats Pinocchio on every cost in Groth's comparison [Gro16, Table 2]. In the linear-PCP view of §4.5, Thaler attributes the three elements to two changes: a linear interactive proof with three queries instead of five, and the generic group model [Tha22, pp. 577–578; BCI+13]. Its zero knowledge is perfect (§4.5), and its knowledge soundness is statistical against adversaries that use generic bilinear-group operations [Gro16, Thm. 2]. Groth calls relying on this model "an aggressive stance" and justifies it partly by the Gentry–Wichs barrier of §3.1 [Gro16, §1.1].

Groth also proves that linear interactive proofs cannot have a linear decision procedure [Gro16, §4.1, Thm. 3; BCI+13]. It follows that, for relations with hard decisional problems, a pairing-based non-interactive argument with a generic prover and verifier needs proof elements in both $\mathbb{G}_1$ and $\mathbb{G}_2$, and so at least two group elements [Gro16, §4.2, Thm. 4, Cor. 2]. Groth calls this "the first lower bound for pairing-based SNARGs" [Gro16, abstract]; §9.1 discusses a two-element SNARG in an extended model. Polymath shows that the bound counts group elements, not bits. Using a square arithmetic program, KZG and Fiat–Shamir, its proof is $3\,\mathbb{G}_1 + 1\,\mathbb{F}$, which is 1408 bits on BLS12-381 against Groth16's 1536, and 1792 against 3072 at the 192-bit level [Lip24, abstract, Table 1, p. 175]. It needs a random oracle, where "no non-trivial lower bound is known", is proven in the algebraic group model with oblivious sampling, and still has a circuit-specific SRS [Lip24, §1, pp. 170, 174, §3, p. 188].

Setup and assumptions

All three schemes have a circuit-specific trusted setup. In the terms of Remark 1, $\mathcal{S}_1$ is empty. $\mathcal{S}_2$ deterministically preprocesses C into its QAP and then samples secret trapdoors, and their encodings form $\mathsf{S_p}$ and $\mathsf{S_v}$. For Groth16 the trapdoors are $\alpha_G, \beta_G, \gamma_G, \delta_G$ (Groth's $\alpha, \beta, \gamma, \delta$, subscripted, as δ_G is in §4.5, to keep them apart from the challenges of §4.1, §4.4 and §4.5 and the proximity parameter δ) and the point τ (Groth's x) [Gro16, §3.2]. (GGPR's function-specific stage also samples fresh secrets [GGPR13, §3.1], so GGPR is circuit-specific however its two generation stages are mapped.) The parameters have size $O_\lambda(|C|)$ (Table 7.1), and anyone who keeps the trapdoor can forge proofs. Deployments therefore use multi-party ceremonies, which are secure if one participant is honest [MBKM19, §1]. In Zcash, for example, a compromised setup would let "an attacker ... create counterfeit money without detection" [MBKM19, §1]. Yet "any modification in the circuit ... requires another cryptographic ceremony" [CHM+20, §1], and the SRS cannot be updated (§5.2) [Tha22, §19.2, pp. 613–614]. The assumptions are non-falsifiable or idealized, as §3.1 explains. The pairing-based compilation also "appears to rely heavily on" the verifier computing a degree-2 function, so it does not extend easily to higher-degree gates [Tha22, §19.3.3, p. 627].

Section 7 compares this family with the others (the GGPR, Pinocchio and Groth16 rows of Tables 7.1 and A.1).

5.2 Universal and Updatable Setup — Sonic, PLONK, Marlin

The universal family removes the ceremony per circuit. In the terms of Remark 1, $\mathcal{S}_1$ runs once, with secret randomness, and outputs an SRS for all circuits up to size N; $\mathcal{S}_2$ is deterministic. Groth et al. [GKM+18] made the SRS *updatable* as well: security holds "if at least one of the users updating the CRS is honest" [GKM+18, abstract]. A CRS that encodes secret non-monomial polynomials cannot be updatable, because updates reveal the monomials. They demonstrate this on Pinocchio and believe it extends to most QSP/QAP-based SNARKs [GKM+18, §1, p. 3, §6, p. 29]. Their own QAP-based zk-SNARK has a monomial-only universal CRS of quadratic size, which is specialized into linear-size CRSs for particular relations [GKM+18, abstract]. The obstruction is therefore the content of the CRS, not the arithmetization. A quadratic CRS is costly, though:

for a Zcash circuit with 2^{17} multiplication gates it "would be on the order of terabytes" [MBKM19, §1, p. 2]. The schemes below use a linear-size SRS of encoded powers of a secret point, as in KZG [KZG10]; Sonic and Marlin also include these powers scaled by a second secret.

Sonic

Sonic [MBKM19] has a linear-size universal and updatable SRS that needs no specialization for a given circuit [MBKM19, abstract, §1.1]. Marlin's authors call it "the first efficient construction" of this kind [CHM+20, §1]. Sonic encodes the constraint system in a bivariate polynomial and commits with a KZG variant proven secure in the algebraic group model. It is made non-interactive in the random oracle model [MBKM19, §1.1, §3]. The verifier needs the value of a sparse bivariate polynomial that describes the circuit. In the *unhelped* (succinct) mode, the prover proves that value correct with new permutation and grand-product arguments. Marlin's authors compare against this mode because it is a preprocessing zk-SNARK, while the helped mode "does not achieve succinct verification" [CHM+20, §1.1]. It is Sonic's row in Table 7.1: proofs of $20\,\mathbb{G}_1 + 16\,\mathbb{F}$ and $273n$ prover exponentiations (Table A.1). In the *helped* mode, an untrusted helper computes advice for a batch of proofs [MBKM19, §1.1]. Only the helped mode was implemented, with 256-byte proofs [MBKM19, §9, Table 4].

PLONK

PLONK [GWC19] keeps this setup but simplifies the arithmetization to the gate equation and copy constraints of §4.1, with wire values interpolated over a multiplicative subgroup H of roots of unity [GWC19, §6–7]. Its permutation argument for the copy constraints (§4.1) builds on Bayer and Groth [BG12], as Sonic's does, but runs over evaluations on H rather than coefficients of a bivariate polynomial [GWC19, abstract, §1.1]. The commitments are KZG [GWC19, §3.1; KZG10]. So $\mathcal{S}_1$ is the KZG SRS, and $\mathcal{S}_2$ commits to the selector and permutation polynomials (Table 3.1). Knowledge soundness is proven in the algebraic group model under 2d-DLOG [GWC19, Lemma 4.7], with a random oracle for Fiat–Shamir [GWC19, §8]. Proofs have $9\,\mathbb{G}_1 + 6\,\mathbb{F}$ elements in PLONK's fast-prover flavour ($7\,\mathbb{G}_1 + 6\,\mathbb{F}$ in its small-proof flavour), and verification takes 2 pairings (Table A.1; [GWC19, §1.2, Tables 1–2, p. 4]). Counting group exponentiations only, the prover does about 1.1 to 3 times Groth16's work and 5 to 30 times less than succinct Sonic's [GWC19, §1.2].

PLONK's analysis has since been moved out of the idealized group model. Lipmaa, Parisella and Siim prove that interactive PLONK is computationally special-sound under ARSDH and SplitRSDH, two falsifiable assumptions. Its Fiat–Shamir version is therefore knowledge-sound in the random oracle model under the same assumptions, without the algebraic group model [LPS25, abstract]. SplitRSDH itself is justified in the AGM with oblivious sampling [LPS25, abstract]. Chiesa et al. re-derive the result modularly [CGKY26, Cor. 1, p. 195]. PLONK's multilinear descendant HyperPlonk [CBBZ23] is discussed in §4.2 and §9.1.

Marlin

Marlin [CHM+20] makes the modular structure explicit. Its core is an algebraic holographic proof (AHP) for R1CS (§4.2), and its compiler (§4.5) turns the AHP and KZG into a preprocessing argument with a universal, updatable SRS [CHM+20, §1, Thm. 1]. The indexer is "a deterministic polynomial-time algorithm" [CHM+20, §7]. Marlin is therefore preprocessing but not circuit-specific, which is exactly the distinction drawn in Remark 1. A proof has $13\,\mathbb{G}_1 + 8\,\mathbb{F}$ elements, 704 bytes on BN-256 as printed in Marlin's Figure 1 (Table A.1; [CHM+20, Figs. 1–2, §1.1]). Marlin reports a prover more than 10 times faster

than Sonic's and a verifier more than 3 times faster [CHM+20, §1.1]. These speedups compare complexity, not measured times, because no implementation of unhelped Sonic existed [CHM+20, Fig. 2].

All three schemes are proven secure in the algebraic group model of §3.1 (Marlin can also be based on concrete knowledge assumptions [CHM+20, abstract]).

Compared with §5.1, the family gives up proof size for one reusable, updatable setup: proofs have 15 to 36 group and field elements, against Groth16's 3 group elements (Table A.1). It is still pairing-based, so it is not post-quantum.

5.3 Transparent — Fractal, Spartan, Supersonic (DARK)

A zk-SNARK is *transparent* if neither $\mathcal{S}_1$ nor $\mathcal{S}_2$ uses secret randomness (Remark 1), so there is no trapdoor whose leakage would allow forged proofs. Transparency is decided by the cryptographic compiler, not by the information-theoretic protocol (Figure 4). PLONK's polynomial IOP, for example, needs a universal trusted setup with KZG but is transparent with a class-group commitment (Supersonic, below). This subsection follows the three transparent compilers that give strict zk-SNARKs in Table 7.1: hash-based IOPs (Fractal), sum-check with a transparent multilinear commitment (Spartan$_{\mathrm{RO}}$), and groups of unknown order (Supersonic with class groups). Halo and Nova are also transparent. Recursion is orthogonal to the setup model, so they appear in §6.

Hash-based IOPs: FRI and Fractal

The hash-based route compiles an interactive oracle proof (IOP) with Merkle-tree commitments and Fiat–Shamir (§4.2). It needs no algebraic structure. The interactive argument rests on collision-resistant hashing, and the non-interactive one on the random oracle (Table A.1). Its low-degree test is FRI [BBHR18b] or one of its successors, STIR and WHIR (§4.4), which make proofs and verification polylogarithmic.

STARK [BBHR18] has a polylogarithmic verifier only for uniform computations, so we discuss it with the conditional schemes in §5.4.

Fractal [COS20] meets the strict bound for arbitrary R1CS through *holography* (§4.2): the verifier queries an encoding of the circuit that an indexer computes once, instead of reading the circuit [COS20, §1.1, p. 2]. Fractal compiles any holographic IOP into a preprocessing SNARG that is "unconditionally secure in the random oracle model" and in the quantum random oracle model [COS20, Thm. 1, p. 2]. In Remark 1's terms, $\mathcal{S}_1$ is empty apart from the oracle. $\mathcal{S}_2$ is the deterministic indexer, whose output, a Merkle root, is an $O_\lambda(1)$ verification key: "anyone can publicly preprocess an index" [COS20, Thm. 3, p. 3]. Proofs are $O_\lambda(\log^2|C|)$, and verification takes time $O_\lambda(|x|+\log^2|C|)$ (Table 7.1). At 128-bit security over a 181-bit prime field, proofs are 80–160 KB for up to millions of constraints, two orders of magnitude larger than pre-quantum preprocessing zk-SNARKs with trusted setup [COS20, p. 3]. Holography also makes Fractal recursion-friendly. Preprocessing simplifies recursive composition [COS20, §2.1], and with no elliptic curves there is no field mismatch for a curve cycle to resolve (§6). Recursing requires replacing the oracle with a concrete hash, which the authors call "a heuristic" [COS20, p. 3, fn. 3].

Proven and conjectured soundness of FRI

Concrete FRI parameters depend on which soundness bound one trusts; Figure 5 summarizes the regimes discussed here. Theorem 2 of [BBHR18b] proves soundness only up to a proximity parameter $\delta_0 \geq \frac{1}{4}(1-3R) - 1/\sqrt{|\mathcal{L}|}$ for rate R and block length $|\mathcal{L}|$ (§2.1). The authors conjecture that it extends to every $\delta \leq 1-R$, i.e. up to capacity [BBHR18b, Conj. 3, p. 14:5]. Later work proved soundness arbitrarily close to the Johnson radius $1-\sqrt{R}$ [BCIKS20; Tha22, Thm. 10.4, p. 383]. Parameters that assume the conjecture

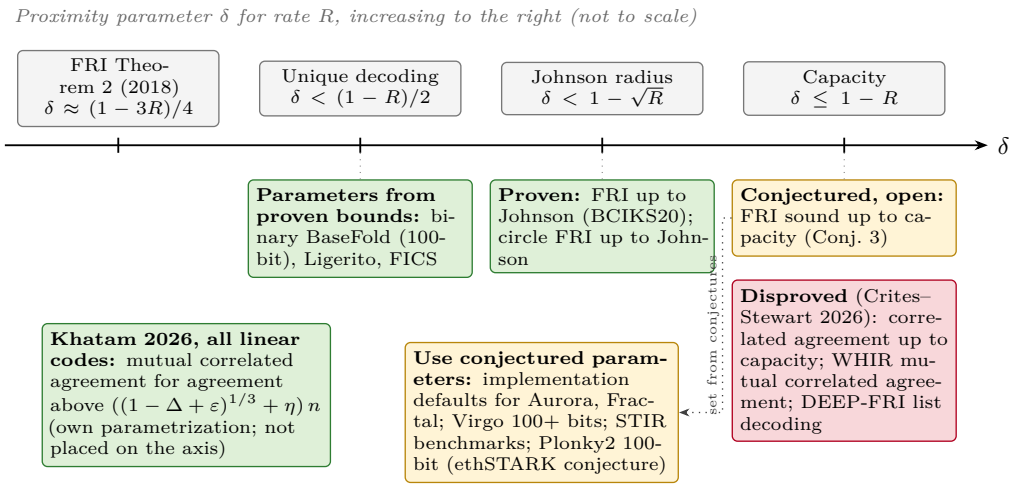


Figure 5: What is proven, conjectured and disproved about Reed–Solomon proximity. The top row orders the proximity parameter δ for rate R, from FRI's original proven bound through the unique-decoding and Johnson radii to capacity. Green results are proven, amber are conjectured and open, and red were disproved by Crites and Stewart [CS26]. Schemes on the left set concrete parameters from proven bounds; the bottom-right box lists results that use conjectured parameters, whose conjectures differ in detail. Khatam's bound holds for every linear code, not only Reed–Solomon codes, and is stated in its own parametrization. Sources: [BBHR18b, Thm. 2, Conj. 3], [BCIKS20], [HLP24, Thm. 6], [DP26, §5.2], [NA25, §6.3], [BMMS26], [Zei26, Thm. 1], [CS26], [Set20, §9.1], [ZXZS20, §5.2], [ACFY24, §6.2], [Pol22, §8].

need far fewer queries. According to Spartan, the default implementation parameters of Aurora and Fractal "rely on non-standard conjectures related to Reed-Solomon codes" and use about 10× fewer FRI query repetitions than the proven ones [Set20, §9.1, p. 33]. Virgo's "100+ bits" also rests on the conjecture [ZXZS20, §5.2].

Crites and Stewart [CS26] have since disproved the correlated-agreement up-to-capacity conjecture of [BCIKS20], as well as the up-to-capacity versions of WHIR's mutual-correlated-agreement conjecture and of the list-decoding conjecture for DEEP-FRI (a FRI variant that samples outside the evaluation domain); the latter, they note, already fails by a classical bound of Elias [CS26, abstract, p. 128, p. 129]. FRI's soundness "is analyzed under the assumption of correlated agreement" [CS26, p. 129], so this closes one route to proving FRI's conjecture. Crites and Stewart do not address FRI's own Conjecture 3, which remains unproven, and whether "a polynomial-time attack on FRI-based SNARKs" exists in this regime is open [CS26, p. 131]. They also show that correlated agreement with small enough error implies list decoding of Reed–Solomon codes [CS26, abstract, p. 128]. Security claims based on conjectured parameters are therefore conditional. At Virgo's parameters, proven bounds give fewer than 100 bits, by our estimate (Table A.1).

On the proven side, the gap is narrowing for multilinear evaluation. Zeilberger proves a relaxed form of the mutual-correlated-agreement property that BaseFold and WHIR rely on, for every linear code over every finite field, but only for agreement sets larger than about a $(1-\Delta)^{1/3}$ fraction of the block, for relative distance Δ, which is well short of capacity [Zei26, abstract, Thm. 1, Remark 1]. Even this bound roughly halves BaseFold's verifier repetitions at low rates [Zei26, Table 1, p. 608].

Baweja et al.'s FICS, an IOPP for linear-time-encodable codes with an $O(n)$ prover and $O(\lambda \log\log n + \log n)$ queries, is likewise analyzed within the unique-decoding radius [BMMS26, Thm. 3, p. 571, §2.2, p. 568, §2.7, fn. 10, p. 577].

The sum-check route: Spartan

Spartan [Set20] encodes an R1CS instance as a degree-3 polynomial that decomposes into multilinear polynomials, and runs the sum-check protocol on it [Set20, §1, p. 4]. In Thaler's taxonomy, a multilinear commitment replaces the second prover of an MIP (Figure 4). Checking the final sum-check claim would cost the verifier $O(n)$ for a statement of size n, because it must evaluate polynomials that encode the R1CS matrices $\mathbf{A}$, $\mathbf{B}$, $\mathbf{C}$ (§4.1). Spartan's *computation commitments* move this to a one-time public preprocessing step. The verifier keeps a short commitment to those sparse polynomials, and "creating a computation commitment does not involve any secret trapdoors" [Set20, §1, p. 4]. This is a deterministic $\mathcal{S}_2$ in Remark 1's terms. Spark, a compiler built on offline memory checking (§4.3, where it is written Spark, as in Lasso), then turns any extractable multilinear commitment into one for sparse polynomials with a time-optimal prover [Set20, §1, p. 4]. Spartan is a family, and the commitment decides where each variant falls [Set20, Fig. 2, p. 6]:

- $\text{Spartan}_{\text{RO}}$ is hash-based, plausibly post-quantum, and strict and transparent ($O_\lambda(\log^2|C|)$ proofs and verification, Table 7.1).
- $\text{Spartan}_{\text{KE}}$ is strict, but its "q-type, knowledge of exponent" commitment needs a universal private setup [Set20, §1, p. 5]. By its setup it belongs with the universal schemes of §5.2.
- $\text{Spartan}_{\text{DL}}$ uses Hyrax's discrete-log commitment and is transparent, but has $O_\lambda(\sqrt{|C|})$ proofs and verification (§5.4).

Groups of unknown order: Supersonic (DARK)

Bünz, Fisch and Szepieniec [BFS20] commit to polynomials by encoding them as integers in a group of unknown order. Their DARK commitment has $O(\mu \log d)$-size evaluation proofs for μ-variate degree-d polynomials, verifiable in $O(\mu \log d)$ time [BFS20, §1.1, p. 3]. Supersonic compiles the polynomial IOPs of Sonic, PLONK and Marlin with it [BFS20, §1.1, p. 4]. Whether the setup is trusted depends on the group. An RSA modulus "cannot be generated in a publicly verifiable way without exposing the order, and thus requires a trusted setup". The discriminant of a class group, by contrast, "can be generated from public coins" [BFS20, §3.1, pp. 8–9]. Supersonic is thus transparent with class groups, and a universal scheme with "constant-size setup parameters" with RSA groups [BFS20, §1.1, p. 3]. Its $O_\lambda(\log|C|)$ proofs and verification are the best asymptotics among the transparent schemes of Table 7.1. The only concrete figures are estimates: 10.1 KB proofs and about 72 ms verification for 2^{20} gates at 120-bit security [BFS20, abstract, §1.1, §6, §7, Table 3].

Two caveats apply. First, the EUROCRYPT 2020 security proof had "a significant gap", found by Block et al. [BHR+21]. The revised version (June 2022) shows "the original protocol with a slightly adjusted parameter is still secure", with extraction relying on the random order assumption [BFS20, abstract, §1, p. 4, §3.1, p. 9]. The abstract and Theorem 5 still name the strong RSA and adaptive root assumptions [BFS20, Thm. 5, p. 25]. Later work confirms the issue. DARK's attempt to avoid range proofs for degree bounds "was shown to have security vulnerabilities", and its successors restored security with range proofs at the cost of an $O(n^3)$ prover for polynomials of degree less than n [PTX26, §1.2, fn. 4]. Morphic accumulators bring that prover back to $O(n)$ with $O(1)$ proofs, but their instantiation needs an RSA setup and is not transparent [PTX26, Table 3]. DewTwo, a later commitment in groups of unknown order, is transparent and avoids the generic group model, but its prover is impractical today (§4.4) [BMSS25]. Second, Supersonic "is not quantum-secure due to its reliance on groups of unknown order" [BFS20, §1.1, p. 4].

Post-quantum security separates the three routes. Hash-based schemes are *plausibly* post-quantum: Spartan lists Ligero, Virgo, STARK, Aurora, Fractal and Spartan$_{\mathrm{RO}}$ as such [Set20, Fig. 2 caption, p. 6]. The qualifier matters. STARK's claim rests informally on hashing, and its authors note that this "covers only the interactive setting" [BBHR18, §1.3.1, fn. 12, p. 9]. Fractal proves security in the quantum random oracle model [COS20, Thm. 1, p. 2], but that says nothing about the concrete hash. Discrete-log schemes (Spartan$_{\mathrm{DL}}$, Bulletproofs, Halo, Nova) and Supersonic are not post-quantum.

5.4 Zero-Knowledge Arguments Outside the Strict Definition

Several systems that call themselves zk-SNARKs under the looser notions of succinctness quoted in §1, among them Hyrax and Aurora, are zero-knowledge arguments whose verifier is not polylogarithmic in $|C|$, or is polylogarithmic only for restricted circuits. Table A.2 lists them with the reason each misses the bound: a linear verifier (Bulletproofs, Aurora, Ligero), sublinear but not polylogarithmic proofs and verification (Hyrax, Spartan$_{\mathrm{DL}}$), or a polylogarithmic verifier only for uniform computations (STARK) or for low-depth, log-space uniform circuits (Libra, Virgo) [BBB+17, abstract; BCR+19, abstract; AHIV17, abstract; WTS+18, abstract; Set20, Fig. 2, p. 6; BFS20, §1, fn. 1, p. 2; XZZ+19, abstract; ZXZS20, abstract]. Many have what strict schemes lack: very fast provers, short transparent proofs, or plausible post-quantum security. Circle STARKs carry STARK's design to Mersenne-31 (§4.1), and their FRI analogue is proven sound up to the Johnson radius in the oracle model [HLP24, Thm. 6, p. 38]. LaBRADOR (§4.4) also belongs here, with very short proofs but a verification time that is not sublinear [BS23b, §1, p. 520]; it is not zero-knowledge as presented in its main construction, and zero knowledge is added by a separate, costed variant [BS23b, §6, pp. 539–541]. The GKR-based schemes carry the Fiat–Shamir attack of §4.2: Khovratovich, Rothblum and Soukhanov target the argument used in Hyrax and Libra [KRS25, §1, p. 3 and fn. 3], and Virgo has the same structure, so we infer that it is affected too (§9.4). Brakedown and Shockwave are excluded, since "neither Shockwave's nor Brakedown's implementations are currently zero-knowledge" [GLS+23, §1.1, p. 5]; their codes appear as polynomial commitments in §4.4 (Table 7.2).

6 Recursion, Accumulation and Folding

The families of §5 differ in their setup. Recursion is orthogonal to that choice: it is a layer on top of a base argument.

Halo and Nova are transparent: neither needs a trusted setup [BGH19, abstract; KST22, abstract]. Pairing-based SNARK recursion and SnarkPack aggregation do need one (§6.1, §6.4). We include these constructions, although Nova is an IVC scheme rather than a zk-SNARK, for two reasons. Recursion is how zk-SNARKs are applied to long or unbounded computations. And the recursion ends in a proof that does meet §3: Nova's compressed proof in the SXDH variant is strict per step, with the step circuit F in place of C, and its cost is independent of the number of steps (Table A.2).

6.1 Incrementally verifiable computation

Valiant defined *incrementally verifiable computation* (IVC): a long computation periodically outputs its state with a short proof of correctness, the prover stays close to the computation's own time and space, and the verifier's cost is "essentially constant" [Val08, abstract, §1, pp. 1–2]. His construction merges proofs recursively. Proofs of knowledge compose ("A is convinced that B is convinced"), whereas merely sound proofs only assert that a convincing proof exists [Val08, §1.3]. SNARK-based recursion proves "the t-th step ... was executed correctly, and there exists a proof" for $t-1$ steps, with the SNARK verifier inside the

circuit [BCMS20, §1, p. 2] [BCCT13] [BCTV14]. This requires a verifier sublinear in its statement, which otherwise grows exponentially in t. It also requires verifier arithmetic native to the circuit's field, which for pairing-based SNARKs meant cycles of pairing-friendly curves [BCMS20, §1, pp. 2, 5], "an expensive algebraic object" [COS20, abstract]. The Fiat–Shamir hash must then be computed inside the circuit, which makes security heuristic (§3.1).

Pairing-friendly cycles remain scarce: at 128-bit security the MNT cycle needs $p \approx q \approx 2^{1000}$ by Guillevic's estimate, and the first new pairing-friendly cycles in a decade, from higher-dimensional abelian varieties, are suboptimal compared with an ideal 2-cycle [CCN24, abstract, §1, pp. 222–225].

The in-circuit hash is also provably hard to avoid. Hall-Andersen and Nielsen prove Valiant's conjecture under mild extra conditions: if collision-resistant hash functions exist, no zero-knowledge, knowledge-sound, succinct non-deterministic IVC exists in the random oracle model, even if its security also uses standard-model assumptions, provided the zero-knowledge simulator only programs the oracle [HN23, Thm. 1, Remark 3, p. 21]. Barbara, Chiesa and Guan show that no succinct argument in the random oracle model can prove computations that themselves query the oracle, so relativized IVC and proof-carrying data (PCD, a generalization of IVC; §6.2) do not exist there either [BCG25, abstract, §1.1]. Whether ordinary IVC exists in the random oracle model without Hall-Andersen and Nielsen's extra conditions is open [BCG25, §1.1, p. 5].

6.2 Halo and accumulation schemes

Halo [BGH19] removes both the trusted setup and the pairing-friendly cycle. It uses a Bulletproofs-style inner-product commitment under the discrete-log assumption over a cycle of ordinary 255-bit curves [BGH19, abstract, §1.1]. The commitment's verifier has one linear-time step. *Nested amortization* keeps that step out of the recursive circuit: its input and claimed output become public inputs, and successive claims are merged, so the linear-time check "is thus only performed once at the end of the recursive chain" [BGH19, §1.1, pp. 3–4]. Fully-recursive proofs are 3.5 KiB [BGH19, §1.2, p. 4]. The cost is the final check: Halo is "not fully succinct (in that the verifier's work is linear in the circuit size)" [BGH19, §1.2, p. 4]. Halo's "fully succinct" refers to the verifier, not to §3's parameter-size notion. By §3's bound, a Halo proof is outside the strict definition.

Halo's authors prove their SNARK secure but not its recursive technique: they "do not prove that it satisfies any security property" [BCMS20, abstract, §1, p. 2]. Bünz, Chiesa, Mishra and Spooner fill this gap. They define *accumulation schemes*, with a prover, a verifier and a decider, and show that any SNARK with a sublinear accumulation verifier yields PCD, which generalizes IVC, "even if the argument itself does not have a sublinear-time verifier" [BCMS20, abstract, Thm. 1, p. 3]. Their concrete accumulation schemes are proven in the random oracle model, so the resulting PCD is heuristic [BCMS20, §1.1, Thm. 3, pp. 3–5]. Halo's successor Halo2 pairs PLONK's IOP with an inner-product commitment [Tha22, §19.1] (Figure 4).

6.3 Folding: Nova

Nova [KST22] drops SNARKs from intermediate steps. A *folding scheme* "reduces the task of checking two instances in some relation to the task of checking a single instance" [KST22, abstract]. R1CS is quadratic, so a random combination of two R1CS instances is not an R1CS instance. Nova instead folds *relaxed* R1CS, $(\mathbf{A}Z) \circ (\mathbf{B}Z) = u \cdot (\mathbf{C}Z) + E$, with the R1CS matrices of §4.1 and Nova's assignment vector Z, where the scalar u and the error vector E absorb the cross terms ($u = 1$, $E = 0$ is plain R1CS). The folding scheme is zero-knowledge, with an $O_\lambda(N)$ prover for N-sized instances and $O_\lambda(1)$ verifier and communication, from any additively homomorphic commitment such as Pedersen's

[KST22, Thm. 1, p. 3]. Each IVC step folds the previous instance into a running one. The recursive circuit is then "dominated by two group scalar multiplications", and the prover's work per step by two multi-exponentiations of size $O(|F|)$ [KST22, abstract]. The IVC proof has size $O(|F|)$. A Spartan-based zk-SNARK compresses it to $O(\log|F|)$, verified in time $O(\log|F|)$ or $O(|F|)$ depending on the commitment [KST22, Thm. 2, p. 4]. The logarithmic case is the SXDH variant [KST22, Fig. 2, p. 7]. Nova needs "neither ... a trusted setup nor FFTs", and its Fiat–Shamir step is instantiated "(heuristically)" with a concrete hash [KST22, abstract, §1.1, p. 3].

The deployed system diverged from the analyzed one. The paper analyzes a single curve group, but the implementation ran a 2-cycle scheme that was "only described in the code". Nguyen, Boneh and Setty forged a proof of 2^{75} rounds of the MinRoot verifiable delay function in 1.46 s, and then gave a proven, more efficient fix that is now in use [NBS23, abstract]. They also show that Nova's IVC proofs are malleable, so an attacker could swap the payment address in a proven output [NBS23, §8, p. 20] (§3, Remark 3).

6.4 Generalizations

The folding line has been generalized in several directions, which Figure 6 arranges by the algebra each construction uses.

Over groups. SuperNova gives *non-uniform* IVC, in which a step costs only "the size of the circuit representing the instruction invoked" [KS22, abstract]. HyperNova folds CCS and adds zero knowledge through NovaBlindFold, and an efficient instantiation over a curve cycle through CycleFold [KS24, abstract]. Protostar compiles special-sound protocols into non-uniform IVC for PLONK with high-degree gates and lookups, and observes that its accumulation scheme "is also naturally a folding scheme as defined in Nova" [BC23, abstract, Remark 2, p. 20]. ProtoGalaxy folds k instances into an accumulator in one step, and beyond combining commitments its verifier performs $kd + \log n$ field operations and as many hash evaluations for degree-d constraints over n rows. Like Protostar, it is proven knowledge-sound only in the random oracle model and does not treat zero knowledge [EG23, §1.2, pp. 3–4, Table 1, p. 4, §3.3, p. 7]. NeutronNova keeps a constant-size recursive verifier, three group scalar multiplications and a constant number of hash computations, while folding many instances at once without Nova's cross-term commitments, and it folds two 2^{20}-constraint SHA-256 instances in 91 ms, against 891 ms for Nova, about 10× faster [KS26, abstract, §1, Fig. 2, p. 528]. Mangrove uses folding for tree-based PCD with a two-pass prover whose memory is $O(1)$ for suitable parameters; its estimated 34 MB proof shrinks to 12 KB after compression with Spartan [NDC+24, abstract, §1, p. 311, §6.3, p. 339]. Kothapalli and Parno's *reductions of knowledge* give the formal language in which folding is a reduction from two instances to one [KP23, Def. 1, Thms. 1–2, Def. 4, p. 680].

Over lattices and hashes. LatticeFold and LatticeFold+ replace Pedersen with Module-SIS (Ajtai) commitments, for plausibly post-quantum folding over 64-bit fields [BC25a, abstract; BC25b, abstract]. Neo and SuperNeo, lattice-based folding schemes for CCS, the relation HyperNova folds, move lattice folding to small fields such as Goldilocks [NS26, abstract] (§8). FACS gives an accumulation scheme for NP in the random oracle model with a linear-time prover [BMMS26, Thm. 2], and ARC removes the depth bound beyond which the first hash-based accumulation scheme's PCD can be attacked [BMNW25, abstract, §1.2]. Fractal's cycle-free route (§5.3) is deployed in Plonky2, which compiles an arithmetization based on TurboPLONK (PLONK with custom gates) with FRI over the 64-bit field $p = 2^{64} - 2^{32} + 1$ and reports a recursion threshold of 2^{12} gates, against Fractal's 2^{21} constraints [Pol22, abstract, §1, p. 3, §8, p. 14]. Its 100-bit setting relies on the ethSTARK soundness conjecture for FRI [Pol22, §8, fn. 8, p. 14], and the same team's

later Plonky3 is a toolkit for STARK-based zkVMs (§8). For Supersonic, by contrast, Halo's authors write that "it is not clear ... if recursion can be practically achieved" [BGH19, §1.2, p. 4].

Aggregation. When the goal is only to check many independent proofs at once, aggregation is a lighter alternative. SnarkPack combines n Groth16 proofs for the same verification key into one argument of size and verification time $O(\log n)$ using inner pairing product arguments, with keys taken from two existing powers-of-tau ceremonies. It aggregates 8,192 proofs in 8.7 s into a 40 KB proof [GMN22, abstract, §1, pp. 4, 6, §5, pp. 15–18]. Unlike recursion, this batches a fixed set of proofs rather than giving incrementally verifiable computation.

7 Comparison

This section compares the schemes of Section 5, Nova from Section 6, and five later schemes, discussed where their components appear (HyperPlonk in §4.2, Orion in §4.4, Polymath in §5.1, Plonky2 and HyperNova in §6.4). Only zero-knowledge schemes are compared, and every cell names its source. Table 7.1 lists the zk-SNARKs that meet the strict definition of §3, with every cost an $O_\lambda(\cdot)$ bound in the circuit size $|C|$, so that rows can be compared column by column, and Table 7.2 compares the polynomial commitment families of §4.4. Appendix A holds the zero-knowledge arguments that are often called zk-SNARKs but miss the strict bound, or meet it only for restricted circuits, with the reason for each (Table A.2), and the exact counts behind both tables in each paper's own units, with setup models, assumptions and post-quantum status (Table A.1). Brakedown and Shockwave have no rows, because they are not zero-knowledge as presented (§3, §5.4). Orion is in Table A.2 because its theorem gives only a linear verifier (§9.1); Plonky2 is there because its whitepaper states no bounds, and it is the one exception to the zero-knowledge rule, kept as the deployed instance of the cycle-free recursion route of §6.4 (§3); HyperNova is there, like Nova, because it is a folding scheme rather than a zk-SNARK. The rest of the work since 2022 mostly changed components rather than added schemes (§8), so it has no rows. We do not tabulate measured proof sizes and running times, because the papers report them on different hardware, curves, security levels and benchmark circuits.

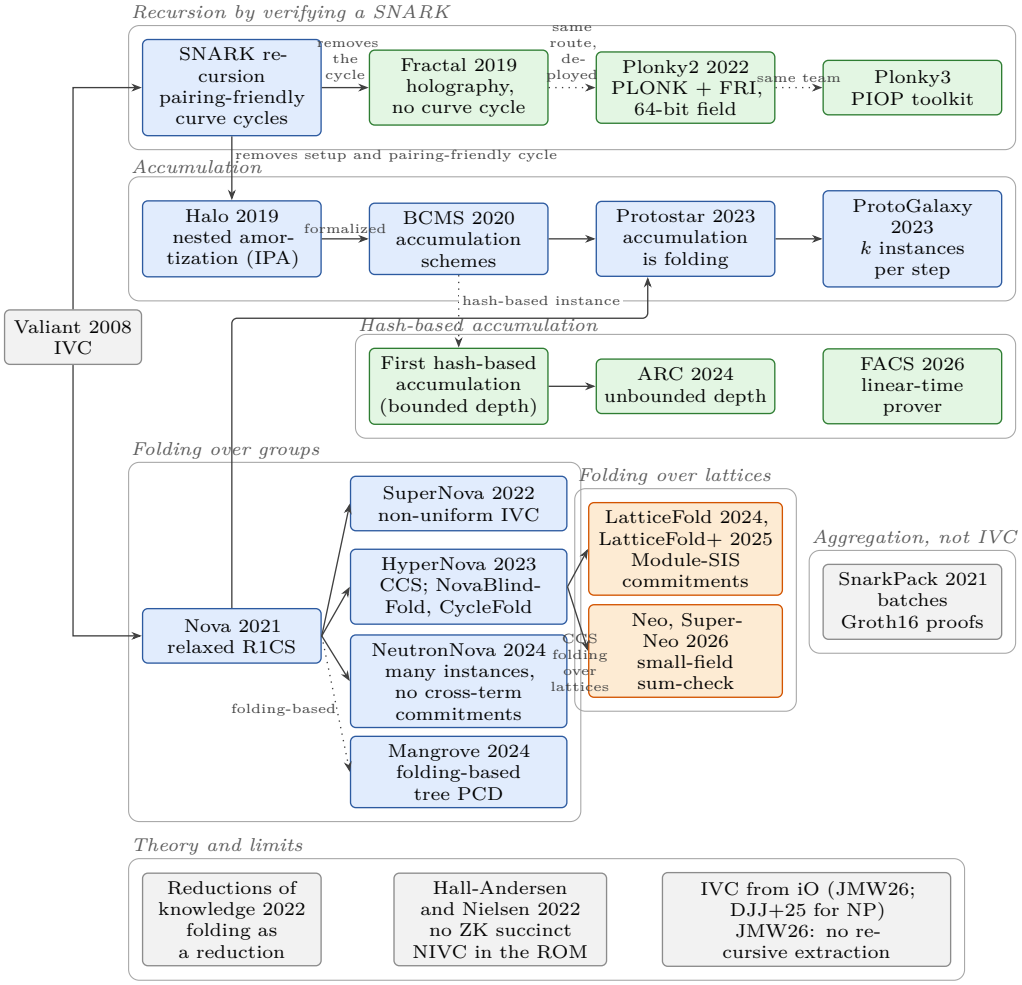


Figure 6: Recursion, accumulation and folding. Arrows mark a construction that generalizes, formalizes or replaces the earlier one; dotted arrows mark a looser relation named on the edge. Blue boxes are constructions over discrete-log or pairing groups (Figures 1 and 4 colour pairings red; here the two share one colour), orange boxes use lattices and green boxes hashes, the last two plausibly post-quantum; grey boxes are theory or batching rather than IVC constructions, and the iO-based IVC results are discussed in §9.3. Sources: [Val08], [BCMS20], [COS20], [Pol22], [Plonky3], [BGH19], [BC23], [EG23], [KST22], [KS22], [KS24], [KS26], [NDC+24], [BC25a], [BC25b], [NS26], [BMNW25], [BMMS26], [GMN22], [KP23], [HN23], [BCG25], [JMW26], [DJJ+25].

Table 7.1: **zk-SNARKs meeting the strict definition (costs in $|C|$).** Every entry is $O_\lambda(\cdot)$ in $|C|$, the number of gates or constraints of C, hiding poly(λ) factors; $|x|$, $|w|$, d and G are the statement size, witness size, circuit depth and width (Table 7.1 uses only $|x|$; the others appear in Table A.2, and where Table A.1 quotes a paper's own d, as for Sonic, HyperPlonk and HyperNova, it keeps that paper's meaning). "Setup params" counts what setup generates with fresh randomness (gp plus any randomized part of $\mathsf{S_p}$); deterministic encodings of C do not count, so a scheme is fully succinct exactly when this column is $O_\lambda(1)$. "Setup model; assumptions; PQ" gives the setup model of §5, the main hardness or idealized-model assumptions, and whether the scheme is plausibly post-quantum; Table A.1 states each in full with its source. "Derived" means computed from the paper's own counts; blank means the source gives no bound. For the universal schemes the SRS has size $O_\lambda(N)$ for the size bound $N \geq |C|$ of §3, written $O_\lambda(|C|)$ for $N = \Theta(|C|)$.

Scheme	Setup model; assumptions; PQ	Setup params (beyond C)	Proof size	Prover	Verifier	Notes	Source
GGPR	circuit-specific (trusted); q-PDH + d-PKE; not PQ	$O_\lambda(\|C\|)$	$O_\lambda(1)$	$O_\lambda(\|C\| \log \|C\|)$	$O_\lambda(\|x\|)$	verifier $O_\lambda(1)$ if the statement is hashed	[GGPR13, abstract]; [Set20, Fig. 2]
Pinocchio	circuit-specific (trusted); d-PKE, q-PDH, 2q-SDH; not PQ	$O_\lambda(\|C\|)$	$O_\lambda(1)$	$O_\lambda(\|C\| \log^2 \|C\|)$	$O_\lambda(\|x\|)$	prover entry from the $O(\|C\| \log^2 \|C\|)$ field work for $h(X)$ (Pinocchio p. 4); exact counts in Table A.1	[Gro16, Table 2]; [PHGR13, p. 4]
Groth16	circuit-specific (trusted); generic group model; not PQ	$O_\lambda(\|C\|)$	$O_\lambda(1)$	$O_\lambda(\|C\| \log \|C\|)$	$O_\lambda(\|x\|)$	exact counts in Table A.1	[Gro16, Table 2]; [CHM+20, Fig. 1]
Sonic	universal, updatable; AGM + ROM; not PQ	$O_\lambda(\|C\|)$	$O_\lambda(1)$	$O_\lambda(\|C\| \log \|C\|)$	$O_\lambda(\|x\| + \log \|C\|)$	unhelped (succinct) mode; the deterministic index proving key is not counted	[MBKM19, Table 2]; [GWC19, Table 1]; [CHM+20, Fig. 1]
PLONK	universal, updatable; AGM + q-DLOG + ROM, or falsifiable ARSDH + SplitRSDH in the ROM; not PQ	$O_\lambda(\|C\|)$	$O_\lambda(1)$	$O_\lambda(\|C\| \log \|C\|)$	$O_\lambda(\|x\| + \log \|C\|)$	verifier **derived**: $Z_H(z) = z^n - 1$ costs $O(\log n)$ and $\mathrm{PI}(z)$ sums ℓ terms	[GWC19, §1, Table 1, §8]
Marlin	universal, updatable; AGM + ROM; not PQ	$O_\lambda(\|C\|)$	$O_\lambda(1)$	$O_\lambda(\|C\| \log \|C\|)$	$O_\lambda(\|x\| + \log \|C\|)$	universal SRS for the size bound $N \geq \|C\|$ of §3; the deterministic index proving key is not counted	[CHM+20, Fig. 1, Thm. 2]
Fractal	transparent; ROM; plausibly PQ	$O_\lambda(1)$	$O_\lambda(\log^2 \|C\|)$	$O_\lambda(\|C\| \log \|C\|)$	$O_\lambda(\|x\| + \log^2 \|C\|)$	verifier bound after a transparent, deterministic preprocessing of C	[COS20, Thm. 3]
$\mathrm{Spartan}_{\mathrm{KE}}$	universal (trusted); q-type knowledge of exponent; not PQ		$O_\lambda(\log^2 \|C\|)$	$O_\lambda(\|C\|)$	$O_\lambda(\|x\| + \log^2 \|C\|)$	universal *private* setup (q-type, knowledge-of-exponent PCS); its size is not stated	[Set20, Fig. 2]
$\mathrm{Spartan}_{\mathrm{RO}}$	transparent; ROM (hash-based PCS); plausibly PQ	$O_\lambda(1)$	$O_\lambda(\log^2 \|C\|)$	$O_\lambda(\|C\| \log \|C\|)$	$O_\lambda(\|x\| + \log^2 \|C\|)$	setup **derived**: Spartan's only hash-based (CRHF) commitment option is Virgo-VPD, whose setup is the hash function alone	[Set20, Fig. 2, Fig. 4, p. 20]

continued on next page

Table 7.1, continued

Scheme	Setup model; assumptions; PQ	Setup params (beyond C)	Proof size	Prover	Verifier	Notes	Source
Supersonic (DARK)	transparent with class groups, trusted with RSA; strong RSA + adaptive root; not PQ	$O_\lambda(1)$	$O_\lambda(\log\|C\|)$	$O_\lambda(\|C\|\log\|C\|)$	$O_\lambda(\|x\|+\log\|C\|)$	class groups transparent, RSA trusted; univariate PIOPs ($\mu=1$) in the $O(\mu\log d)$ bounds	[BFS20, §1, §4.3]; [Set20, Fig. 2]
HyperPlonk (multilinear KZG)	universal (trusted); those of the PCS + ROM; not PQ		$O_\lambda(\log\|C\|)$	$O_\lambda(\|C\|)$	$O_\lambda(\|x\|+\log^2\|C\|)$	universal SRS, size not stated; prover linear for a fixed gate degree; verifier **derived** upper bound; with the FRI-based PCS the setup is transparent but the prover quasi-linear	[CBBZ23, §1, Thm. 4.1, Table 2, §3.8, Cor. A.4]
Polymath	circuit-specific (trusted); AGM with oblivious sampling, PDL + TOFR, ROM; not PQ	$O_\lambda(\|C\|)$	$O_\lambda(1)$	$O_\lambda(\|C\|\log\|C\|)$	$O_\lambda(\|x\|\log\|x\|)$	per-circuit SRS; prover **derived** from a $\Theta(n\log n)$ KZG prover (§1, p. 173)	[Lip24, §1, pp. 173, 175, Table 1, §5, pp. 199–202]

Table 7.2: **Polynomial commitment families (§4.4).** Costs are for one evaluation proof for a polynomial with s coefficients; in a compiled zk-SNARK, $s = O(|C|)$. Rows from Thaler's Table 16.1 are univariate; "Strict" says whether a holographic PIOP compiled with the scheme meets the bound of §3. Blank = the source gives no such bound.

Family	Scheme (used by)	Setup: $\mathcal{S}_1$ output	Commitment	Evaluation proof	Verifier	Assumption	Plausibly PQ	Strict (§3)	Source
Pairing	KZG (Sonic, PLONK, Marlin)	trusted, universal, updatable SRS of powers g^{τ^i}	1 group element	$O(1)$ group elements	$O(1)$ group ops + 2 pairings	t-SDH (binding); AGM for extraction	no	yes	[KZG10, §1, p. 2]; [Tha22, p. 545]; [GWC19, §3.1, p. 12, Lemma 4.7]
Pairing, multilinear	multilinear KZG (HyperPlonk)	universal SRS	constant	$O(\log s)$ $\mathbb{G}_1$ elements; 0.8 KB at $s = 2^{25}$ (128-bit security)	$O(\log s)$ pairings	bilinear groups (no assumption named in the table)	no	yes	[CBBZ23, Table 2, p. 15]; [Tha22, p. 545]
Discrete log	Bulletproofs IPA (Bulletproofs, Halo)	transparent: $O(s)$ random group elements	1 group element	$O(\log s)$ group elements	multi-exp. of size $O(s)$	discrete log	no	no (linear verifier)	[Tha22, Table 16.1]; [BBB+17, §3]
Discrete log	Hyrax-commit (Hyrax)	transparent: $O(\sqrt{s})$ random group elements	$O(\sqrt{s})$ group elements	$O(\sqrt{s})$, or $O(\log s)$ with Bulletproofs	multi-exp. of size $O(\sqrt{s})$	discrete log	no	no ($\sqrt{s}$)	[Tha22, Table 16.1]; [WTS+18, §6.1]

continued on next page

Table 7.2, continued

Family	Scheme (used by)	Setup: $\mathcal{S}_1$ output	Commitment	Evaluation proof	Verifier	Assumption	Plausibly PQ	Strict (§3)	Source
Pairing, transparent	Dory	transparent: $O(\sqrt{s})$ random group elements; $O(\sqrt{s})$-exp. preprocessing	1 group element	$O(\log s)$ group elements	multi-exp. of size $O(\log s) + O(1)$ pairings	SXDH (standard model, interactive); ROM after Fiat–Shamir	no	yes, in principle	[Lee21, abstract]; [Tha22, Table 16.1]
Hash, RS proximity	FRI; STIR, WHIR (STARK, Aurora, Fractal, Virgo)	transparent: hash function only	1 hash	$O(\lambda \log^2 s)$ hashes (FRI's bound; STIR and WHIR need fewer queries)	$O(\lambda \log^2 s)$ hash evaluations	collision-resistant hash / ROM	yes	yes (STARK: uniform only)	[Tha22, Table 16.1]; [ACFY24]; [ACFY25]
Hash, linear codes	Ligero-commit, Brakedown, Binius	transparent: hash function (Brakedown: also a code sampled from a public seed)	1 hash	$O(\sqrt{s\lambda})$ field elements	$O(\sqrt{s\lambda})$ field ops	collision-resistant hash / ROM	yes	no ($\sqrt{s}$)	[Tha22, Table 16.1, §10.5]; [GLS+23]; [DP25, Thm. 3.14]
Hash, linear codes	Orion-PC	transparent		$O(\log^2 s)$	$O(\log^2 s)$ after preprocessing	collision-resistant hash / ROM	yes	claimed, with preprocessing (only a linear verifier is proven, §9.1)	[XZS22, Table 1, p. 3] (bounds for Orion's ZKP, with N the circuit size)
Hash, foldable codes	BaseFold (used with HyperPlonk in [ZCF24])	transparent: hash function; random foldable code from public randomness		polylog(s)	$O(\log^2 s)$	collision-resistant hash / ROM	yes	yes	[ZCF24, abstract]
Hash, binary fields	binary BaseFold with ring-switching ("FRI-Binius")	transparent: hash function only	1 hash	polylog(s)	$O(\lambda \log s)$ field + $O(\lambda \log^2 s)$ hash ops	collision-resistant hash / ROM	plausibly	yes (ZK not discussed)	[DP26, §4.4, p. 44]
Hash, RAA codes	Blaze	transparent: hash function; randomized code parameters	1 hash	$O_\lambda(\log^2 s)$	$O_\lambda(\log^2 s)$	collision-resistant hash / ROM	plausibly	yes (not hiding)	[BCF+25, Thm. 1, p. 4]
Hash, recursive Ligero	Ligerito	transparent: hash function only	1 hash	about $\log^2 s / \log\log s$	polylog for tensorizable codes	collision-resistant hash / ROM	plausibly	yes for tensorizable codes (not ZK)	[NA25, §6.2, p. 13, §6.4, p. 17]

continued on next page

Table 7.2, continued

Family	Scheme (used by)	Setup: $\mathcal{S}_1$ output	Commitment	Evaluation proof	Verifier	Assumption	Plausibly PQ	Strict (§3)	Source
Unknown order	DARK (Supersonic)	class groups: transparent; RSA: trusted; constant size		$O(\mu \log d)$	$O(\mu \log d)$	strong RSA + adaptive root (Thm. 5); revised 2022 proof: random order + adaptive root	no	yes	[BFS20, §1.1, §3.1, Thm. 5]; Table A.1
Unknown order	DewTwo	transparent: class groups or hyperelliptic Jacobians		$O(\log \log s)$ group elements; $\leq$ 4.5 KB for $s \leq 2^{30}$	$O(\log s)$ field + group ops	falsifiable group-of-unknown-order (GUO) assumptions (interactive version)	no	yes	[BMSS25, Table 1, p. 554]
Lattice	Greyhound	transparent (ROM)		polylog(s)	$O(\sqrt{s})$	Module-SIS	yes	no ($\sqrt{s}$)	[NS24, §1.1, p. 3]
Lattice	Cini–Malavolta–Nguyen–Wee	transparent (ROM)		polylog(s)	polylog(s)	Module-SIS	yes	yes (not hiding)	[CMNW24, abstract, §1.1, §2.1, p. 212]

Discussion: what the tables show

Constant-size versus polylogarithmic proofs. Table 7.1 splits by proof size, with two exceptions. Every row built on a univariate pairing-based commitment or a linear PCP (GGPR, Pinocchio, Groth16, Sonic, PLONK, Marlin, Polymath) has an $O_\lambda(1)$ proof. HyperPlonk is pairing-based too, but its multilinear KZG opening has $\log n$ group elements, so its proof is $O_\lambda(\log|C|)$ [CBBZ23, §1, p. 6, Table 2]. Spartan$_{\mathrm{KE}}$ also has a trusted (universal, private) setup, but its proof is $O_\lambda(\log^2|C|)$ [Set20, Fig. 2]. The transparent rows that meet the strict bound have proofs that grow with the circuit: $O_\lambda(\log^2|C|)$ for Fractal and Spartan$_{\mathrm{RO}}$ [COS20, Thm. 3; Set20, Fig. 2], and $O_\lambda(\log|C|)$ for Supersonic [BFS20, §1; Set20, Fig. 2]. The verifier follows the same pattern. Groth16 needs three pairings and one exponentiation per public input [Gro16, Table 2], so its verifier is $O_\lambda(|x|)$. Polymath replaces these exponentiations by an $O(|x|\log|x|)$ interpolation and needs two pairings [Lip24, Table 1]. The univariate universal-SRS schemes (Sonic, PLONK, Marlin) add a $\log|C|$ term, which in PLONK comes from evaluating the vanishing polynomial $Z_H(z) = z^n - 1$ (derived from [GWC19, §8]). The verifiers of Fractal and Spartan$_{\mathrm{RO}}$ are $O_\lambda(|x| + \log^2|C|)$ [COS20, Thm. 3; Set20, Fig. 2], and Supersonic's is $O_\lambda(|x| + \log|C|)$ [BFS20, §1; Set20, Fig. 2]. Table A.2 adds that a short proof is not enough for the strict definition. The commonest reason a scheme lands in Table A.2 is the verifier, not the proof. Bulletproofs has a logarithmic proof and Aurora and Orion polylogarithmic ones, but the verifiers that the three papers prove are linear [BBB+17, §3–4; BCR+19, Thm. 1.2; XZS22, Thm. 5], and STARK's verifier is polylogarithmic only for uniform (AIR) computations [BBHR18, §2.2, Thm. 3.5].

Trusted setup versus transparency. The constant proof size is paid for in the setup. Every row of Table 7.1 with an $O_\lambda(1)$ proof has setup parameters of size $O_\lambda(|C|)$ generated from secret randomness. For GGPR, Pinocchio and Groth16 this is a per-circuit CRS, and Polymath also has a per-circuit SRS [Lip24, Table 1, §3, p. 188]. For Sonic, PLONK and Marlin it is a universal, updatable SRS (Table A.1). HyperPlonk's multilinear KZG also needs a universal setup, although its proof is logarithmic [CBBZ23, Table 2]. In the terms of Remark 1, the universal schemes move the secret randomness from $\mathcal{S}_2$ to $\mathcal{S}_1$: a new circuit then needs only deterministic preprocessing, but the trusted step is still there. Only three rows of Table 7.1 have $O_\lambda(1)$ setup parameters, i.e. are fully succinct (the setup sizes of Spartan$_{\mathrm{KE}}$ and HyperPlonk are not stated). Fractal needs nothing beyond the random oracle [COS20, Thm. 3], and neither does Spartan$_{\mathrm{RO}}$ with its hash-based commitment [Set20, Fig. 4, p. 20]. Supersonic needs only a group description and a generator [BFS20, §1, §4.3], and it is transparent only with class groups; with RSA groups its setup is trusted (Table A.1). No row of Tables 7.1 and A.2 combines an $O_\lambda(1)$ proof with a transparent setup.

Prover cost. Most strict rows have quasi-linear provers, $O_\lambda(|C|\log|C|)$. The logarithmic factor comes from the field arithmetic of the polynomial operations, on top of a linear number of group exponentiations, as Table A.1's counts for Groth16 and PLONK show [Gro16, Table 2; CHM+20, Fig. 1; GWC19, Table 1]. Pinocchio's entry has a second logarithmic factor because Pinocchio states $O(|C|\log^2|C|)$ field work for computing $h(x)$ [PHGR13, p. 4]; its exponentiation count, taken from Groth16, omits this work (Table 7.1, Notes). Two strict rows have linear-time provers, Spartan$_{\mathrm{KE}}$ and HyperPlonk, and both pay for this with a universal trusted setup [Set20, §1, Fig. 2; CBBZ23, Table 2]. With multilinear KZG or Orion+, both of which need a universal setup, HyperPlonk's prover is linear; with its transparent FRI-based commitment it becomes quasi-linear [CBBZ23, §1, p. 5, Table 2]. HyperPlonk's Table 2 also lists transparent multilinear commitments whose commitment and evaluation take linear time (Dory, Bulletproofs and Orion), but the paper does not instantiate HyperPlonk with them [CBBZ23, Table 2]. So no row of Table 7.1 has

both a linear-time prover and a transparent setup. Orion claims both: an $O(N)$ prover with no trusted setup, $O(\log^2 N)$ proofs and, in the preprocessing setting that §3 allows, $O(\log^2 N)$ verification for circuits of size N [XZS22, abstract, Table 1, p. 3]. Its theorem, however, states a linear verifier [XZS22, Thm. 5, p. 23], so its row is in Table A.2 (§9.1). Polymath moves the other way. To shorten Groth16's proof in bits, its prover does about $40.0n$ $\mathbb{G}_1$ scalar multiplications against Groth16's $8.5n$, by its author's estimate [Lip24, §5, p. 200].

Linear-time provers are more common in Table A.2, but each gives up something else: Libra a transparent setup and depth-independent costs, Hyrax arbitrary circuits (its prover is linear only with enough data parallelism), Spartan$_{\mathrm{DL}}$ and Bulletproofs a polylogarithmic proof or verifier, and Orion a proven polylogarithmic verifier [XZZ+19, Table 1; WTS+18, §1, §6; Set20, Fig. 2; BBB+17, §3–4; XZS22, Thm. 5]. Since O_λ counts a group exponentiation and a field multiplication alike, rows with the same asymptotic prover can differ greatly in practice, as Table A.1's exponentiation counts for unhelped Sonic and PLONK show [GWC19, Table 1]. Section 9.1 takes up the combination of a linear-time prover with the other properties as an open problem.

Post-quantum security and assumptions. Table A.1 records each row's assumptions and post-quantum status, and they follow the dichotomy of §3.1. The linear-PCP rows (GGPR, Pinocchio, Groth16) need no random oracle and rest on knowledge assumptions or the generic group model, whereas Polymath, also circuit-specific, uses KZG with Fiat–Shamir in the algebraic group model (§5.1); the universal rows rest on the algebraic group model, with the random oracle model for Fiat–Shamir (PLONK is now also knowledge-sound in the ROM from falsifiable assumptions, §5.2); and the hash-based rows need only the random oracle. Only the hash-based rows are plausibly post-quantum (§5.3), and they pay with proofs that grow with the circuit; Plonky2's security is only conjectured [Pol22, §8, fn. 8]. Choosing a row is therefore also choosing an assumption: §9.4 shows that replacing the random oracle with a concrete hash can fail, and §9.5 discusses the search for standard assumptions.

8 Developments since 2022

Tables 7.1 and A.2 have only five rows for work after Nova (HyperPlonk, Orion, Plonky2, Polymath and HyperNova), because most work since 2022 has changed the components (§4.1, §4.3, §4.4, §5.3, §6.4) rather than added families. This section draws those threads together around five questions.

Why have fields become small, and what does it cost? Group-based commitments fix the field to the group's 256-bit scalar field (§2.1, Table 8.1). Hash-based commitments (§4.4) lift this constraint; §4.1 describes what a fixed field costs when the computation lives in another algebraic structure. Diamond and Posen credit small fields, fields with fast reduction such as Goldilocks ($p = 2^{64} - 2^{32} + 1$, Table 8.1), and fast hashes for the speed of FRI- and Brakedown-based SNARKs. With Goldilocks and Keccak-256, FRI commits 1 MiB about 7-fold faster than KZG over BN254 [DP25, §1, p. 2]. Table 8.1 lists the fields in use. The cost is soundness. By Schwartz–Zippel (§2.1), a random check misses a false identity with probability at most its degree over the field size. In a circle STARK, for instance, the two algebraic error terms are roughly the constraint count, and the degree times the trace length, each divided by the challenge field's size and multiplied by a list-size bound [HLP24, §5.2]. Over a 31-bit field one such term exceeds 2^{-31}. The small-field systems of Table 8.1 therefore keep the witness in $\mathbb{F}$ and draw challenges from an extension $\mathbb{K} \supset \mathbb{F}$. Part of the prover's work, such as Binius's sum-check, then runs in $\mathbb{K}$ [DP25, §1, p. 3]. Binary towers go furthest: Binius commits to $\mathbb{F}_2$-valued data with no

Table 8.1: Fields used by recent systems. Each row has one source. The "Challenges" column gives the field that the source's configuration samples random challenges from.

Field	Order	Why it is chosen	Challenges	Source
Scalar field of bn256 / bls381	about 256-bit prime	fixed by the pairing group (KZG, Groth16)	the field itself	[BS23, §15.4, p. 626]
Goldilocks	$p = 2^{64} - 2^{32} + 1$	an element fits in a 64-bit word, and reduction is fast	$\mathbb{F}_p[X]/(X^2 - 7)$ (Plonky2)	[Pol22, §2–2.1, pp. 3–4]
BabyBear	$p = 2^{31} - 2^{27} + 1$	31-bit prime used in Plonky3's FRI commitment	$\mathbb{F}_p[X]/(X^4 - 11)$ (Plonky3)	[DP26, §5.2, p. 47]
Mersenne-31	$p = 2^{31} - 1$	"exceptionally efficient arithmetic"; $p + 1$ is a power of two, hence circle STARKs (§4.1)	any finite extension of $\mathbb{F}_p$, sized for soundness	[HLP24, §2, §5.2]
Binary tower	$\mathbb{F}_2 \subset \mathbb{F}_{2^2} \subset \cdots \subset \mathbb{F}_{2^{128}}$	addition is XOR, and bits are committed with no embedding overhead	$\mathbb{F}_{2^{128}}$	[DP25, §1, pp. 2–3]

embedding overhead [DP25, abstract], and ring-switching moves the evaluation claim to the large field (§4.4).

How are zkVMs built? A zkVM (§1) proves that a program for a fixed instruction set ran correctly, and the two designs in use differ in how they arithmetize a CPU step. The *lookup-centric* design reduces almost everything a CPU step does to memory operations [STZ26, §1, p. 268]. Jolt's circuits "primarily perform lookups into a gigantic lookup table, of size more than 2^{128}, that depends only on the ISA", which is structured and never materialized [AST24, abstract], so the decomposable-table lookups and one-hot memory checking of §4.3 carry the whole computation. The *STARK-based* design instead proves an AIR trace (§4.1) over a small field with a hash-based commitment, as in Plonky3, "mainly used to power STARK-based zkVMs" [Plonky3]. Both designs end alike: a large zkVM proof "can be generically shrunk via SNARK composition", so the bottleneck is the prover, still "hundreds of thousands of times slower than native execution" [STZ26, §1, p. 268].

Why do recent IVC schemes fold instead of verifying recursively? Classical recursion (§6.1) proves the previous verifier inside the next circuit; §6.4 gives the resulting recursion thresholds for Fractal and Plonky2. Folding (§6.3) proves no SNARK verifier. Each step folds the new instance into a running one, which is proven once at the end, so the circuit holds only the cheap folding verifier. Nova's verifier circuit has 10,000 R1CS constraints, by its authors' count over 10× below SNARK-based recursion with a per-circuit trusted setup and over 100× below a transparent one [KST22, §1.3, p. 5]. Later schemes fold CCS, high-degree gates, lookups and many instances, NeutronNova with a constant-size recursive verifier (§6.4, Figure 6). Two limits remain. The folded proof must still be compressed by a SNARK (§6.3). And the in-circuit hash leaves IVC security heuristic (§6.1).

How much of the security is proven? A hash-based commitment sets its query count, and so its proof size, from a proximity-gap bound (§5.3). The error terms of these bounds grow with the problem size and shrink only with the field, so WHIR samples the relevant challenges from "a sufficiently large extension of the base field" and adds proof-of-work [ACFY25, §6.2, p. 44]. Even 128 bits can be too few. Diamond and Posen note that over a 128-bit field the best proven list-decoding bound gives an error of at least 2^{-100} on any reasonably large instance, so 100 bits of provable security need the unique-decoding regime, which their benchmark of Plonky3 also uses [DP26, §5.2, p. 47]. The price of proven bounds is measurable. For 24 variables at rate 1/2 and 128-bit security, over Goldilocks with challenges from its cubic extension, WHIR's arguments are 621 KiB

with the proven unique-decoding bound, 299 KiB under its Johnson-bound conjecture and 156 KiB under its up-to-capacity conjecture [ACFY25, §6.2, p. 44, §6.3.4, p. 53]. Crites and Stewart disprove the up-to-capacity conjecture (§5.3). Their weaker replacements would cost deployed systems "only a small loss", but they "should serve only as benchmarks" [CS26, abstract, §1, p. 130]. Newer commitments set parameters from proven bounds alone: binary BaseFold and Ligerito in the unique-decoding regime, and circle FRI up to the Johnson radius [DP26, §5.2, p. 47; NA25, §6.3, p. 14; HLP24, Thm. 6, p. 38] (Figure 5). Khatam extends the proven regime to every linear code (§5.3).

Which post-quantum options are not hash-based? Hash-based accumulation recurses expensively: Nguyen and Setty put Arc's verifier circuit at about 1,600,000 R1CS constraints with Poseidon [NS26, §1.1, p. 495]. Group-based folding schemes "rely on an additively homomorphic commitment scheme based on discrete log, and are therefore not post-quantum secure and require a large (256-bit) field" [BC25a, abstract]. LatticeFold (§6.4) uses Ajtai commitments under Module-SIS instead, and keeps extracted witnesses of low norm through many folds with sum-check [BC25a, abstract]. The small-field theme returns. LatticeFold's rings split completely over fields such as Goldilocks, which forces an extension and a matching multiplicative overhead [NS26, §1.1, p. 495]. Neo and SuperNeo instead run one sum-check over a small-field extension, with commitment costs that scale with the witness's bit-width [NS26, abstract]. A constant-size lattice verifier circuit like Nova's "remains an open problem" [NS26, §1.1, p. 495]. Among lattice commitments, Greyhound's verifier is only sublinear, and several schemes rest on the contested knowledge k-R-ISIS assumption (§4.4). Section 9.2 takes up what a complete lattice-based zk-SNARK still lacks.

9 Open problems and future directions

Tables 7.1, 7.2, A.1 and A.2 show what current zk-SNARKs achieve and at what cost. This section collects the open problems those trade-offs point to, together with recent work on them.

9.1 Prover efficiency

The prover is the main practical bottleneck. Brakedown's authors observe that prover cost "has limited the use of SNARKs to practical applications in which NP statements of interest are relatively small" [GLS+23, §1, p. 1], and HyperFond's that "proving time remains a bottleneck for applying SNARKs to large-scale circuits" [YLZ+26, abstract]. One line of work removes the FFT, as HyperPlonk does by moving PLONK's PIOP to the Boolean hypercube and reducing every check to sum-check (§4.2). HyperPlonk is made zero-knowledge by masking, for any additive, spanning commitment scheme [CBBZ23, App. A, Cor. A.4].

A second line uses linear-time-encodable codes, starting with Brakedown, which this survey treats only as a commitment scheme (§3, §4.4). Orion is a zero-knowledge argument with a linear-time prover and $O_\lambda(\log^2|C|)$ proofs [XZS22, abstract]. Its theorem, however, states a linear verifier [XZS22, Thm. 5, p. 23]. The polylogarithmic verifier is claimed only in the preprocessing setting, where the verifier holds a commitment describing the circuit, and no theorem states it [XZS22, §1, pp. 3–4]. Also, a soundness flaw in an earlier version of its code-switching step had to be fixed with a commit-and-prove SNARK [XZS22, §4.2, fn. 3, p. 20].

A third line works over small and binary fields (§8). The successor of Binius (§4.4) is parametrized for 100 bits of provable security in the unique-decoding regime [DP26, §5.2, p. 47], and Blaze, over a 128-bit binary field, has a linear-time prover but proofs larger than BaseFold's [BCF+25, §8.1, p. 37, Fig. 4, p. 39].

The 2026 commitment schemes push the prover's constant factors further. In its theory-oriented instantiation, Bolt commits with about $(3 + \varepsilon)$ field additions per committed element plus a Merkle hash over about as many field elements [GNR26, abstract, p. 396]. TensorSwitch opens with about six field multiplications per coefficient [BFRW26, abstract, p. 297]. Chopin is pairing-based, with constant-size proofs and a single large multi-scalar multiplication [BHK+26, abstract, p. 428]. None of these three is zero-knowledge as presented (§3). Bolt "does not provide hiding, aka zero-knowledge, guarantees" [GNR26, §1.3, p. 407], and for Chopin's zero-knowledge variant a standard-model proof of knowledge soundness is left to future work [BHK+26, p. 436]. An open problem, then, is to combine a linear-time prover, zero knowledge, a polylogarithmic verifier and a transparent setup in one fully proven scheme. For arbitrary circuits, the verifier bound can hold only after preprocessing, since otherwise the verifier must read the circuit [XZS22, §1, pp. 3–4], so the target is a transparent preprocessing of the kind §3 allows. Orion comes closest but proves only a linear verifier (above), and no row of Table 7.1 has both a linear-time prover and a transparent setup (§7).

Distributed proving splits the prover across machines; HyperFond does so with a transparent setup, plausible post-quantum security and polylogarithmic communication, but its proof size and verification time grow linearly with the number of machines, removing this overhead is left open, and its PIOPs are presented without zero knowledge [YLZ+26, abstract, §6, p. 12, Remark 3, p. 8].

How small can a proof get? Arnon, Dujmovic and Yogev construct a publicly verifiable SNARG whose proof is exactly two $\mathbb{G}_1$ elements (768 bits on BLS12-381), with imperfect completeness, and they prove that no SNARG in the generic bilinear group model plus random oracle can have a one-element proof, under mild restrictions on the verifier [ADY26, abstract, p. 125, §1.1, p. 128]. Their scheme is "not yet concretely efficient" and is not claimed to be zero-knowledge. The PARI scheme already has two $\mathbb{G}_1$ elements plus two field elements [ADY26, p. 127], but its authors leave zero knowledge to future work. Khurana, Padhye and Sahu add it with the same 1280-bit proof on BLS12-381, the same verification key and the same verifier, at the cost of 13 more $\mathbb{G}_1$ elements in the proving key; zero knowledge is perfect in the random oracle model, and knowledge soundness is proven in the algebraic group model with random oracles under SDH [KPS26, abstract, §1.1, pp. 2–3, §7, pp. 22–23]. A zero-knowledge argument with fewer group elements than Groth16's three therefore exists, although its costs are analytical only, since the scheme has not been implemented [KPS26, §7, p. 23]. Two recent results sharpen the picture. Polymath (§5.1) is a zk-SNARK that is shorter than Groth16 in bits, though not in group elements [Lip24]. In the designated-verifier setting, one element of a generic group plus a soundness-dependent number of extra bits suffices: Arnon, Dujmovic and Ishai reach 2^{-80} soundness at the 128-bit level with 695-bit proofs [ADI25, §1.1], though without reusable soundness [ADY26, p. 127]. A transparent counterpart is also open: no transparent row of Tables 7.1 and A.2 has constant-size proofs (§7), and DewTwo, which has "the smallest proof size amongst transparent PCS" by its authors' account, still has $O(\log \log s)$ evaluation proofs for s coefficients, under 4.5 KB for $s \leq 2^{30}$ [BMSS25, abstract] (§4.4, Table 7.2).

9.2 Post-quantum zk-SNARKs

Table A.1 marks only hash-based schemes as plausibly post-quantum, and two points need care even for them. First, STARK's post-quantum claim covers the interactive protocol only [BBHR18, §1.3.1, fn. 12]. For the BCS transform that compiles such IOPs into non-interactive arguments, Rotem and Tessaro report a quantum-random-oracle proof of knowledge soundness by Chiesa, Manohar and Spooner [RT25, §1, p. 100; CMS19, Thm. 8.6, p. 40]. Second, concrete security often rests on conjectures about Reed–Solomon proximity gaps: after Crites and Stewart's disproofs, FRI's own up-to-capacity conjecture is unproven but not disproved, and whether a polynomial-time attack exists in this regime

is open (§5.3, Figure 5). Chaliasos et al. raise a related practical question about the choice of curves and fields: "How many bits of security does each SNARK have when using specific configurations?" [CET+24, §10, p. 14]. Choosing deployed parameters from proven rather than conjectured bounds, whose cost §8 measures for WHIR, and narrowing the gap between the two, is therefore open.

Plain Fiat–Shamir applied to other many-round protocols, such as folding and sum-check-based arguments, still has a quantum-specific gap. The known quantum-random-oracle analysis of Fiat–Shamir for $(2s+1)$-round protocols loses a factor of order q^{2s+1} for q oracle queries, which gives no guarantee for a super-constant number of rounds [RT25, §1, p. 99]. Rotem and Tessaro give a multi-round transform with straight-line extraction that avoids this loss in the quantum random oracle model, using Merkle-tree commitments to transcript trees [RT25, abstract, §1].

Lattices are the other post-quantum route. LatticeFold and LatticeFold+ give plausibly post-quantum folding (§6); by its authors' evaluation, LatticeFold is about as fast as HyperNova [BC25a, abstract]. Neo and SuperNeo move lattice folding to small fields such as Goldilocks (§6.4, §8). FACS gives a hash-based alternative, accumulation in the random oracle model with a linear-time prover (§6.4). Greyhound (§4.4) has 53 KB evaluation proofs at degree 2^{30}, but a verifier that is sublinear rather than polylogarithmic [NS24, abstract]. Lattice-based folding schemes and commitments therefore exist, but none of the works cited here builds and analyzes a complete lattice-based zk-SNARK, zero knowledge included, that meets the strict bound of Section 3 (cf. Figure 4). The gap is visible in Cini et al.'s commitment, which has polylogarithmic proofs and verification (Table 7.2) but is not hiding, since "the focus of this work is on succinctness and not zero-knowledge" [CMNW24, §2.1, p. 212]. Building a complete scheme is an open direction.

9.3 Recursion overhead and the security of folding

Recursion has a measurable cost, in the size of the verifier circuit and the recursion threshold (§6.4, §8). Its security rests on the in-circuit hash heuristic (§3.1, §6.1). Nova, for example, instantiates its random oracle with a concrete hash function and *assumes* that the result remains knowledge-sound [NBS23, §2.3, p. 4]. The circuits in Khovratovich–Rothblum–Soukhanov's attack (§4.2) compute the Fiat–Shamir hash and the commitment internally, and the authors note that such circuits are "quite common in practice. In particular, in the context of recursive proof composition" [KRS25, §1.1.2, p. 5]. The gap between paper and code is a second problem, as the forged proofs against Nova's deployed 2-cycle variant show (§6.3) [NBS23, abstract]. Nguyen, Boneh and Setty's analysis also shows a limit of recursive extraction: if the extractor is slower than the prover by a constant factor $c > 1$, iterating it works only for $O(\log \lambda)$ steps [NBS23, §2.1, Remark 1, p. 2].

On the theory side, Jain, Mathialagan and Waters build fully succinct IVC for deterministic computations from subexponential indistinguishability obfuscation (iO) and one-way functions without recursive extraction. They contrast this with SNARK-based recursion, since SNARKs "are only known from non-standard assumptions or in non-standard models" [JMW26, abstract, p. 3, §1, p. 4]. Their IVC is not claimed to be zero-knowledge. Datta et al. obtain IVC for NP from subexponential iO together with either learning with errors (LWE) or bilinear maps, with proof size $\mathrm{poly}(|x_i|, \log T)$, where T is the number of steps and x_i the configuration after step i [DJJ+25, abstract].

Recursion overhead also separates group-based from post-quantum folding. Neutron-Nova's recursive verifier is constant-size (§6.4), but "achieving constant recursion overheads in the post-quantum setting remains an open problem" [KS26, §1, p. 527], and Neo and SuperNeo aim for logarithmic recursion overhead, like HyperNova's [NS26, §1.1, p. 495]. Recursion that is efficient, zero-knowledge and proven secure with the concrete hash it uses remains open.

9.4 Fiat–Shamir in practice

The Fiat–Shamir step of the pipeline in Section 4 is where many deployed failures occur. *Weak* Fiat–Shamir (§2.3, §3.1) breaks the adaptive knowledge soundness of Bulletproofs, PLONK and Spartan [DMWG23, abstract], and Chaliasos et al. record this "Frozen Heart" class and the related "Last Challenge" attack among the proof-system flaws in their dataset [CET+24, App. A.2–A.3, p. 20]. *Strong* Fiat–Shamir can fail too: the attack on GKR-based arguments (§4.2, §5.4) works for every choice of hash [KRS25, Thm. 1], so soundness depends on how the circuit is implemented, not just on what it computes [KRS25, §1.1.3]. Its authors know of no such attack on protocols that commit to the full computation trace, as most PIOP-based schemes do, but stress that "this does not mean that such attacks do not exist" [KRS25, §5, p. 16]. On the positive side, Chiesa et al. prove that the PIOP-plus-commitment recipe gives state-restoration-sound arguments, the property Fiat–Shamir security needs in the random oracle model, when the PIOP is state-restoration sound and the commitment satisfies a state-restoration form of function binding (§4.5) [CGKY26, abstract, p. 189].

Extraction is a separate issue. Fiat–Shamir extraction rewinds the prover, which is often at odds with concurrent and universally composable security; Rotem and Tessaro initiate the study of generic transforms with straight-line extraction for multi-round protocols that use only the random oracle, and show that they apply to Bulletproofs [RT25, abstract, §1, pp. 96–98, 100]. Faonio and Russo show that the multivariate PST commitment of Papamanthou et al. is malleable only through its linear homomorphism, and that this suffices for sum-check-based zk-SNARKs compiled with PST to be simulation-extractable in the algebraic group model, provided every oracle is queried in some sum-check [FR26, abstract, p. 35, §1, p. 42]. They note that "removing the AGM remains a significant challenge" [FR26, §1, p. 37]. Two problems remain open: characterizing which deployed protocols are Fiat–Shamir-secure with a concrete hash, and proving simulation extractability without idealized models.

9.5 Foundations: SNARGs from standard assumptions

The Gentry–Wichs barrier (Section 3.1) explains why every scheme in this survey uses a knowledge assumption, an idealized group model or a random oracle [GW11, Thm. 5.1]. Recent theory works around it for SNARGs, which are sound but not claimed to be knowledge-sound. Waters and Wu build an adaptively sound SNARG for NP from subexponential iO and one-way functions, with a CRS whose size grows with the relation's circuit [WW25, abstract, p. 3]. Freitag and Wichs explain why this does not contradict the barrier: such constructions use subexponential hardness and a CRS longer than the statement [FW26, abstract, p. 190]. Restricting the language also helps. For UP, where each statement has at most one witness, Mathialagan et al. obtain designated-verifier zk-SNARKs with adaptive soundness from subexponential LWE and evasive LWE, a new assumption that is not falsifiable on its face [MPV24, abstract, §1, p. 42]. Batch arguments (BARGs) relax the goal differently. Proofs may grow with a single witness, and security is somewhere extractability, so BARGs "evade the known impossibility result for SNARKs" and are realizable from falsifiable assumptions [BFN26, §1, p. 363]. None of these results yields a practical zk-SNARK, and whether zk-SNARKs for NP can be built from standard falsifiable assumptions without idealized models remains open.

10 Conclusion

Under one strict definition, the zk-SNARK families of 2013–2020 reduce to a few trade-offs. Pairing-based schemes built on a linear PCP or on univariate KZG have constant-size proofs; HyperPlonk's multilinear KZG gives logarithmic ones (§7, Table 7.1). With a

circuit-specific setup (GGPR, Pinocchio, Groth16), Groth16's three group elements remain the benchmark (Polymath is shorter in bits, not in group elements, §5.1). With a universal, updatable setup (Sonic, PLONK, Marlin), one ceremony serves all circuits up to a size bound. Either way the price is a trusted setup with parameters linear in the circuit size, and assumptions that are non-falsifiable or idealized (§§5.1–5.2). Transparent schemes (Fractal, Spartan$_{\mathrm{RO}}$, Supersonic with class groups) remove the trapdoor at the cost of logarithmic or polylogarithmic proofs, and among the compared rows only the hash-based ones are plausibly post-quantum (§5.3, §7). Bulletproofs, Ligero, Aurora and Hyrax miss the strict bound, Bulletproofs and Aurora because of the verifier alone (§5.4). In the pipeline of §4, the polynomial commitment largely decides the setup, the proof size and the post-quantum status, and Fiat–Shamir adds a random-oracle heuristic that has failed in practice (§3.1, §9.4).

Since 2022 the components have changed more than the families: Tables 7.1 and A.2 add only HyperPlonk, Polymath, Orion, Plonky2 and HyperNova, while most new work changes one component of the pipeline (§7, §8). Hash-based commitments lift the 256-bit field that group-based commitments impose, so recent systems run over Goldilocks, 31-bit primes or binary towers and draw challenges from an extension field (§4.1, §8). Lookups have moved from plookup's sorted grand product to log-derivative identities and sum-check, and Twist and Shout prove memory reads and writes with no grand products at all, which lookup-centric zkVMs such as Jolt build on (§4.3, §8). GKR has returned because its prover commits only to the input, the property that the Khovratovich–Rothblum–Soukhanov attack on its Fiat–Shamir version exploits (§4.2). FRI and Ligero have grown into a family of hash-based commitments (STIR, WHIR, BaseFold and its binary successor, Brakedown, Blaze, Ligerito) with linear-time provers or polylogarithmic verification, none of them zero-knowledge as presented in the sense of §3 (§4.4, Figure 3). Folding has become the main alternative to recursion by verification, and now covers CCS, high-degree gates, lookups, many instances and, with LatticeFold and Neo, lattices (§6.4, §8).

Crites and Stewart disproved the up-to-capacity correlated-agreement conjecture and WHIR's mutual version of it, FRI's own conjecture remains open, and in one WHIR configuration proven unique-decoding parameters give arguments about four times larger than its up-to-capacity conjecture does (§5.3, §8). The open problems follow (§9): a transparent scheme with constant-size proofs (no compared row has both, §7, §9.1); one fully proven scheme with a linear-time prover, zero knowledge, a polylogarithmic verifier and transparent preprocessing (§9.1); deployed parameters from proven bounds, and a complete lattice-based zk-SNARK that meets the strict bound (§9.2); constant recursion overhead for post-quantum folding (§9.3); Fiat–Shamir and recursion that are secure with the concrete hash they use (§§9.3–9.4); and zk-SNARKs for NP from standard falsifiable assumptions without idealized models (§9.5).

Acknowledgments

This survey was written as a personal project and was not supervised. AI tools were used to assist in writing this paper.

References

[ACFY24] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. "STIR: Reed–Solomon Proximity Testing with Fewer Queries". In: *Journal of the ACM* (2026). Just Accepted (online 2026-10-16); Preliminary version in CRYPTO 2024, LNCS, pp. 380–413, doi:10.1007/978-3-031-68403-6_12; IACR ePrint 2024/390. DOI: 10.1145/3858414.

[ACFY25] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. "WHIR: Reed–Solomon Proximity Testing with Super-Fast Verification". In: *Advances in Cryptology – EUROCRYPT 2025*. LNCS. Full version: IACR ePrint 2024/1586. Springer, 2025, pp. 214–243. DOI: 10.1007/978-3-031-91134-7_8.

[ADI25] Gal Arnon, Jesko Dujmovic, and Yuval Ishai. "Designated-Verifier SNARGs with One Group Element". In: *Advances in Cryptology – CRYPTO 2025, Part VII*. Vol. 16006. LNCS. Springer, 2025, pp. 192–224. DOI: 10.1007/978-3-032-01907-3_7.

[ADY26] Gal Arnon, Jesko Dujmovic, and Eylon Yogev. "Pairing-Based SNARGs with Two Group Elements". In: *Advances in Cryptology – CRYPTO 2026, Part IX*. Vol. 16808. LNCS. Full version: IACR ePrint 2025/2160. Springer, 2026, pp. 125–156. DOI: 10.1007/978-3-032-35424-2_5.

[AHIV17] Scott Ames, Carmit Hazay, Yuval Ishai, and Muthuramakrishnan Venkitasubramaniam. "Ligero: Lightweight Sublinear Arguments Without a Trusted Setup". In: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS '17)*. Extended version: IACR ePrint 2022/1608. ACM, 2017, pp. 2087–2104. DOI: 10.1145/3133956.3134104.

[AST24] Arasu Arun, Srinath Setty, and Justin Thaler. "Jolt: SNARKs for Virtual Machines via Lookups". In: *Advances in Cryptology – EUROCRYPT 2024, Part VI*. Vol. 14656. LNCS. Full version: IACR ePrint 2023/1217. Springer, 2024, pp. 3–33. DOI: 10.1007/978-3-031-58751-1_1.

[BBB+17] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. "Bulletproofs: Short Proofs for Confidential Transactions and More". In: *2018 IEEE Symposium on Security and Privacy (SP)*. Full version: IACR ePrint 2017/1066, https://eprint.iacr.org/2017/1066. IEEE, 2018, pp. 315–334. DOI: 10.1109/SP.2018.00020.

[BBHR18] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. *Scalable, Transparent, and Post-Quantum Secure Computational Integrity*. IACR Cryptology ePrint Archive, Report 2018/046. 2018.

[BBHR18b] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. "Fast Reed-Solomon Interactive Oracle Proofs of Proximity". In: *45th International Colloquium on Automata, Languages, and Programming (ICALP 2018)*. Vol. 107. LIPIcs. Schloss Dagstuhl, 2018, 14:1–14:17. DOI: 10.4230/LIPIcs.ICALP.2018.14.

[BC23] Benedikt Bünz and Binyi Chen. "Protostar: Generic Efficient Accumulation/Folding for Special-Sound Protocols". In: *Advances in Cryptology – ASIACRYPT 2023*. LNCS. Full version: IACR ePrint 2023/620. Springer, 2023, pp. 77–110. DOI: 10.1007/978-981-99-8724-5_3.

[BC25a] Dan Boneh and Binyi Chen. "LatticeFold: A Lattice-Based Folding Scheme and Its Applications to Succinct Proof Systems". In: *Advances in Cryptology – ASIACRYPT 2025*. LNCS. Full version: IACR ePrint 2024/257. Springer, 2025, pp. 330–362. DOI: 10.1007/978-981-95-5099-9_11.

[BC25b] Dan Boneh and Binyi Chen. "LatticeFold+: Faster, Simpler, Shorter Lattice-Based Folding for Succinct Proof Systems". In: *Advances in Cryptology – CRYPTO 2025, Part VII*. Vol. 16006. LNCS. Full version: IACR ePrint 2025/247. Springer, 2025, pp. 327–361. DOI: 10.1007/978-3-032-01907-3_11.

[BCC+16] Jonathan Bootle, Andrea Cerulli, Pyrros Chaidos, Jens Groth, and Christophe Petit. "Efficient Zero-Knowledge Arguments for Arithmetic Circuits in the Discrete Log Setting". In: *Advances in Cryptology – EUROCRYPT 2016*. Vol. 9666. LNCS. Springer, 2016, pp. 327–357. DOI: 10.1007/978-3-662-49896-5_12.

[BCC88] Gilles Brassard, David Chaum, and Claude Crépeau. "Minimum Disclosure Proofs of Knowledge". In: *Journal of Computer and System Sciences* 37.2 (1988), pp. 156–189. DOI: 10.1016/0022-0000(88)90005-0.

[BCCT12] Nir Bitansky, Ran Canetti, Alessandro Chiesa, and Eran Tromer. "From Extractable Collision Resistance to Succinct Non-Interactive Arguments of Knowledge, and Back Again". In: *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference (ITCS '12)*. ACM, 2012. DOI: 10.1145/2090236.2090263.

[BCCT13] Nir Bitansky, Ran Canetti, Alessandro Chiesa, and Eran Tromer. "Recursive Composition and Bootstrapping for SNARKs and Proof-Carrying Data". In: *Proceedings of the 45th Annual ACM Symposium on Theory of Computing (STOC 2013)*. 2013, pp. 111–120. DOI: 10.1145/2488608.2488623.

[BCF+25] Martijn Brehm, Binyi Chen, Ben Fisch, Nicolas Resch, Ron D. Rothblum, and Hadas Zeilberger. "Blaze: Fast SNARKs from Interleaved RAA Codes". In: *Advances in Cryptology – EUROCRYPT 2025*. LNCS. Cited version: Cryptology ePrint Archive, Paper 2024/1609 (version of 2024-10-09). Springer, 2025, pp. 123–152. DOI: 10.1007/978-3-031-91134-7_5.

[BCG+14] Eli Ben-Sasson, Alessandro Chiesa, Christina Garman, Matthew Green, Ian Miers, Eran Tromer, and Madars Virza. "Zerocash: Decentralized Anonymous Payments from Bitcoin". In: *2014 IEEE Symposium on Security and Privacy (S&P)*. Cited version: the extended version (May 18, 2014; 56 pp.); locators in the note refer to it. 2014, pp. 459–474. DOI: 10.1109/SP.2014.36.

[BCG25] Annalisa Barbara, Alessandro Chiesa, and Ziyi Guan. "Relativized Succinct Arguments in the ROM Do Not Exist". In: *Theory of Cryptography – TCC 2025, Part I*. Vol. 16268. LNCS. Cited version: Cryptology ePrint Archive, Paper 2024/728 (version 2025-09-23); TCC 2025 Part I, LNCS 16268, from the Springer book page (2026-10-23). Springer, 2025, pp. 400–416. DOI: 10.1007/978-3-032-12287-2_14.

[BCI+13] Nir Bitansky, Alessandro Chiesa, Yuval Ishai, Rafail Ostrovsky, and Omer Paneth. "Succinct Non-interactive Arguments via Linear Interactive Proofs". In: *Theory of Cryptography (TCC 2013)*. LNCS. Journal version: J. Cryptology 35 (2022), doi:10.1007/s00145-022-09424-4. Springer, 2013, pp. 315–333. DOI: 10.1007/978-3-642-36594-2_18.

[BCIKS20] Eli Ben-Sasson, Dan Carmon, Yuval Ishai, Swastik Kopparty, and Shubhangi Saraf. "Proximity Gaps for Reed–Solomon Codes". In: *2020 IEEE 61st Annual Symposium on Foundations of Computer Science (FOCS 2020)*. Journal version: J. ACM 70(5), 2023, doi:10.1145/3614423. 2020, pp. 900–909. DOI: 10.1109/FOCS46700.2020.00088.

[BCMS20] Benedikt Bünz, Alessandro Chiesa, Pratyush Mishra, and Nicholas Spooner. "Recursive Proof Composition from Accumulation Schemes". In: *Theory of Cryptography – 18th International Conference, TCC 2020, Part II*. Vol. 12551. LNCS. Full version: "Proof-Carrying Data from Accumulation Schemes", IACR ePrint 2020/499. Springer, 2020, pp. 1–18. DOI: 10.1007/978-3-030-64378-2_1.

[BCR+19] Eli Ben-Sasson, Alessandro Chiesa, Michael Riabzev, Nicholas Spooner, Madars Virza, and Nicholas P. Ward. "Aurora: Transparent Succinct Arguments for R1CS". In: *Advances in Cryptology – EUROCRYPT 2019, Part I*. Vol. 11476. LNCS. Full version: IACR ePrint 2018/828. Springer, 2019, pp. 103–128. DOI: 10.1007/978-3-030-17653-2_4.

[BCS16] Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner. "Interactive Oracle Proofs". In: *Theory of Cryptography (TCC 2016-B)*. LNCS. Springer, 2016, pp. 31–60. DOI: 10.1007/978-3-662-53644-5_2.

[BCTV14] Eli Ben-Sasson, Alessandro Chiesa, Eran Tromer, and Madars Virza. "Scalable Zero Knowledge via Cycles of Elliptic Curves". In: *Advances in Cryptology – CRYPTO 2014*. LNCS. Springer, 2014, pp. 276–294. DOI: 10.1007/978-3-662-44381-1_16.

[BDH+25] Juraj Belohorec, Pavel Dvořák, Charlotte Hoffmann, Pavel Hubáček, Kristýna Mašková, and Martin Pastyřík. "On Extractability of the KZG Family of Polynomial Commitment Schemes". In: *Advances in Cryptology – CRYPTO 2025, Part VI*. Vol. 16005. LNCS. Springer, 2025, pp. 584–616. DOI: 10.1007/978-3-032-01887-8_19.

[BFK+24] Alexander R. Block, Zhiyong Fang, Jonathan Katz, Justin Thaler, Hendrik Waldner, and Yupeng Zhang. "Field-Agnostic SNARKs from Expand-Accumulate Codes". In: *Advances in Cryptology – CRYPTO 2024, Part X*. Vol. 14929. LNCS. Springer, 2024, pp. 276–307. DOI: 10.1007/978-3-031-68403-6_9.

[BFL90] László Babai, Lance Fortnow, and Carsten Lund. "Non-Deterministic Exponential Time Has Two-Prover Interactive Protocols". In: *Proceedings of the 31st Annual Symposium on Foundations of Computer Science (FOCS 1990)*. Journal version: Computational Complexity 1 (1991), 3–40. IEEE Computer Society, 1990, pp. 16–25. DOI: 10.1109/FSCS.1990.89520.

[BFN26] David Balbás, Dario Fiore, and Duy Nguyen. "Splitting Bilinear Groups: New Translations from Composite- to Prime-Order with Applications to Batch Arguments for NP". In: *Advances in Cryptology – CRYPTO 2026, Part IX*. Vol. 16808. LNCS. Springer, 2026, pp. 362–395. DOI: 10.1007/978-3-032-35424-2_13.

[BFRW26] Benedikt Bünz, Giacomo Fenzi, Ron D. Rothblum, and William Wang. "TensorSwitch: Nearly Optimal Polynomial Commitments from Tensor Codes". In: *Advances in Cryptology – CRYPTO 2026, Part IX*. Vol. 16808. LNCS. Springer, 2026, pp. 297–329. DOI: 10.1007/978-3-032-35424-2_11.

[BFS20] Benedikt Bünz, Ben Fisch, and Alan Szepieniec. "Transparent SNARKs from DARK Compilers". In: *Advances in Cryptology – EUROCRYPT 2020, Part I*. Vol. 12105. LNCS. IACR ePrint 2019/1229; revised security proof after a gap found by Block et al. (CRYPTO 2021). Springer, 2020, pp. 677–706. DOI: 10.1007/978-3-030-45721-1_24.

[BG12] Stephanie Bayer and Jens Groth. "Efficient Zero-Knowledge Argument for Correctness of a Shuffle". In: *Advances in Cryptology – EUROCRYPT 2012*. LNCS. Springer, 2012, pp. 263–280. DOI: 10.1007/978-3-642-29011-4_17.

[BGH19] Sean Bowe, Jack Grigg, and Daira Hopwood. *Recursive Proof Composition without a Trusted Setup*. IACR Cryptology ePrint Archive, Report 2019/1021. 2019.

[BGKW88] Michael Ben-Or, Shafi Goldwasser, Joe Kilian, and Avi Wigderson. “Multi-Prover Interactive Proofs: How to Remove Intractability Assumptions”. In: *Proceedings of the Twentieth Annual ACM Symposium on Theory of Computing (STOC ’88)*. ACM, 1988, pp. 113–131. DOI: 10.1145/62212.62223.

[BHK+26] Juraj Belohorec, Pavel Hubáček, Aleksi Kalsta, and Kristýna Mašková. “Chopin: Optimal Pairing-Based Multilinear Polynomial Commitments from Bivariate KZG”. In: *Advances in Cryptology – CRYPTO 2026, Part IX*. Vol. 16808. LNCS. Full version: IACR ePrint 2026/480. Springer, 2026, pp. 428–459. DOI: 10.1007/978-3-032-35424-2_15.

[BHR+21] Alexander R. Block, Justin Holmgren, Alon Rosen, Ron D. Rothblum, and Pratik Soni. “Time- and Space-Efficient Arguments from Groups of Unknown Order”. In: *Advances in Cryptology – CRYPTO 2021, Part IV*. Vol. 12828. LNCS. cited for finding the gap in the DARK (BFS20) security proof, as reported in BFS20’s revised abstract. Springer, 2021, pp. 123–152. DOI: 10.1007/978-3-030-84259-8_5.

[BMMS26] Anubhav Baweja, Pratyush Mishra, Tushar Mopuri, and Matan Shtepel. “FICS and FACS: Fast IOPPs and Accumulation via Code-Switching”. In: *Advances in Cryptology – CRYPTO 2026, Part IX*. Vol. 16808. LNCS. Springer, 2026, pp. 557–591. DOI: 10.1007/978-3-032-35424-2_19.

[BMNW25] Benedikt Bünz, Pratyush Mishra, Wilson Nguyen, and William Wang. “ARC: Accumulation for Reed–Solomon Codes”. In: *Advances in Cryptology – CRYPTO 2025, Part VII*. Vol. 16006. LNCS. Springer, 2025, pp. 128–160. DOI: 10.1007/978-3-032-01907-3_5.

[BMSS25] Benedikt Bünz, Tushar Mopuri, Alireza Shirzad, and Sriram Sridhar. “DewTwo: A Transparent PCS with Quasi-Linear Prover, Logarithmic Verifier and 4.5KB Proofs from Falsifiable Assumptions”. In: *Advances in Cryptology – CRYPTO 2025, Part VI*. Vol. 16005. LNCS. Springer, 2025, pp. 549–583. DOI: 10.1007/978-3-032-01887-8_18.

[BR93] Mihir Bellare and Phillip Rogaway. “Random Oracles are Practical: A Paradigm for Designing Efficient Protocols”. In: *Proceedings of the 1st ACM Conference on Computer and Communications Security (CCS 1993)*. 1993, pp. 62–73. DOI: 10.1145/168588.168596.

[BS23] Dan Boneh and Victor Shoup. *A Graduate Course in Applied Cryptography*. Free online draft, Version 0.6, January 2023. 2023.

[BS23b] Ward Beullens and Gregor Seiler. “LaBRADOR: Compact Proofs for R1CS from Module-SIS”. In: *Advances in Cryptology – CRYPTO 2023, Part V*. Vol. 14085. LNCS. Springer, 2023, pp. 518–548. DOI: 10.1007/978-3-031-38554-4_17.

[CBBZ23] Binyi Chen, Benedikt Bünz, Dan Boneh, and Zhenfei Zhang. “HyperPlonk: Plonk with Linear-Time Prover and High-Degree Custom Gates”. In: *Advances in Cryptology – EUROCRYPT 2023*. LNCS. Full version: IACR ePrint 2022/1355. Springer, 2023, pp. 499–530. DOI: 10.1007/978-3-031-30617-4_17.

[CCH+19] Ran Canetti, Yilei Chen, Justin Holmgren, Alex Lombardi, Guy N. Rothblum, Ron D. Rothblum, and Daniel Wichs. “Fiat-Shamir: From Practice to Theory”. In: *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing (STOC 2019)*. 2019, pp. 1082–1090. DOI: 10.1145/3313276.3316380.

[CCN24] Maria Corte-Real Santos, Craig Costello, and Michael Naehrig. "On Cycles of Pairing-Friendly Abelian Varieties". In: *Advances in Cryptology – CRYPTO 2024, Part IX*. Vol. 14928. LNCS. Springer, 2024, pp. 221–253. DOI: 10.1007/978-3-031-68400-5_7.

[CET+24] Stefanos Chaliasos, Jens Ernstberger, David Theodore, David Wong, Mohammad Jahanara, and Benjamin Livshits. "SoK: What Don't We Know? Understanding Security Vulnerabilities in SNARKs". In: *33rd USENIX Security Symposium (USENIX Security 24)*. arXiv:2402.15293. USENIX Association, 2024, pp. 3855–3872.

[CGH04] Ran Canetti, Oded Goldreich, and Shai Halevi. "The Random Oracle Methodology, Revisited". In: *Journal of the ACM* 51.4 (2004), pp. 557–594. DOI: 10.1145/1008731.1008734.

[CGKS23] Matteo Campanelli, Chaya Ganesh, Hamidreza Khoshakhlagh, and Janno Siim. "Impossibilities in Succinct Arguments: Black-Box Extraction and More". In: *Progress in Cryptology – AFRICACRYPT 2023*. Vol. 14064. LNCS. Cited version: revised full version, Cryptology ePrint Archive, Paper 2022/638 (2024). Springer, 2023, pp. 465–489. DOI: 10.1007/978-3-031-37679-5_20.

[CGKY26] Alessandro Chiesa, Ziyi Guan, Christian Knabenhans, and Zihan Yu. "On the Fiat–Shamir Security of Succinct Arguments from Functional Commitments". In: *Advances in Cryptology – CRYPTO 2026, Part IX*. Vol. 16808. LNCS. Springer, 2026, pp. 189–213. DOI: 10.1007/978-3-032-35424-2_7.

[CHM+20] Alessandro Chiesa, Yuncong Hu, Mary Maller, Pratyush Mishra, Psi Vesely, and Nicholas Ward. "Marlin: Preprocessing zkSNARKs with Universal and Updatable SRS". In: *Advances in Cryptology – EUROCRYPT 2020, Part I*. Vol. 12105. LNCS. IACR ePrint 2019/1047. Springer, 2020, pp. 738–768. DOI: 10.1007/978-3-030-45721-1_26.

[CMNW24] Valerio Cini, Giulio Malavolta, Ngoc Khanh Nguyen, and Hoeteck Wee. "Polynomial Commitments from Lattices: Post-quantum Security, Fast Verification and Transparent Setup". In: *Advances in Cryptology – CRYPTO 2024, Part X*. Vol. 14929. LNCS. Springer, 2024, pp. 207–242. DOI: 10.1007/978-3-031-68403-6_7.

[CMS19] Alessandro Chiesa, Peter Manohar, and Nicholas Spooner. "Succinct Arguments in the Quantum Random Oracle Model". In: *Theory of Cryptography – TCC 2019, Part II*. Vol. 11892. LNCS. Cited version: Cryptology ePrint Archive, Paper 2019/834 (version of 2020-01-14), whose page numbers differ from the published version. Springer, 2019, pp. 1–29. DOI: 10.1007/978-3-030-36033-7_1.

[COS20] Alessandro Chiesa, Dev Ojha, and Nicholas Spooner. "Fractal: Post-Quantum and Transparent Recursive Proofs from Holography". In: *Advances in Cryptology – EUROCRYPT 2020, Part I*. Vol. 12105. LNCS. IACR ePrint 2019/1076. Springer, 2020, pp. 769–793. DOI: 10.1007/978-3-030-45721-1_27.

[CS26] Elizabeth Crites and Alistair Stewart. "On Reed–Solomon Proximity Gaps Conjectures". In: *Advances in Cryptology – CRYPTO 2026, Part IV*. LNCS. note: crites-et-al-2026-proximity-gaps-conjectures. Springer, 2026, pp. 128–156. DOI: 10.1007/978-3-032-35398-6_5.

[DG26] Marian Dietz and Emanuele Giunta. “Bulletproofs are Optimal: Lower Bounds for Vector Commitments from Fiat-Shamir in Pairing-Free Groups”. In: *Advances in Cryptology – CRYPTO 2026, Part I*. Vol. 16800. LNCS. Springer, 2026, pp. 218–249. DOI: 10.1007/978-3-032-35367-2_8.

[DJJ+25] Pratish Datta, Abhishek Jain, Zhengzhong Jin, Alexis Korb, Surya Mathialagan, and Amit Sahai. “Incrementally Verifiable Computation for NP from Standard Assumptions”. In: *Advances in Cryptology – CRYPTO 2025, Part VII*. Vol. 16006. LNCS. Springer, 2025, pp. 611–642. DOI: 10.1007/978-3-032-01907-3_20.

[DMWG23] Quang Dao, Jim Miller, Opal Wright, and Paul Grubbs. “Weak Fiat-Shamir Attacks on Modern Proof Systems”. In: *2023 IEEE Symposium on Security and Privacy (SP)*. IACR ePrint 2023/691. IEEE, 2023, pp. 199–216. DOI: 10.1109/SP46215.2023.10179408.

[DP25] Benjamin E. Diamond and Jim Posen. “Succinct Arguments over Towers of Binary Fields”. In: *Advances in Cryptology – EUROCRYPT 2025*. LNCS. Full version: IACR ePrint 2023/1784. Springer, 2025, pp. 93–122. DOI: 10.1007/978-3-031-91134-7_4.

[DP26] Benjamin E. Diamond and Jim Posen. “Polylogarithmic Proofs for Multilinears over Binary Towers”. In: *Advances in Cryptology – EUROCRYPT 2026*. LNCS. Cited version: Cryptology ePrint Archive, Paper 2024/504 (version of 2025-09-22). Springer, 2026, pp. 3–32. DOI: 10.1007/978-3-032-25336-1_1.

[EFG22] Liam Eagen, Dario Fiore, and Ariel Gabizon. *cq: Cached quotients for fast lookups*. Cryptology ePrint Archive, Paper 2022/1763. Version of 2024-07-05. 2022.

[EG23] Liam Eagen and Ariel Gabizon. *Protogalaxy: Efficient Protostar-style folding of multiple instances*. Cryptology ePrint Archive, Paper 2023/1106. Version of January 29, 2024. 2023.

[FR26] Antonio Faonio and Luigi Russo. “Sumcheck-Based zkSNARKs are Non-malleable”. In: *Advances in Cryptology – CRYPTO 2026, Part IX*. Vol. 16808. LNCS. Springer, 2026, pp. 35–61. DOI: 10.1007/978-3-032-35424-2_2.

[FS86] Amos Fiat and Adi Shamir. “How To Prove Yourself: Practical Solutions to Identification and Signature Problems”. In: *Advances in Cryptology – CRYPTO 1986*. LNCS. Springer, 1986, pp. 186–194. DOI: 10.1007/3-540-47721-7_12.

[FW26] Cody Freitag and Daniel Wichs. “Unique SNARGs with Adaptive Security: Constructions and Black-Box Separations”. In: *Advances in Cryptology – CRYPTO 2026, Part I*. Vol. 16800. LNCS. Springer, 2026, pp. 190–217. DOI: 10.1007/978-3-032-35367-2_7.

[GGPR13] Rosario Gennaro, Craig Gentry, Bryan Parno, and Mariana Raykova. “Quadratic Span Programs and Succinct NIZKs without PCPs”. In: *Advances in Cryptology – EUROCRYPT 2013*. Vol. 7881. LNCS. Springer, 2013, pp. 626–645. DOI: 10.1007/978-3-642-38348-9_37.

[GKM+18] Jens Groth, Markulf Kohlweiss, Mary Maller, Sarah Meiklejohn, and Ian Miers. “Updatable and Universal Common Reference Strings with Applications to zk-SNARKs”. In: *Advances in Cryptology – CRYPTO 2018, Part III*. Vol. 10993. LNCS. Springer, 2018, pp. 698–728. DOI: 10.1007/978-3-319-96878-0_24.

[GKRRS21] Lorenzo Grassi, Dmitry Khovratovich, Christian Rechberger, Arnab Roy, and Markus Schofnegger. "Poseidon: A New Hash Function for Zero-Knowledge Proof Systems". In: *30th USENIX Security Symposium (USENIX Security 21)*. Cited version: updated version of Cryptology ePrint Archive, Report 2019/458 (version of 2023-07-05). USENIX Association, 2021, pp. 519–535.

[GKS23] Lorenzo Grassi, Dmitry Khovratovich, and Markus Schofnegger. "Poseidon2: A Faster Version of the Poseidon Hash Function". In: *Progress in Cryptology – AFRICACRYPT 2023*. Vol. 14064. LNCS. Cited version: Cryptology ePrint Archive, Paper 2023/323 (version of 2024-02-08). Springer, 2023, pp. 177–203. DOI: 10.1007/978-3-031-37679-5_8.

[GLS+23] Alexander Golovnev, Jonathan Lee, Srinath Setty, Justin Thaler, and Riad S. Wahby. "Brakedown: Linear-Time and Field-Agnostic SNARKs for R1CS". In: *Advances in Cryptology – CRYPTO 2023, Part II*. Vol. 14082. LNCS. Full version: IACR ePrint 2021/1043. Springer, 2023, pp. 193–226. DOI: 10.1007/978-3-031-38545-2_7.

[GMN22] Nicolas Gailly, Mary Maller, and Anca Nitulescu. "SnarkPack: Practical SNARK Aggregation". In: *Financial Cryptography and Data Security (FC 2022)*. LNCS. Cited version: Cryptology ePrint Archive, Report 2021/529 (version of 2021-09-06). Springer, 2022, pp. 203–229. DOI: 10.1007/978-3-031-18283-9_10.

[GMR85] S. Goldwasser, S. Micali, and C. Rackoff. "The Knowledge Complexity of Interactive Proof-Systems". In: *Proceedings of the Seventeenth Annual ACM Symposium on Theory of Computing (STOC '85)*. ACM, 1985, pp. 291–304. DOI: 10.1145/22145.22178.

[GNR26] Kobi Gurkan, Andrija Novakovic, and Ron D. Rothblum. "Bolt: Faster SNARKs from Sketched Codes". In: *Advances in Cryptology – CRYPTO 2026, Part IX*. Vol. 16808. LNCS. Springer, 2026, pp. 396–427. DOI: 10.1007/978-3-032-35424-2_14.

[Gro16] Jens Groth. "On the Size of Pairing-based Non-interactive Arguments". In: *Advances in Cryptology – EUROCRYPT 2016*. Springer, 2016. DOI: 10.1007/978-3-662-49896-5_11.

[GW11] Craig Gentry and Daniel Wichs. "Separating Succinct Non-Interactive Arguments from All Falsifiable Assumptions". In: *Proceedings of the 43rd Annual ACM Symposium on Theory of Computing (STOC '11)*. Full version: IACR ePrint 2010/610. ACM, 2011, pp. 99–108. DOI: 10.1145/1993636.1993651.

[GW20] Ariel Gabizon and Zachary J. Williamson. *plookup: A simplified polynomial protocol for lookup tables*. Cryptology ePrint Archive, Paper 2020/315. Version of November 20, 2020. 2020.

[GWC19] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. *PlonK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge*. IACR Cryptology ePrint Archive, Report 2019/953. 2019.

[Hab22] Ulrich Haböck. *Multivariate lookups based on logarithmic derivatives*. Cryptology ePrint Archive, Paper 2022/1530. Version of 2023-03-31. 2022.

[HLP24] Ulrich Haböck, David Levit, and Shahar Papini. *Circle STARKs*. Cryptology ePrint Archive, Paper 2024/278. Version dated 2024-12-17. 2024.

[HN23] Mathias Hall-Andersen and Jesper Buus Nielsen. "On Valiant's Conjecture: Impossibility of Incrementally Verifiable Computation from Random Oracles". In: *Advances in Cryptology – EUROCRYPT 2023, Part II*. Vol. 14005. LNCS. Cited version: Cryptology ePrint Archive, Paper 2022/542. Springer, 2023, pp. 438–469. DOI: 10.1007/978-3-031-30617-4_15.

[IKO07] Yuval Ishai, Eyal Kushilevitz, and Rafail Ostrovsky. "Efficient Arguments without Short PCPs". In: *Twenty-Second Annual IEEE Conference on Computational Complexity (CCC 2007)*. IEEE Computer Society, 2007, pp. 278–291. DOI: 10.1109/CCC.2007.10.

[JLX+25] Yuhao Jia, Songsong Li, Chaoping Xing, Yizhou Yao, and Chen Yuan. "Polynomial Commitments for Galois Rings and Applications to SNARKs Over $\mathbb{Z}_{2^k}$". In: *Advances in Cryptology – CRYPTO 2025, Part VI*. Vol. 16005. LNCS. Springer, 2025, pp. 515–548. DOI: 10.1007/978-3-032-01887-8_17.

[JMW26] Abhishek Jain, Surya Mathialagan, and Brent Waters. "Incrementally Verifiable Computation Without Extraction". In: *Advances in Cryptology – CRYPTO 2026, Part IX*. Vol. 16808. LNCS. Springer, 2026, pp. 3–34. DOI: 10.1007/978-3-032-35424-2_1.

[Kil92] Joe Kilian. "A Note on Efficient Zero-Knowledge Proofs and Arguments". In: *Proceedings of the 24th Annual ACM Symposium on Theory of Computing (STOC '92)*. ACM, 1992, pp. 723–732. DOI: 10.1145/129712.129782.

[KP23] Abhiram Kothapalli and Bryan Parno. "Algebraic Reductions of Knowledge". In: *Advances in Cryptology – CRYPTO 2023, Part IV*. Vol. 14084. LNCS. Springer, 2023, pp. 669–701. DOI: 10.1007/978-3-031-38551-3_21.

[KPS26] Shubham Khurana, Sahadeo Padhye, and Rajeev Anand Sahu. "Zero-Knowledge Extension of PARI". In: *IACR Communications in Cryptology* 3.2 (2026). DOI: 10.62056/ahjb0l2hd.

[KRS25] Dmitry Khovratovich, Ron D. Rothblum, and Lev Soukhanov. "How to Prove False Statements: Practical Attacks on Fiat-Shamir". In: *Advances in Cryptology – CRYPTO 2025, Part VI*. Vol. 16005. LNCS. Full version: IACR ePrint 2025/118. Springer, 2025, pp. 3–26. DOI: 10.1007/978-3-032-01887-8_1.

[KS22] Abhiram Kothapalli and Srinath Setty. *SuperNova: Proving Universal Machine Executions Without Universal Circuits*. IACR Cryptology ePrint Archive, Paper 2022/1758. 2022.

[KS24] Abhiram Kothapalli and Srinath Setty. "HyperNova: Recursive Arguments for Customizable Constraint Systems". In: *Advances in Cryptology – CRYPTO 2024, Part X*. Vol. 14929. LNCS. Full version: IACR ePrint 2023/573. Springer, 2024, pp. 345–379. DOI: 10.1007/978-3-031-68403-6_11.

[KS26] Abhiram Kothapalli and Srinath Setty. "NeutronNova: Group-Based Folding Done Right". In: *Advances in Cryptology – CRYPTO 2026, Part IX*. Vol. 16808. LNCS. Extended version: IACR ePrint 2024/1606. Springer, 2026, pp. 524–556. DOI: 10.1007/978-3-032-35424-2_18.

[KST22] Abhiram Kothapalli, Srinath Setty, and Ioanna Tzialla. "Nova: Recursive Zero-Knowledge Arguments from Folding Schemes". In: *Advances in Cryptology – CRYPTO 2022, Part IV*. Vol. 13510. LNCS. IACR ePrint 2021/370. Springer, 2022, pp. 359–388. DOI: 10.1007/978-3-031-15985-5_13.

[KZG10] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. *Polynomial Commitments*. Tech. rep. 2010-10. Extended version of "Constant-Size Commitments to Polynomials and Their Applications", ASIACRYPT 2010, LNCS 6477, pp. 177–194, doi:10.1007/978-3-642-17373-8_11. Centre for Applied Cryptographic Research (CACR), 2010.

[Lee21] Jonathan Lee. "Dory: Efficient, Transparent Arguments for Generalised Inner Products and Polynomial Commitments". In: *Theory of Cryptography Conference (TCC 2021), Part II*. Vol. 13043. LNCS. IACR ePrint 2020/1274. Springer, 2021, pp. 1–34. DOI: 10.1007/978-3-030-90453-1_1.

[LHW+25] Junkai Liang, Daqi Hu, Pengfei Wu, Yunbo Yang, Qingni Shen, and Zhonghai Wu. "SoK: Understanding zk-SNARKs: The Gap Between Research and Practice". In: *34th USENIX Security Symposium (USENIX Security 25)*. Full version: IACR ePrint 2025/172; arXiv:2502.02387. USENIX Association, 2025.

[Lip24] Helger Lipmaa. "Polymath: Groth16 Is Not the Limit". In: *Advances in Cryptology – CRYPTO 2024, Part X*. Vol. 14929. LNCS. Springer, 2024, pp. 170–206. DOI: 10.1007/978-3-031-68403-6_6.

[LPS25] Helger Lipmaa, Roberto Parisella, and Janno Siim. "On Knowledge-Soundness of Plonk in ROM from Falsifiable Assumptions". In: *Advances in Cryptology – CRYPTO 2025, Part VII*. Vol. 16006. LNCS. Full version: IACR ePrint 2024/994. Springer, 2025, pp. 362–395. DOI: 10.1007/978-3-032-01907-3_12.

[MBKM19] Mary Maller, Sean Bowe, Markulf Kohlweiss, and Sarah Meiklejohn. "Sonic: Zero-Knowledge SNARKs from Linear-Size Universal and Updatable Structured Reference Strings". In: *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS '19)*. ACM, 2019. DOI: 10.1145/3319535.3339817.

[Mer87] Ralph C. Merkle. "A Digital Signature Based on a Conventional Encryption Function". In: *Advances in Cryptology – CRYPTO 1987*. LNCS. Springer, 1987, pp. 369–378. DOI: 10.1007/3-540-48184-2_32.

[Mic94] Silvio Micali. "CS Proofs". In: *Proceedings of the 35th Annual IEEE Symposium on Foundations of Computer Science (FOCS '94)*. IEEE, 1994, pp. 436–453. DOI: 10.1109/SFCS.1994.365746.

[MPV24] Surya Mathialagan, Spencer Peters, and Vinod Vaikuntanathan. "Adaptively Sound Zero-Knowledge SNARKs for UP". In: *Advances in Cryptology – CRYPTO 2024, Part X*. Vol. 14929. LNCS. Springer, 2024, pp. 38–71. DOI: 10.1007/978-3-031-68403-6_2.

[NA25] Andrija Novakovic and Guillermo Angeris. *Ligerito: A Small and Concretely Fast Polynomial Commitment Scheme*. Cryptology ePrint Archive, Paper 2025/1187. Version of May 2025. 2025.

[NBS23] Wilson Nguyen, Dan Boneh, and Srinath Setty. "Revisiting the Nova Proof System on a Cycle of Curves". In: *5th Conference on Advances in Financial Technologies (AFT 2023)*. Vol. 282. LIPIcs. Full version: IACR ePrint 2023/969. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023, 18:1–18:22. DOI: 10.4230/LIPIcs.AFT.2023.18.

[NDC+24] Wilson Nguyen, Trisha Datta, Binyi Chen, Nirvan Tyagi, and Dan Boneh. "Mangrove: A Scalable Framework for Folding-Based SNARKs". In: *Advances in Cryptology – CRYPTO 2024, Part X*. Vol. 14929. LNCS. Springer, 2024, pp. 308–344. DOI: 10.1007/978-3-031-68403-6_10.

[Nit20] Anca Nitulescu. *zk-SNARKs: A Gentle Introduction*. Manuscript. 2020.

[NS24] Ngoc Khanh Nguyen and Gregor Seiler. "Greyhound: Fast Polynomial Commitments from Lattices". In: *Advances in Cryptology – CRYPTO 2024, Part X*. Vol. 14929. LNCS. Full version: IACR ePrint 2024/1293. Springer, 2024, pp. 243–275. DOI: 10.1007/978-3-031-68403-6_8.

[NS26] Wilson Nguyen and Srinath Setty. "Neo and SuperNeo: Post-quantum Folding with Pay-per-Bit Costs over Small Fields". In: *Advances in Cryptology – CRYPTO 2026, Part IX*. Vol. 16808. LNCS. Springer, 2026, pp. 492–523. DOI: 10.1007/978-3-032-35424-2_17.

[Ped91] Torben Pryds Pedersen. "Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing". In: *Advances in Cryptology – CRYPTO 1991*. LNCS. Springer, 1991, pp. 129–140. DOI: 10.1007/3-540-46766-1_9.

[PH23] Shahar Papini and Ulrich Haböck. *Improving logarithmic derivative lookups using GKR*. Cryptology ePrint Archive, Paper 2023/1284. Version of 2023-09-18. 2023.

[PHGR13] Bryan Parno, Jon Howell, Craig Gentry, and Mariana Raykova. "Pinocchio: Nearly Practical Verifiable Computation". In: *2013 IEEE Symposium on Security and Privacy*. IEEE, 2013, pp. 238–252. DOI: 10.1109/SP.2013.47.

[Plonky3] Polygon Zero Team. *Plonky3: A Toolkit for Polynomial IOPs*. https://github.com/Plonky3/Plonky3. Software repository.

[Pol22] Polygon Zero Team. *Plonky2: Fast Recursive Arguments with PLONK and FRI*. Whitepaper (draft), https://github.com/0xPolygonZero/plonky2. Dated 2022-09-07; not peer-reviewed. 2022.

[PTX26] Dimitrios Papadopoulos, Qiang Tang, and Jiajun Xin. "Morphic Accumulators and Applications". In: *Advances in Cryptology – CRYPTO 2026, Part IX*. Vol. 16808. LNCS. Springer, 2026, pp. 460–491. DOI: 10.1007/978-3-032-35424-2_16.

[QGX+26] Wenjie Qu, Yanpei Guo, Zhen Xuan, Xuanming Liu, and Jiaheng Zhang. "Celer: A Lookup Argument for Large-Scale Queries". In: *Advances in Cryptology – CRYPTO 2026, Part IX*. Vol. 16808. LNCS. Springer, 2026, pp. 243–266. DOI: 10.1007/978-3-032-35424-2_9.

[RRR16] Omer Reingold, Guy N. Rothblum, and Ron D. Rothblum. "Constant-Round Interactive Proofs for Delegating Computation". In: *Proceedings of the 48th Annual ACM Symposium on Theory of Computing (STOC 2016)*. 2016, pp. 49–62. DOI: 10.1145/2897518.2897652.

[RT25] Lior Rotem and Stefano Tessaro. "Straight-Line Knowledge Extraction for Multi-Round Protocols". In: *Advances in Cryptology – CRYPTO 2025, Part VII*. Vol. 16006. LNCS. Springer, 2025, pp. 95–127. DOI: 10.1007/978-3-032-01907-3_4.

[SAKK25] Nojan Sheybani, Anees Ahmed, Michel Kinsy, and Farinaz Koushanfar. *Zero-Knowledge Proof Frameworks: A Systematic Survey*. arXiv:2502.07063. 2025.

[Sch80] J. T. Schwartz. "Fast Probabilistic Algorithms for Verification of Polynomial Identities". In: *Journal of the ACM* 27.4 (1980), pp. 701–717. DOI: 10.1145/322217.322225.

[Set20] Srinath Setty. "Spartan: Efficient and General-Purpose zkSNARKs Without Trusted Setup". In: *Advances in Cryptology – CRYPTO 2020, Part III*. Vol. 12172. LNCS. IACR ePrint 2019/550. Springer, 2020, pp. 704–737. DOI: 10.1007/978-3-030-56877-1_25.

[STW23] Srinath Setty, Justin Thaler, and Riad Wahby. *Customizable Constraint Systems for Succinct Arguments*. Cryptology ePrint Archive, Paper 2023/552. Preprint. 2023.

[STW24] Srinath Setty, Justin Thaler, and Riad Wahby. "Unlocking the Lookup Singularity with Lasso". In: *Advances in Cryptology – EUROCRYPT 2024, Part VI*. Vol. 14656. LNCS. Full version: IACR ePrint 2023/1216. Springer, 2024, pp. 180–209. DOI: 10.1007/978-3-031-58751-1_7.

[STZ26] Srinath Setty, Justin Thaler, and Michael Zhu. "Twist and Shout: Faster Memory Checking Arguments via One-Hot Addressing and Increments". In: *Advances in Cryptology – CRYPTO 2026, Part IX*. Vol. 16808. LNCS. Springer, 2026, pp. 267–296. DOI: 10.1007/978-3-032-35424-2_10.

[Tha22] Justin Thaler. "Proofs, Arguments, and Zero-Knowledge". In: *Foundations and Trends in Privacy and Security* 4.2–4 (2022), pp. 117–660. DOI: 10.1561/3300000030.

[Val08] Paul Valiant. "Incrementally Verifiable Computation or Proofs of Knowledge Imply Time/Space Efficiency". In: *Theory of Cryptography – Fifth Theory of Cryptography Conference, TCC 2008*. Vol. 4948. LNCS. Springer, 2008, pp. 1–18. DOI: 10.1007/978-3-540-78524-8_1.

[WTS+18] Riad S. Wahby, Ioanna Tzialla, abhi shelat abhi, Justin Thaler, and Michael Walfish. "Doubly-Efficient zkSNARKs Without Trusted Setup". In: *2018 IEEE Symposium on Security and Privacy (SP)*. IACR ePrint 2017/1132. IEEE, 2018, pp. 926–943. DOI: 10.1109/SP.2018.00060.

[WW23] Hoeteck Wee and David J. Wu. "Lattice-Based Functional Commitments: Fast Verification and Cryptanalysis". In: *Advances in Cryptology – ASIACRYPT 2023*. Vol. 14442. LNCS. Cited version: Cryptology ePrint Archive, Paper 2024/028. Springer, 2023, pp. 201–235. DOI: 10.1007/978-981-99-8733-7_7.

[WW25] Brent Waters and David J. Wu. "A Pure Indistinguishability Obfuscation Approach to Adaptively-Sound SNARGs for NP". In: *Advances in Cryptology – CRYPTO 2025, Part VII*. Vol. 16006. LNCS. Full version: Cryptology ePrint Archive, Paper 2024/933. Springer, 2025, pp. 292–326. DOI: 10.1007/978-3-032-01907-3_10.

[XZS22] Tiancheng Xie, Yupeng Zhang, and Dawn Song. "Orion: Zero Knowledge Proof with Linear Prover Time". In: *Advances in Cryptology – CRYPTO 2022, Part IV*. Vol. 13510. LNCS. Revised full version with errata: IACR ePrint 2022/1010 (major revision, last revised 2023-11-02). Springer, 2022, pp. 299–328. DOI: 10.1007/978-3-031-15985-5_11.

[XZZ+19] Tiancheng Xie, Jiaheng Zhang, Yupeng Zhang, Charalampos Papamanthou, and Dawn Song. "Libra: Succinct Zero-Knowledge Proofs with Optimal Prover Computation". In: *Advances in Cryptology – CRYPTO 2019*. Vol. 11694. LNCS. Springer, 2019, pp. 733–764. DOI: 10.1007/978-3-030-26954-8_24.

[YLZ+26] Yuanzhuo Yu, Mengling Liu, Yuncong Zhang, Tianyi Ma, Shi-Feng Sun, Man Ho Au, and Dawu Gu. "HyperFond: A Transparent and Post-Quantum Distributed SNARK with Polylogarithmic Communication". In: *ACM Asia Conference on Computer and Communications Security (ASIA CCS '26)*. ACM, 2026, pp. 869–885. DOI: 10.1145/3779208.3805973.

[ZCF24] Hadas Zeilberger, Binyi Chen, and Ben Fisch. "BaseFold: Efficient Field-Agnostic Polynomial Commitment Schemes from Foldable Codes". In: *Advances in Cryptology – CRYPTO 2024, Part X*. Vol. 14929. LNCS. Springer, 2024, pp. 138–169. DOI: 10.1007/978-3-031-68403-6_5.

[Zei26] Hadas Zeilberger. "Khatam: Proximity Gaps for Multilinear Evaluation for all Linear Codes". In: *Advances in Cryptology – CRYPTO 2026, Part IX*. Vol. 16808. LNCS. Springer, 2026, pp. 592–611. DOI: 10.1007/978-3-032-35424-2_20.

[Zip79] Richard Zippel. "Probabilistic Algorithms for Sparse Polynomials". In: *Symbolic and Algebraic Computation (EUROSAM 1979)*. LNCS. Springer, 1979, pp. 216–226. DOI: 10.1007/3-540-09519-5_73.

[ZXZS20] Jiaheng Zhang, Tiancheng Xie, Yupeng Zhang, and Dawn Song. "Transparent Polynomial Delegation and Its Applications to Zero Knowledge Proof". In: *2020 IEEE Symposium on Security and Privacy (SP)*. Full version: IACR ePrint 2019/1482. IEEE, 2020, pp. 859–876. DOI: 10.1109/SP40000.2020.00052.

A Supplementary Tables and Timeline

Table A.1: **Setup, assumptions and the costs as each paper reports them.** It gives the exact counts behind Tables 7.1 and A.2, in each paper's own units and symbols. E and P count exponentiations and pairings (with a subscript for the group), PPE counts pairing-product equations, and F mul or MUL counts field multiplications; unless a row says otherwise, m is the number of wires, n of multiplication gates, a of addition gates, ℓ of public inputs, d the circuit depth and N the number of constraints. Spartan's single row covers all variants (DL in Table A.2; KE and RO in Table 7.1) and is placed with the strict rows.

Scheme	Proof size	Prover	Verifier	Setup	Assumptions	PQ?	Arith. / PCS	Source
Strict zk-SNARKs (Table 7.1 rows)								
GGPR	QSP: 7 $\mathbb{G}$; QAP: 9 $\mathbb{G}$ (symmetric)	quasi-linear (linear group ops)	linear in statement size (computing $E(v_{in}(s))$), otherwise constant; constant apart from hashing if the statement is hashed	per-circuit, CRS linear in circuit; two stages: generic powers-of-s Gen, then per-function Regen_f (§3.1)	q-PDH + d-PKE, $q \geq \max\{2d-1, d+2\}$ (Thms. 7, 14; §5.1 calls it q-PKE; knowledge, non-falsifiable)	no	QSP/QAP / pairing	[GGPR13, abstract, §1.2, §3.1, §3.3, §5.1, §8]; [PHGR13, p. 4]
Pinocchio	8 $\mathbb{G}$ (symmetric); 288 B as implemented (asymmetric BN-256)	$7m + n - 2\ell$ E (Groth16's count; derived from Protocol 2: $7(m-N)+d$, N = number of I/O elements)	ℓ E (Groth16's count; derived from Protocol 2: about $3N$), 11P, 5 PPE	per-circuit	d-PKE, q-PDH, 2q-SDH ($q = 4d+4$)	no	QAP / pairing	[Gro16, Table 2] (symmetric); [PHGR13, Thm. 1, Protocol 2, Fig. 8]
Groth16	3 elts ($2\,\mathbb{G}_1 + 1\,\mathbb{G}_2$)	$m + 3n - \ell$ E_1, n E_2	ℓ E_1, 3P, 1 PPE	per-circuit	generic bilinear group model	no	QAP / pairing	[Gro16, Table 2] (asymmetric)
Sonic	succinct: $20\,\mathbb{G}_1 + 16\,\mathbb{F}$; helped: $4\,\mathbb{G}_1 + 2\,\mathbb{F}$ (PLONK) or $7\,\mathbb{G} + 5\,\mathbb{F}$ (Sonic Table 2)	succinct $273n$ $\mathbb{G}_1$ E; helped $18n$	13P (10P helped), counted unbatched as in Sonic Table 2; 7P with standard pairing batching (Marlin Fig. 1)	universal, updatable, linear SRS	AGM + ROM (interactive, made non-interactive by Fiat–Shamir in the ROM, §3 p. 4)	no	own constraint system / KZG variant (univariate Laurent, §6.2)	[GWC19, Tables 1–2]; [MBKM19, Table 2, §1, §3, §6.2]; [CHM+20, Fig. 1] (batched count)

continued on next page

Table A.1, continued

Scheme	Proof size	Prover	Verifier	Setup	Assumptions	PQ?	Arith. / PCS	Source
PLONK	$9\,\mathbb{G}_1 + 6\,\mathbb{F}$ (fast prover)	$9n + 9a$ $\mathbb{G}_1$ E $+ \approx 54(n+a)\log(n+a)$ F mul	2P, 18 $\mathbb{G}_1$ E (fast prover; 16 for the small variant)	universal and updatable (PLONK's "universal" includes updatability, §1); SRS degree = #gates	original: AGM + Q-DLOG (knowledge soundness under 2d-DLOG), ROM for Fiat–Shamir; alternatively, knowledge-sound in the ROM under the falsifiable ARSDH + SplitRSDH (Lipmaa–Parisella–Siim, CRYPTO 2025, via special soundness of interactive PLONK; re-derived modularly by Chiesa et al., CRYPTO 2026, Cor. 1)	no	Plonkish / KZG	[GWC19, Tables 1–2, §2.2, Lemma 4.7, §8]; [LPS25]; [CGKY26]
Marlin	$13\,\mathbb{G}_1 + 8\,\mathbb{F}$	22 v-MSM(m) (variable-base multi-scalar multiplications, MSMs, of size m), $O(m \log m)$	2P, $O(\lvert x\rvert + \log m)$	universal, updatable (Remark 7.1)	AGM (fallback: knowledge assumptions) + ROM (Fiat–Shamir, §1.1, §7)	no	R1CS / AHP + KZG	[CHM+20, Fig. 1, §7]
Fractal	$O_\lambda(\log^2 m)$	$O_\lambda(m \log m)$	$O_\lambda(\lvert x\rvert + \log^2 m)$ after preprocessing	transparent	ROM	plausibly (security proven in the QROM, Thm. 3)	R1CS / holographic IOP	[COS20, Thm. 3]
Spartan	$O(\log^2 n)$ to $O(\sqrt{n})$ by PCS	$O(n)$, time-optimal via Spark (DL, KE); $O(n \log n)$ (RO, CL; Fig. 2)	$O(\log^2 n)$ to $O(\sqrt{n})$ by PCS	transparent (DL, RO, and CL, which uses DARK's commitment scheme, Fig. 2 caption); universal trusted setup for KE ("private⋆", Fig. 2)	DL + ROM ($\text{Spartan}_{\text{DL}}$, implemented, §8); q-type knowledge-of-exponent PCS ($\text{Spartan}_{\text{KE}}$, §1); ROM only ($\text{Spartan}_{\text{RO}}$)	no (DL, KE, CL); plausibly (RO, Fig. 2 caption)	R1CS / sum-check + Spark	[Set20, abstract, §1, Fig. 2, §8]

continued on next page

Table A.1, continued

Scheme	Proof size	Prover	Verifier	Setup	Assumptions	PQ?	Arith. / PCS	Source
Supersonic (DARK)	$O(\mu \log d)$	$O(n \log n)$ MUL (quasi-linear)	$O(\mu \log d)$	transparent with class groups (trusted with RSA)	strong RSA + adaptive root (Thm. 5); revised 2022 proof: random-order + adaptive root	no	Sonic / PLONK / Marlin polynomial IOPs (10.1 KB headline = PLONK) / DARK PCS	[BFS20, §1.1, §3.1, Thm. 5, §6, Table 4]
HyperPlonk	$\mu + 5\ \mathbb{G}_1 + 4\mu + 29\ \mathbb{F}$ for $n = 2^\mu$ degree-2 gates (multilinear KZG, optimized as in App. C); $176\mu + 1168$ bytes as implemented (§6.2)	PIOP $O(nd \log^2 d)$ F (Thm. 4.1); KZG commitment + evaluation $n\ \mathbb{G}_1$ (Table 2); $O(n + d \log n)$ exponentiations for n gates of total degree d (§1, p. 5, which writes s for the gate count)	PIOP $O(\mu + \ell)$, ℓ = public-input length (Thm. 4.1); KZG opening $\log(n)$ P (Table 2); batching k evaluations $O(k\mu)$ (§3.8)	universal with multilinear KZG or Orion+; transparent with the FRI-based PCS (Table 2)	those of the multilinear PCS, through PIOP compilation with witness-extended emulation (Thm. 2.4), not restated; ROM for Fiat–Shamir (§2.3); zero knowledge needs an additive, spanning PCS (Cor. A.4)	no (KZG); "quantum-resistant" with the FRI-based PCS (§1, p. 5)	Plonkish on the Boolean hypercube / multilinear KZG (also Orion+ or FRI-based)	[CBBZ23, §1, §2.3, Table 2, Thm. 2.4, §3.8, Thm. 4.1, §6.2, Cor. A.4]
Polymath	$3\,\mathbb{G}_1 + 1\,\mathbb{F}$	$(\tilde{m} + 13\tilde{n})\ \mathbb{G}_1$ scalar mult. (SAP, $\tilde{m} \approx 2m$, $\tilde{n} \approx 2n$)	2P + 2 $\mathbb{G}_1$ + 1 $\mathbb{G}_2$ scalar mult. + $O(m_0 \log m_0)$ F, m_0 = input length	per-circuit SRS: $(\tilde{m} + 12\tilde{n})\ \mathbb{G}_1$ (Table 1)	special soundness of the interactive argument in the AGM with oblivious sampling, under PDL and TOFR (Thm. 3); knowledge soundness after Fiat–Shamir via Attema et al. (§4.7); perfect zero knowledge (Thm. 2)	no	SAP / KZG	[Lip24, Table 1, p. 175, Thm. 2, p. 188, Thm. 3, p. 193, §4.7, p. 199]
Outside the strict definition (Table A.2 rows)								
STARK	$O(\log^2 \lvert C \rvert)$	$O(\lvert C \rvert \log^2 \lvert C \rvert)$	$O(\log^2 \lvert C \rvert)$ for uniform computations (AIR; DARK §1.1); $O(\lvert C \rvert)$ for an arbitrary circuit (Aurora Fig. 1)	transparent	collision-resistant hashing (interactive protocol); ROM once Fiat–Shamir'd	plausibly (post-quantum claim covers the interactive setting only, §1.3.1 fn. 12)	AIR / FRI	[ZXZS20, Table 2]; [BBHR18, §2.2, Thm. 3.5, §1.3.1]

continued on next page

Table A.1, continued

Scheme	Proof size	Prover	Verifier	Setup	Assumptions	PQ?	Arith. / PCS	Source
Libra	$O(d \log\|C\|)$ for d-depth log-space uniform C	$O(\|C\|)$ (any circuit)	$O(\|x\| + d \log\|C\|)$ for d-depth log-space uniform C	one-time, input-size-only trusted setup	q-SBDH + (d, ℓ)-extended PKE (pairing-based zkVPD); ROM for non-interactivity	no	layered circuits (GKR) / zkVPD	[XZZ+19, abstract, §2, Thms. 5–6, §4.4]
Virgo	$O(d \log\|C\| + \log^2 n)$ (depth d, n inputs)	$O(\|C\| + n \log n)$	$O(\|x\| + d \log\|C\| + \log^2 n)$ (regular C)	transparent	hashing; "100+ bits" rests on FRI's unproven up-to-capacity soundness conjecture (§5.2; discussed in this survey's §5.3); the up-to-capacity correlated-agreement conjecture was disproved (Crites–Stewart, CRYPTO 2026 [CS26]), which closes one route to proving it (§5.3), and no attack on FRI is known; under proven FRI bounds these parameters appear to give fewer than 100 bits (our estimate)	plausibly	layered circuits (GKR) / FRI-based zkVPD	[ZXZS20, Thm. 3, Table 2]
Hyrax	$\approx 10d \log G + \|w\|^{1/\iota}$ $\mathbb{G}$	linear in $\|C\|$ if C has sufficient (data) parallelism; $O(d \log G + \|w\|)$ crypto ops	$O(\|x\| + d \log G + \|w\|^{(\iota-1)/\iota})$ with sufficient parallelism (sub-linear without it only for regular wiring, fn. 2); with a redistribution layer (RDL, §6.2) V pays an additional $O(\|x\| + \|w\| + NG)$ for N parallel sub-computations of width G, which dominates V on the Merkle-tree and image-scaling benchmarks (§8)	transparent	discrete log + ROM	no	layered circuits (GKR) / Hyrax-commit	[WTS+18, §1, §6.2, Thm. 6, §8]

continued on next page

Table A.1, continued

Scheme	Proof size	Prover	Verifier	Setup	Assumptions	PQ?	Arith. / PCS	Source
Aurora	$O(\log^2 N)$	$O(N \log N)$	$O(N)$	transparent	unconditional in ROM	plausibly	R1CS / univariate sum-check + FRI	[BCR+19, Thm. 1.2]
Ligero	$\tilde{O}(\sqrt{\lvert C\rvert})$	$O(\lvert C\rvert \log \lvert C\rvert)$	$O(\lvert C\rvert)$	transparent	collision-resistant hashing (interactive protocol); ROM once Fiat–Shamir'd (abstract, §1)	plausibly	arith. circuits / interleaved RS (IPCP)	[AHIV17, Thm. 1.1]; [ZXZS20, Table 2]
Bulletproofs	IPA: $2\lceil \log_2 n \rceil$ $\mathbb{G}$ + 2 $\mathbb{Z}_p$; circuit proof $2\log_2 n + 13$ elements (§5)	IPA: $\approx 8n$ E	IPA: $\approx 4n$ E (one MSM of size $2n + 2\log n$), linear	transparent	discrete log + ROM (Fiat–Shamir, §4.4)	no	arith. circuits / IPA	[BBB+17, §3, §4.4, §5]
Orion	$O(\log^2 N)$	$O(N)$ field operations and hashes	$O(N)$ (Thm. 5); $O(\log^2 N)$ claimed in the preprocessing setting (Table 1 note; §1, pp. 3–4)	transparent (no trusted setup, §1, p. 3)	collision-resistant hash, hiding for zero knowledge (§2.2); made non-interactive by the Fiat–Shamir heuristic (p. 23)	plausibly ("plausible post-quantum security", p. 23)	R1CS / sum-check + Orion-PC with code switching (Virgo's IOP as the inner argument, p. 23)	[XZS22, abstract, Table 1, §1, §2.2, Thm. 5, p. 23]
Plonky2	not stated	not stated	not stated	transparent; preprocessing commits to the constant and permutation polynomials in one Merkle tree (§7.1)	none proven: 100 bits of "conjectured" security, from ethSTARK's soundness conjecture (§8, fn. 8); soundness after Fiat–Shamir not analysed (§3.4.3, fn. 3); Poseidon sponge (§4)	not discussed (hash-based)	TurboPLONK-style Plonkish with custom gates / FRI over Goldilocks, challenges from $\mathbb{F}_p[X]/(X^2 - 7)$	[Pol22, abstract, §2.1, §3.4.3, §4, §7.1, §8]
Nova	IVC: $O(\lvert F\rvert)$ field elements (a relaxed-R1CS witness) plus $O(1)$ commitments; compressed $O(\log \lvert F\rvert)$	per step: 2 MSMs of size $O(\lvert F\rvert)$	$O_\lambda(\lvert F\rvert)$; compressed $O(\log \lvert F\rvert)$ or $O(\lvert F\rvert)$	transparent	additively homomorphic commitment (e.g. Pedersen / DLOG), curve cycle; ROM instantiated by a concrete hash (heuristic, fn. 1, §4.2)	no	relaxed R1CS / folding	[KST22, §1.2, Thms. 1–2]

continued on next page

Table A.1, continued

Scheme	Proof size	Prover	Verifier	Setup	Assumptions	PQ?	Arith. / PCS	Source
HyperNova	NIVC proof linear in the circuit of each supported function (§8, p. 35); compressed by a non-zero-knowledge SNARK of the user's choice (§8, p. 37)	per step: one MSM of size $O(n+n_j)+O((N+N_j)+(t+t_j)(m+m_j)+(q+q_j)(m+m_j)\,d\log^2 d)$ F (Thm. 3)	folding verifier $O(d\log m)$ F and $O(1)$ group operations (Thm. 1); verifier circuit with one $\mathbb{G}$ scalar multiplication (Thm. 3)	Pedersen key, as for Nova	Pedersen commitments (discrete log, App. E); ROM for Fiat–Shamir, heuristically instantiated with a concrete hash (§1.3, p. 9); zero knowledge honest-verifier, then the Fiat–Shamir heuristic (§8, p. 37)	no	CCS / multi-folding via sum-check	[KS24, §1.3, Thms. 1, 3, §8, App. E]

Table A.2: **Zero-knowledge arguments outside the strict definition (costs in $|C|$).** "Conditional" rows meet the bound only for a restricted class of circuits. Notation as in Table 7.1.

Scheme	Why outside the strict definition	Setup params (beyond C)	Proof size	Prover	Verifier	Notes	Source
STARK	conditional: polylog verifier only for uniform computations (AIR); linear for an arbitrary circuit	$O_\lambda(1)$	$O_\lambda(\log^2\|C\|)$	$O_\lambda(\|C\|\log^2\|C\|)$	$O_\lambda(\log^2\|C\|)$ or $O_\lambda(\|C\|)$	$\log^2$ verifier for uniform (AIR) computations, linear for an arbitrary circuit; hash only	[BBHR18, §2.2, Thm. 3.5]; [ZXZS20, Table 2]; [BCR+19, Fig. 1]
Libra	conditional: proof and verifier grow with depth d; polylog only for low-depth, log-space uniform circuits	$O_\lambda(n)$	$O_\lambda(d\log\|C\|)$	$O_\lambda(\|C\|)$	$O_\lambda(\|x\|+d\log\|C\|)$	n = input size; bounds hold for log-space uniform circuits	[XZZ+19, Table 1, §4, Thm. 6]
Virgo	conditional: as Libra (depth d; regular circuits)	$O_\lambda(1)$	$O_\lambda(d\log\|C\|+\log^2 n)$	$O_\lambda(\|C\|+n\log n)$	$O_\lambda(\|x\|+d\log\|C\|+\log^2 n)$	n = input size; regular (uniform) circuits; transparent, hash only	[ZXZS20, Thm. 3, Table 2]
Hyrax	proof and verifier sublinear, not polylog ($\|w\|^{1/\iota}$, $\|w\|^{(\iota-1)/\iota}$ terms); grow with depth d	$O_\lambda(\|w\|^{(\iota-1)/\iota})$	$O_\lambda(d\log G+\|w\|^{1/\iota})$	$O_\lambda(\|C\|)$	$O_\lambda(\|x\|+d\log G+\|w\|^{(\iota-1)/\iota})$	prover linear only with data parallelism; setup parameters **derived** from Lemma 5	[WTS+18, §1, §6, Lemma 5, p. 9]
$\text{Spartan}_{\text{DL}}$	proof and verifier $O_\lambda(\sqrt{\|C\|})$	$O_\lambda(\sqrt{\|C\|})$	$O_\lambda(\sqrt{\|C\|})$	$O_\lambda(\|C\|)$	$O_\lambda(\|x\|+\sqrt{\|C\|})$	setup parameters **derived**: Spartan's discrete-log commitment option is Hyrax-PC, whose key is about $\sqrt{\|C\|}$ random group elements	[Set20, Fig. 2, Fig. 4, p. 20]

continued on next page

Table A.2, continued

Scheme	**Why outside the strict definition**	**Setup params (beyond C)**	**Proof size**	**Prover**	**Verifier**	**Notes**	**Source**
Aurora	verifier linear in $\|C\|$ (proof is polylog)	$O_\lambda(1)$	$O_\lambda(\log^2\|C\|)$	$O_\lambda(\|C\|\log\|C\|)$	$O_\lambda(\|C\|)$	random oracle only	[BCR+19, Thm. 1.2]
Ligero	proof $\tilde{O}(\sqrt{\|C\|})$ and verifier linear	$O_\lambda(1)$	$O_\lambda(\sqrt{\|C\|})$	$O_\lambda(\|C\|\log\|C\|)$	$O_\lambda(\|C\|)$	proof $\tilde{O}(\sqrt{\|C\|})$; hash only	[AHIV17, Thm. 1.1]; [ZXZS20, Table 2]
Bulletproofs	verifier linear in $\|C\|$ (proof is logarithmic)	$O_\lambda(\|C\|)$	$O_\lambda(\log\|C\|)$	$O_\lambda(\|C\|)$	$O_\lambda(\|C\|)$	transparent generators; short proofs, linear verifier	[BBB+17, §3, §5]
Orion	proven verifier linear (Thm. 5); the $O_\lambda(\log^2\|C\|)$ verifier is claimed only in the preprocessing setting, and no theorem states it (§9.1)		$O_\lambda(\log^2\|C\|)$	$O_\lambda(\|C\|)$	$O_\lambda(\|C\|)$; claimed $O_\lambda(\log^2\|C\|)$ after preprocessing	N = size of the circuit or R1CS; no trusted setup, but a randomly sampled expander-based code whose description size is not stated	[XZS22, abstract, Table 1, §1, Thm. 5, p. 23]
Plonky2	a draft whitepaper (7 September 2022) with no asymptotic bounds and no full security proof; its 100 bits are "conjectured" and FRI's zero knowledge is marked "TODO"	$O_\lambda(1)$				PLONK-style arithmetization with FRI; setup parameters **derived**: hash only	[Pol22, p. 1, abstract, §3.4, §3.4.3 fn. 3, §3.6 fn. 6, §7.1, §8 fn. 8]
Nova (per step)	an IVC / folding scheme, not itself a zk-SNARK; per step, only the compressed proof in the SXDH variant meets the bound	$O_\lambda(\|F\|)$	IVC $O_\lambda(\|F\|)$; compressed $O_\lambda(\log\|F\|)$	$O_\lambda(\|F\|)$	IVC $O_\lambda(\|F\|)$; compressed $O_\lambda(\|F\|)$ (DLOG) or $O_\lambda(\log\|F\|)$ (SXDH)	$\|F\|$ **(step circuit) plays the role of** $\|C\|$; setup parameters **derived**: a Pedersen key for vectors of length about $\|F\|$	[KST22, §1, Fig. 2, §2, Thms. 1–2]
HyperNova (per step)	a non-uniform IVC / folding scheme, not itself a zk-SNARK; its NIVC proof is linear in the step circuits, and succinctness needs a separate SNARK that the paper does not fix	$O_\lambda(\|F\|)$	NIVC $O_\lambda(\|F\|)$; compressed: that of the chosen SNARK	$O_\lambda(\|F\|)$	NIVC $O_\lambda(\|F\|)$	$\|F\|$ **(step circuit) plays the role of** $\|C\|$; the NIVC proof is linear in each supported function's circuit; setup parameters **derived**: a Pedersen key	[KS24, abstract, Constr. 1, Thm. 3, §8]

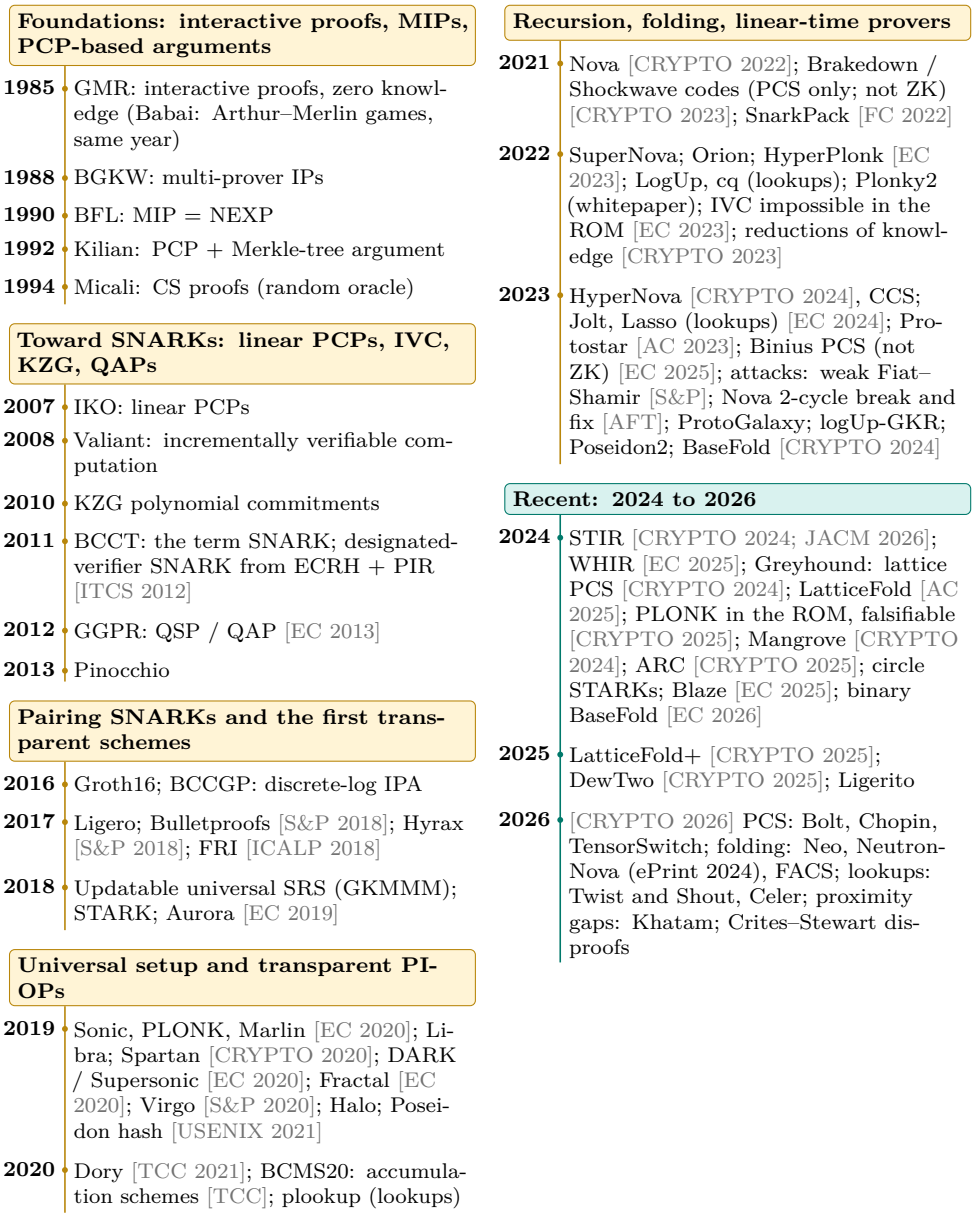


Figure 7: Construction timeline. The papers this survey draws on by year, from interactive proofs and zero knowledge (1985) through PCP-based arguments, the pairing/QAP era, the 2019 wave of universal and transparent SNARKs, and recursion, folding, lookups and linear-time provers, to a 2024–2026 band of recent work. Years are preprint years where these differ from the venue year (except that the CRYPTO 2026 papers are grouped under 2026); the venue is then given in brackets, and some same-year venues are bracketed too (EC = EUROCRYPT, AC = ASIACRYPT). Attacks and disproofs are included because §§5.3, 6.3 and 9.4 rely on them.